
Plaqueette Manual

Release 0.9.3

Feb 24, 2026

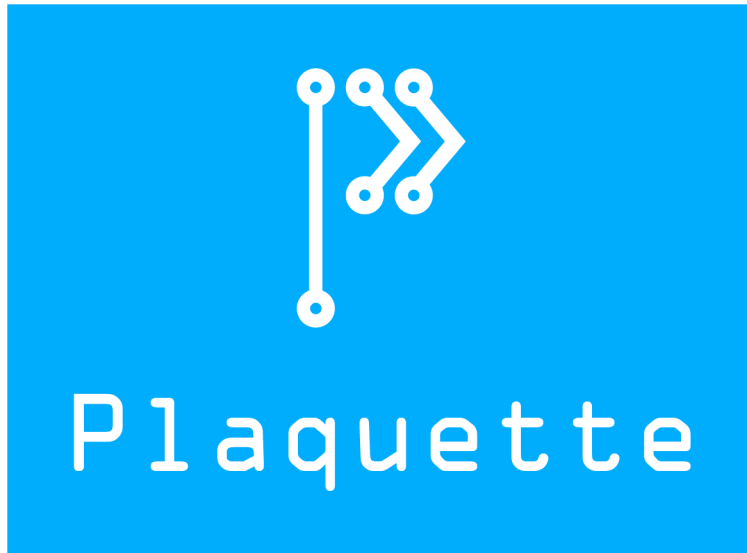
CONTENTS

1	Essentials	3
1.1	Why Plaqueette?	3
1.1.1	The Need for a New Standard	3
1.1.2	Meet Plaqueette	4
1.2	Features	5
1.2.1	Object-oriented	5
1.2.2	User-friendly	5
1.2.3	Signal-centric	5
1.2.4	Data-flow	6
1.2.5	Real-time	6
1.2.6	Arduino-compatible	7
1.3	Reference	8
1.3.1	Base Units	8
1.3.2	Generators	9
1.3.3	Timing	9
1.3.4	Filters	9
1.3.5	Communication	9
1.3.6	Fields	9
1.3.7	Functions	10
1.3.8	Structure	10
1.3.9	Extra	10
2	Guide	11
2.1	Getting started	11
2.1.1	Step 1: Install Plaqueette	11
2.1.2	Step 2: Your first Plaqueette program	11
2.1.3	Step 3 : Experiment!	13
2.1.4	Learning More	20
2.2	Inputs and Outputs	21
2.2.1	Digital vs Analog	22
2.2.2	Digital Inputs and Outputs	22
2.2.3	Analog Outputs and Inputs	24
2.2.4	Using Units as Their Own Values	26
2.2.5	The Flow Operator (>>)	26
2.2.6	Dealing with Noisy Signals: Debouncing and Smoothing	27
2.2.7	Mapping Values to Different Ranges	29
2.2.8	Making Decisions with Conditions	29
2.2.9	Modes for Inputs and Outputs	30
2.2.10	Conclusion	32
2.3	Generating Waveforms	32

2.3.1	Visualizing Waves with the Serial Plotter	33
2.3.2	Types of Waves	34
2.3.3	Wave Properties	35
2.3.4	Wave Addition	38
2.3.5	Modulation	39
2.3.6	Adding Noise with randomFloat()	39
2.3.7	Timing Functions	41
2.3.8	Phase Shifting with shiftBy()	41
2.3.9	Conclusion	42
2.4	Working with Time	42
2.4.1	Measuring Absolute Time with seconds()	43
2.4.2	Timing Units	44
2.4.3	Keeping Track of Time with Chronometer	44
2.4.4	Scheduling with Alarm	45
2.4.5	Triggering Periodic Events with Metronome	46
2.4.6	Creating Smooth Transitions with Ramp	47
2.4.7	Combining Timing Units	53
2.4.8	Conclusion	54
2.5	Regularizing Signals	54
2.5.1	Direct Input-to-Output	54
2.5.2	Getting the Full Range of a Signal	56
2.5.3	Handling Noisy or Unpredictable Signals	57
2.5.4	Reacting to Signal Changes	58
2.5.5	Adapting to Changing Conditions	59
2.5.6	Normalizing Signals to Spot Extreme Values	59
2.5.7	Choosing the Right Regularizer for the Job	60
2.5.8	Detecting Peaks	63
2.5.9	Conclusion	64
2.6	Managing Events	64
2.6.1	Supported Events	65
2.6.2	Using Trigger Conditions	65
2.6.3	Using Callback Functions	66
2.6.4	Managing Multiple Events	67
2.6.5	Coordinating Parallel Events with Metronomes	68
2.6.6	Creating On-the-fly Callbacks	69
2.6.7	Conclusion	69
2.7	Advanced Usage	69
2.7.1	Smoothing Arbitrary Signals	69
2.7.2	Vanilla Coding Style	70
2.7.3	Using Plaquette as an External Library	70
2.7.4	Synchronizing Groups of Units with Secondary Engines	71
2.8	Extending Plaquette	75
2.8.1	Architecture Overview	75
2.8.2	Flowable vs Unit: Choosing Your Base Class	77
2.8.3	Class Hierarchy	79
2.8.4	Unit Lifecycle	81
2.8.5	Step-Invariance Principle	84
2.8.6	Flow Operators	84
2.8.7	Creating Your First Unit	85
2.8.8	Unit Templates	89
2.8.9	Parameter Slots	90
2.8.10	Event System	91
2.8.11	Testing Your Unit	93
2.8.12	Coding Conventions	95

2.8.13	Summary	98
3	Reference	99
3.1	Base Units	99
3.1.1	AnalogIn	99
3.1.2	AnalogOut	102
3.1.3	DigitalIn	104
3.1.4	DigitalOut	108
3.2	Generators	111
3.2.1	Ramp	112
3.2.2	Wave	121
3.3	Timing	134
3.3.1	Alarm	134
3.3.2	Chronometer	139
3.3.3	Metronome	141
3.4	Filters	148
3.4.1	MinMaxScaler	148
3.4.2	Normalizer	152
3.4.3	PeakDetector	157
3.4.4	RobustScaler	162
3.4.5	Smoother	167
3.5	Communication	171
3.5.1	Monitor	171
3.5.2	Plotter	174
3.6	Fields	176
3.6.1	PivotField	176
3.6.2	TimeSliceField	181
3.7	Functions	184
3.7.1	mapFloat()	185
3.7.2	mapFrom01()	186
3.7.3	mapTo01()	187
3.7.4	randomFloat()	189
3.7.5	randomTrigger()	189
3.7.6	seconds()	191
3.7.7	wrap()	191
3.8	Structure	193
3.8.1	Engine	193
3.8.2	Value Units	197
3.8.3	prepare()	199
3.8.4	begin()	200
3.8.5	step()	201
3.8.6	[] (arrays)	202
3.8.7	. (dot)	203
3.8.8	>> (flow)	203
3.9	Extra	204
3.9.1	Easings	204
3.9.2	ContinuousServoOut	205
3.9.3	ServoOut	208
4	Libraries	211
5	Related Info	213
5.1	Community Guidelines	213
5.1.1	Be Respectful and Inclusive	213

5.1.2	Contributing	213
5.1.3	Reporting Bugs and Requesting Features	214
5.1.4	Getting Support	214
5.1.5	License and Attribution	214
5.2	Credits	214
5.2.1	Core Developers	214
5.2.2	Contributors	214
5.2.3	Partners	215
5.2.4	Funding	215
5.2.5	Inspiration	215
5.3	License	215
Index		217



Plaquette is an object-oriented, user-friendly, signal-centric programming framework for **creative physical computing**. It promotes **expressiveness** over technical details while remaining fully compatible with [Arduino](#), allowing **both novice and experienced** creative practitioners to **intuitively** design meaningful physical interactive systems.

Whether you are a beginner working with physical computing for the first time, an intermediate user familiar with creative signal-based softwares (eg. Max, Ossia Score, PureData, SuperCollider, TouchDesigner, etc.), or a seasoned Arduino creative coder: Plaquette is the perfect tool for your creative embedded and IoT projects.

Plaquette allows you to:

- React to multiple sensors and actuators in real-time without interruption.
- Automatically calibrate sensors to design stable interactions in changing environments.
- Design rich interactive behaviors by seamlessly combining powerful effects.

Quick links:

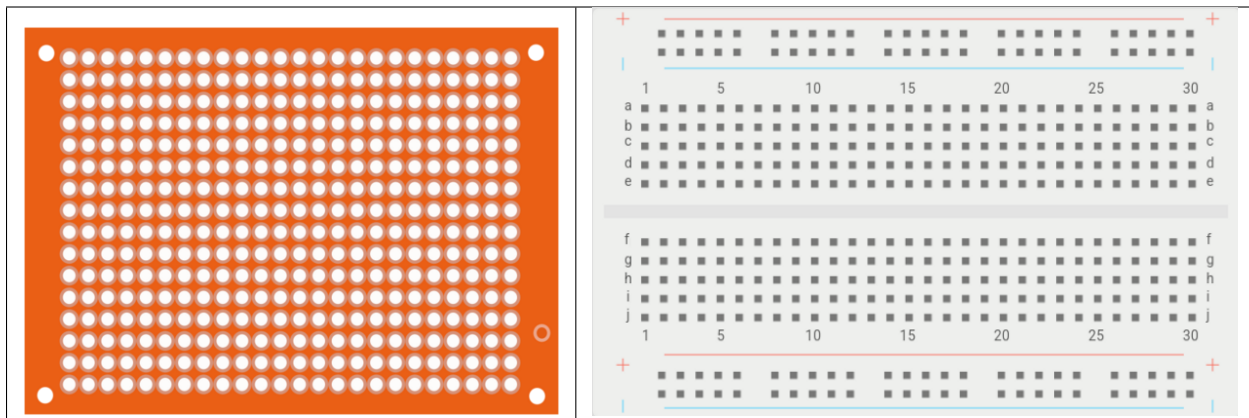
Discover the features	Get started
Interact with the world	Filter your signals
GitHub repository	PDF manual

ESSENTIALS

1.1 Why Plaquette?

Plaquette is a groundbreaking creative coding framework designed to empower creative practitioners by simplifying the way they work with real-time signals in tangible computing applications. By bridging the gap between low-level electronics and high-level creative expression, Plaquette enables creators to focus on what matters most: bringing their visions to life.

Note: Plaquette is a French noun pronounced [pla-kett](#) which refers to prototyping plates or boards (“plaquette de prototypage”) commonly used in designing electronic projects.



1.1.1 The Need for a New Standard

Media artists, interactive designers, digital luthiers, and electronic musicians constantly engage with real-time signals. However, when working with tangible computing systems such as embedded sensors, robotics, connected objects, and electronic music instruments, available tools such as [Arduino](#) are often very low-level and lack expressivity. Creative practitioners thus struggle to implement their vision directly using such platforms.

Consider the following case of learning how to work with a simple light sensor (eg. photoresistor) connected to an Arduino board on analog pin 0. The code reads as follows:

```
int value = analogRead(A0);
```

The value that is read is a raw 10-bit value returned by the Arduino board’s Analog to Digital Converter (ADC), an integer between 0 and 1023. But how is this value intuitively useful for an artist who wants to use this value creatively?

For example, what if one wants to react to a flash of light? Well, one solution would be to look at the value and compare it to a threshold:

```
if (value > 716) {  
  ...  
}
```

There are two problems with this approach.

Firstly, while it might work under certain lighting conditions, it will likely stop working if these conditions change, forcing us to make adjustments to the threshold value by hand.

Secondly, and perhaps more importantly, this piece of code does not really *express* what we are after. As creative practitioners, we don’t care whether the light signal is above 716 or 456 or whatnot: what we really want to know to detect a flash of light is whether the light signal is *significantly high compared to ambient light*.

What this example shows is that the way we are teaching and learning about sensor data is inefficient for creative applications. In other words: **raw digital data lacks expressiveness**.

Continuing with our example, consider how one would take the input value and directly reroute it to an analog (PWM) output on pin 9:

```
analogWrite(9, value / 4);
```

Why do we need to perform that division by 4? That’s because while the ADC gives us 10-bit values (1024 possibilities), the PWM only supports 8 bits (256 possibilities) forcing us to divide the incoming value by 4 (2 bits). But again, why is this detail important to know for an artist, designer, or musician? And what exactly does it have to do with our expressive intention?

1.1.2 Meet Plaquette

As a way to address these issues, Plaquette offers a general-purpose, standard interface for simple, real-time signal processing tailored for media artists.

Plaquette’s key objectives are:

1. **Empowering creators to focus on the creative aspects of their work**, rather than getting lost in irrelevant numerical details, which supports a smoother learning process.
2. **Providing accessible tools for creative practitioners** that capture high-level concepts such as “*normalizing*” and “*detecting peaks*”, without requiring deep technical knowledge of complex techniques like Fast Fourier Transforms or Chebyshev filtering.
3. **Facilitating teamwork and interoperability** by promoting an intuitive, cross-platform approach to real-time signals, such as by keeping all signals consistently scaled between 0 and 1 for easier integration across different applications.

Plaquette achieves these goals by embracing the following core principles:

- **Ease of use:** Offering a carefully selected set of functionalities that address the most common challenges faced by creators—keeping things simple while solving 95% of typical use cases.
- **Real-time performance:** Enabling smooth, uninterrupted interactions to ensure responsiveness in dynamic environments.
- **Signal-oriented approach:** Focusing on meaningful signal manipulation rather than dealing with arbitrary numerical values such as 255, 1024, 716, or 42.

- **Robustness:** Adapting to changes in the sensory context without breaking down, ensuring reliability in unpredictable and evolving environments such as art installations, live performances, and public art.
- **Interoperability and extensibility:** Leveraging an object-oriented architecture that seamlessly integrates with the Arduino ecosystem, ensuring compatibility and future scalability.

1.2 Features

Plaqueette is an *object-oriented, user-friendly, signal-centric* framework that facilitates *data processing* in *real-time*. It is fully *compatible with Arduino*.

1.2.1 Object-oriented

Plaqueette is designed using input, output, and filtering units that are easily interchangeable in a plug-and-play fashion. Units are created using expressive code.

For example, the code `DigitalOut led` creates a new digital output object that can be used to control an LED.

Arduino	Plaqueette
<i>Create digital output to control an LED:</i>	
<code>pinMode(12, OUTPUT);</code>	<code>DigitalOut led(12);</code>
<i>Create digital input push-button:</i>	
<code>pinMode(2, INPUT);</code>	<code>DigitalIn button(2);</code>

1.2.2 User-friendly

Plaqueette allows users to quickly design interactive systems using an expressive language that abstracts low-level functions. This allows both beginners and experts to create truly expressive code. For example, switching our LED object on or off can be achieved by calling: `led.on()`. Find out more about Plaqueette's base units by following [this link](#).

Arduino	Plaqueette
<i>Turn LED on:</i>	
<code>digitalWrite(ledPin, HIGH);</code>	<code>led.on();</code>
<i>Check if button is pushed:</i>	
<code>if (digitalRead(buttonPin) == HIGH)</code>	<code>if (button.isOn())</code>

1.2.3 Signal-centric

Plaqueette helps designers manipulate real-time signals from inputs to outputs. In Plaqueette, signals are represented either as `true/false` conditions (in the case of digital binary signals such as those coming from a button or switch), or as floating-point numbers in the `[0, 1]` range (ie. 0% to 100%) (in the case of analog signals such as those emitted by a light sensor, microphone, or potentiometer.) Because of this, there is no more need for users to perform counter-intuitive conversions on integer values.

Arduino	Plaqueette
<i>Check if button is released:</i>	
<code>if (digitalRead(buttonPin) != HIGH)</code>	<code>if (!button)</code>
<i>Check if sensor value is higher than 70%:</i>	
<code>if (analogRead(sensorPIN) >= 716)</code>	<code>if (sensor >= 0.7)</code>

1.2.4 Data-flow

Plaqueette provides simple yet powerful data filtering tools for debouncing, smoothing, and normalizing data. Removing noise in input signals can be as simple as calling a function such as `debounce()` or `smooth()`. Rather than guessing the right threshold for triggering an event based on input sensor input, one can use auto-normalizing *filters* such as *MinMaxScaler* and *Normalizer*.

Signals in Plaqueette can easily flow between units, in a similar fashion to modern data-flow software such as *Max*, *Pure Data*, and *TouchDesigner*. While this can be achieved using function calls, Plaqueette provides a special **flow operator** (`>>`) which allows data to be sent from one unit to another.

Arduino	Plaqueette
<i>Set LED to ON when button is pressed:</i>	
<code>digitalWrite(12, digitalRead(2));</code>	<code>button >> led;</code>
<i>Set LED to ON when input sensor is high:</i>	
<code>digitalWrite(12, (analogRead(A0) >= 716 ? HIGH : LOW));</code>	<code>(sensor >= 0.7) >> led;</code>
<i>Send multiple values to the serial plotter for visualization</i>	
<pre>Serial.print(sensor1); Serial.print(","); Serial.print(sensor2); Serial.print(","); Serial.println(sensor3);</pre>	<pre>sensor1 >> plotter; sensor2 >> plotter; sensor3 >> plotter;</pre>

Read *Regularizing Signals* to see how you can take full advantage of Plaqueette's signal filtering features.

1.2.5 Real-time

Plaqueette avoids blocking processes such as Arduino's (in)famous `delay()` by providing a set of *timing units* as well as time-based *signal generators*. As such, the processing loop is never interrupted, allowing interactive and generative processes to flow smoothly.

Plaqueette forbids the use of blocking functions such as Arduino's `delay()` and `delayMicroseconds()`. Rather, it invites programmers to adopt a frame-by-frame approach to coding similar to *Processing*.

Compare the following attempt to make an **LED blink** when pressing a button in Arduino using blocking calls to `delay()` to wait 500 milliseconds (0.5 seconds) between each flip of the LED, versus Plaqueette's real-time approach using a square oscillator with a one-second period:

Arduino	Plaquette
<pre> int buttonPin = 2; int ledPin = 12; void setup() { pinMode(buttonPin, OUTPUT); pinMode(ledPin, OUTPUT); } void loop() { // Button is checked once per second. if (digitalRead(buttonPin) == HIGH) { digitalWrite(ledPin, HIGH); delay(500); // do nothing for 500ms digitalWrite(ledPin, LOW); delay(500); // do nothing for 500ms } } </pre>	<pre> DigitalIn button(2); DigitalOut led(12); // Square wave 1 second period. Wave oscillator(1.0); void step() { // Button is checked at all time. if (button) oscillator >> led; } </pre>

1.2.6 Arduino-compatible

Plaquette is installed as an Arduino library and provides a replacement for the core Arduino functionalities while remaining fully compatible with Arduino code. Seasoned Arduino users should consult the [Advanced Usage](#) section for some tips on how to integrate Plaquette into their existing code.

The following example uses Plaquette to control a blinking LED that slows down with each button push, using Arduino's `constrain()` to keep the LED oscillation period within a certain range and `Serial` object to reset the counter to a random integer value using `random()`.

```

#include <Plaquette.h>

DigitalIn button(2, INTERNAL_PULLUP); // button input

DigitalOut led(LED_BUILTIN); // LED output

Wave oscillator(1.0); // square oscillator

int currentPeriod = 0; // oscillator period counter

void begin() {
  button.debounce(); // debounce button
  Serial.begin(115200); // opening a Serial port in Arduino-style
}

void step() {
  if (Serial.read() == 'R') // You can still use Serial.read to monitor incoming_
    ↪ messages. If 'R', reset counter
    currentPeriod = random(1, 10);

  if (button.rose()) // true when value rises (ie. button is pushed)

```

(continues on next page)

(continued from previous page)

```
currentPeriod = constrain(currentPeriod+1, 1, 10); // increment

oscillator.period(currentPeriod); // set period
oscillator >> led; // send signal to LED
}
```

Danger: Plaquette needs the main processing loop to run continuously without interruption to work correctly. Users should thus **avoid using blocking processes** such as Arduino's `delay()` and `delayMicroseconds()` and functions in their code when using Plaquette.

Warning: Many of the core Arduino functions work with integer types such as `int` or `long` rather than floating-point types such as `float`. Plaquette provides alternative *functions* which should be used instead.

In particular, please use:

- `mapFloat()` instead of `map()`
- `randomFloat()` instead of `random()`
- `seconds()` instead of `millis()`

Warning: Plaquette is still at an experimental stage of development. If you have any issues or questions, please contact the developers, or file a bug in our [issue tracker](#).

1.3 Reference

1.3.1 Base Units

- *AnalogIn* Reads analog input values between 0 and 1. Typically used for sensors that output a range of values, such as potentiometers or light sensors.
- *AnalogOut* Writes analog output values between 0 and 1. Useful for controlling devices like dimmable LEDs or motor controllers.
- *DigitalIn* Reads digital input values as boolean true/false. Commonly used with buttons, switches, or other on/off signals.
- *DigitalOut* Writes digital output values as boolean true/false. Often used to control relays, LEDs, or other binary devices.

1.3.2 Generators

- *Ramp* Generates a linear ramp signal over time. Commonly used for smooth parameter transitions like fading lights or scaling values.
- *Wave* Generates a smooth, periodic wave signal with a square, triangle, or sine shape. Commonly used for oscillatory motion or smooth transitions.

1.3.3 Timing

- *Alarm* Triggers an event after a specified time duration. Can be used to schedule delays or time-based actions.
- *Chronometer* Measures elapsed time since the start or reset. Useful for timing events or profiling system performance.
- *Metronome* Produces a periodic tick at specified intervals. Often used in rhythmic applications such as sound or light pulses.

1.3.4 Filters

- *MinMaxScaler* Scales signals to fit within a specified minimum and maximum range. Essential for normalizing input signals from diverse sources.
- *Normalizer* Adjusts signals to have a zero mean and unit variance. Useful in signal processing pipelines where consistent scaling is required.
- *PeakDetector* Detects peaks (local maxima) in input signals, allowing for event-based processing such as edge detection.
- *Smoother* Reduces noise and fluctuations in input signals using smoothing algorithms like exponential moving averages.

1.3.5 Communication

- *Monitor* Prints human-readable text and values to a print device (such as the Arduino serial monitor). Mainly used for debugging and logging.
- *Plotter* Streams numeric values in structured rows for visualization or logging (e.g., Serial Plotter, CSV, JSON). Designed for plotting and data export.

1.3.6 Fields

- *PivotField* Generates a spatial response curve based on a pivot point around which the field transitions happens, making it ideal for creating animations such as VU-meters or fades on arrays of actuators such as LEDs or motors.
- *TimeSliceField* Collects values over time which can then be sampled spatially like an array accross a normalized range. Useful for plotting time-varying signals, such as mapping audio or sensor input onto an LED strip or a motor array.

1.3.7 Functions

- *mapFloat()* Maps a float value from one range to another. Useful for adapting input ranges to the desired output domain.
- *mapFrom01()* Maps a float value from the normalized [0,1] range to a custom range, such as [-10, 10].
- *mapTo01()* Maps a float value from a custom range to the normalized [0,1] range, simplifying calculations for normalized operations.
- *randomFloat()* Generates a random float between 0 and 1, ideal for simulations or procedural generation.
- *seconds()* Returns the current time in seconds since the program started, enabling precise time tracking.
- *wrap()* Wraps a value within a specified range, making it cyclic. Commonly used for angles or periodic parameters.

1.3.8 Structure

- *Engine* A control structure managing an ensemble of units, handling their initialization, update, and timing, ensuring they remain synchronized.
- *Value Units* Wrappers for basic types, allowing to use them in Plaquette flows.
- *prepare()* Optional function called at the very beginning, before units are initialized.
- *begin()* Initializes the system, similar to Arduino's *setup()* function. Sets up necessary configurations and prepares units for operation.
- *step()* Repeatedly called during the program's execution, akin to Arduino's *loop()* function. Drives the execution of the main logic.
- *[] (arrays)* Allows the creation of arrays of Plaquette units for batch operations. Facilitates efficient processing of multiple units simultaneously.
- *.* (*dot*) Provides access to an object's methods and data, enabling intuitive object-oriented programming with Plaquette units.
- *>> (flow)* Sends data across units from left to right, creating a streamlined and intuitive flow of information between connected units.

1.3.9 Extra

- *Easings* Provides easing functions for smooth and natural transitions between values. Commonly used in animations and motion design.
- *ContinuousServoOut* Controls a continuous rotation servo motor by setting its speed and direction. Ideal for robotics or mechanical motion control.
- *ServoOut* Controls a standard servo motor by setting its angle. Useful for applications like robotic arms or pan-tilt systems.

2.1 Getting started

This short introduction will guide you through the first steps of using Plaquette.

2.1.1 Step 1: Install Plaquette

If you do not have Arduino installed on your machine you need to [download and install the Arduino IDE](#) for your platform.

Once Arduino is installed, please install Plaquette as an Arduino library following [these instructions](#).

2.1.2 Step 2: Your first Plaquette program

We will begin by creating a simple program that will make the built-in LED on your microcontroller blink.

Create a new sketch

Create a new empty sketch by selecting **File > New**.

Note: New Arduino sketches are initialized with some “slug” starting code. Make sure to erase the content of the sketch before beginning. You can use **Edit > Select All** and then click **Del** or **Backspace**.

Include library

Include the Plaquette library by typing:

```
#include <Plaquette.h>
```

Create an output unit

Now, we will create a new unit that will allow us to control the built-in LED:

```
DigitalOut myLed(13);
```

In this statement, `DigitalOut` is the **type** of unit that we are creating (there are other types of units, which we will describe later). `DigitalOut` is a type of software unit that can represent one of the many hardware pins for digital output on the Arduino board. One way to think about this is that the `DigitalOut` is a “virtual” version of the Arduino pin. These can be set to one of two states: **on** or **off**.

The word `myLed` is a **name** for the object we are creating.

Finally, `13` is an **argument** that specifies the hardware *pin* that the object `myLed` corresponds to on the board. In English, the statement would thus read as: “Create a unit of type `DigitalOut` named `myLed` on pin `13`.”

Tip: Most Arduino boards have a pin connected to an on-board LED in series with a resistor and on most boards, this LED is connected to digital pin `13`. The constant `LED_BUILTIN` is the number of the pin to which the on-board LED is connected.

Create an input unit

We will now create another unit which will generate a signal switching regularly from **on** to **off**. This signal will be sent to the LED to change its state, thus making it blink. To do so, we will use the `Wave` unit type to generate a *square wave* oscillating between on and off at a regular period of 2 seconds:

```
Wave myWave(2.0);
```

Note: The **argument** here is used not to specify a pin number (as for the `DigitalOut` unit) but rather to define the period of oscillation. Each object **type** provides a unique set of argument combinations and definitions specific to their usage..

Create the `step()` function

Now that you have your input and your output units, you need to tell Plaquette how they interact. For this, we will create a **function**. A function is a self-contained block of code that defines a series of operation.

The function we will create is called `step()`: it is called **repetitively and indefinitely** by Plaquette during the course of the program (like the `loop()` function in Arduino).

Our `step()` function will simply send the signal generated by the `myWave` input unit to the `myLed` output unit. We will do this by using Plaquette’s special `>>` operator:

```
void step() {  
    myWave >> myLed;  
}
```

In plain English, the statement `myWave >> myLed` reads as: “Take the value generated by `myWave` and put it in `myLed`.”

Note: It is beyond the scope of this introduction to explain the keyword `void`. If you are curious, however, you can read the [Arduino documentation](#).

Upload sketch

Upload your sketch to the Arduino board. You should see the LED on the board **blinking once every two seconds at a regular pace**.

Et voilà!

Full code

```
#include <Plaquette.h>

DigitalOut myLed(13);

Wave myWave(2.0);

void step() {
    myWave >> myLed;
}
```

2.1.3 Step 3 : Experiment!

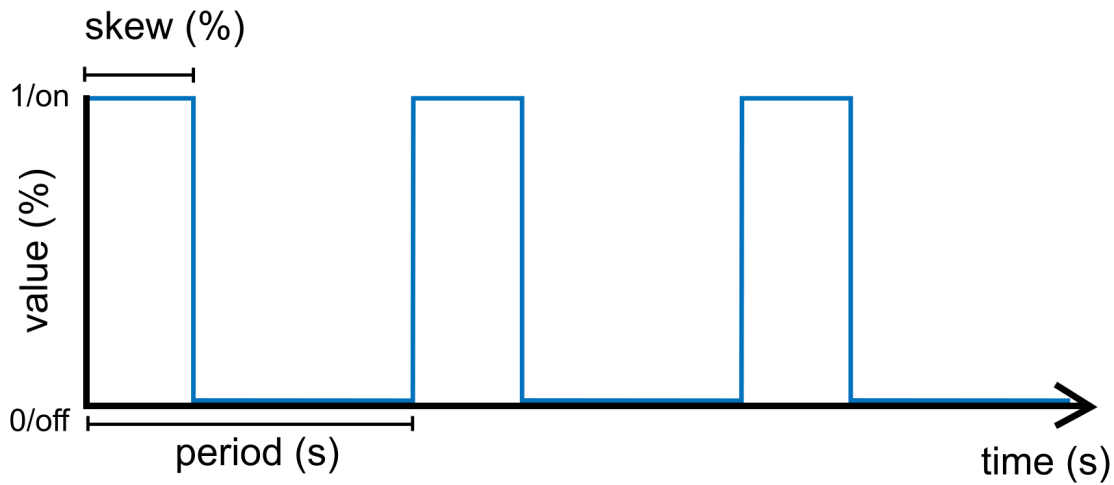
So far so good. Let's see if we can push this a bit further.

The Wave unit type accepts two arguments that allow you to create a wide range of rhythmic patterns:

```
Wave myWave(period, skew);
```

- **period** (which we have already used) can be any positive number representing the period of oscillation (in seconds)
- **skew** (optional) can be any number between 0.0 (0%) and 1.0 (100%), and represents the proportion of the period during which the signal is **on**. Default value: 0.5 (50%).

Note: We call this step the **construction** or **instantiation** of the object `myWave`.



Adjust the period

Try changing the first argument (`period`) in the square oscillator unit to change the period of oscillation.

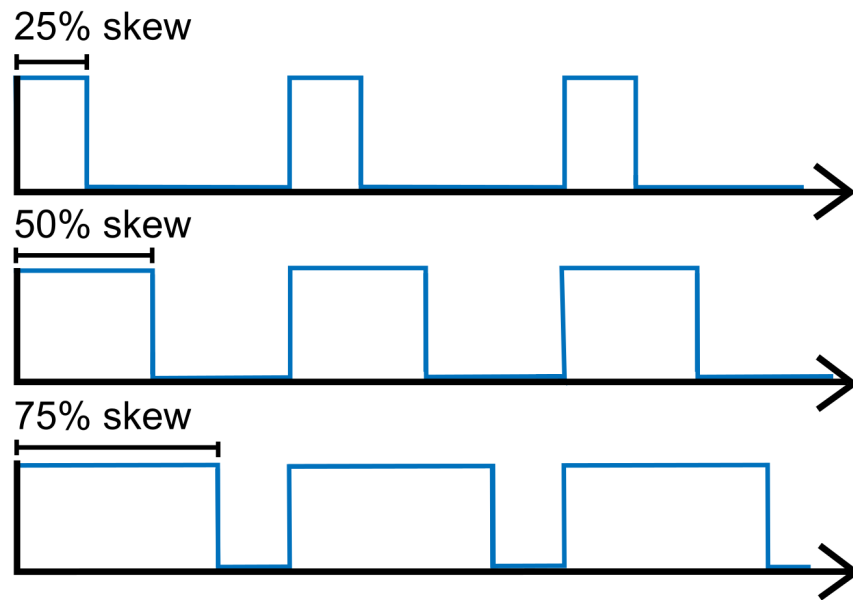
- Wave `myWave(1.0)`; for a period of one second
- Wave `myWave(2.5)`; for a period of 2.5 seconds
- Wave `myWave(10.0)`; for a period of 10 seconds
- Wave `myWave(0.5)`; for a period of half a second (500 milliseconds)

Important: Don't forget to re-upload the sketch after each change.

Skew that wave!

Now try adding a second argument (`skew`) to control the oscillator's `skew`. For a fixed period, try changing the duty cycle to different percentages between 0.0 and 1.0.

- Wave `myWave(2.0, 0.5)`; for a skew of 50% (default)
- Wave `myWave(2.0, 0.25)`; for a skew of 25%
- Wave `myWave(2.0, 0.75)`; for a skew of 75%
- Wave `myWave(2.0, 0.9)`; for a skew of 90%



Understanding parameters

In Plaquette, a **parameter** is a property of a unit that you can read and modify while your program is running. Parameters give you dynamic control over your units, allowing you to change their behavior in real-time.

The Wave unit has several parameters:

- **period**: the duration of one complete cycle (in seconds)
- **skew**: the proportion of the cycle during which the signal is on (0.0 to 1.0)
- **frequency**: the number of cycles per second (in Hz)
- **bpm**: the number of cycles per minute (beats-per-minute)
- **jitter**: adds randomness to the timing (0.0 to 1.0)
- **phaseShift**: the phase offset as a proportion of period (0.0 to 1.0)
- **phase**: the oscillation phase as a proportion of period (0.0 to 1.0)

You may have noticed that when we created our wave, we were able to set some of these parameters (period and skew) directly using the constructor arguments:

```
Wave myWave(2.0);           // sets the period parameter to 2 seconds
Wave myWave(2.0, 0.25);    // sets period to 2 seconds and skew to 25%
```

But what about the other parameters like **frequency** or **bpm**? Let's first see how we can initialize those at startup using the **.** (*dot*) operator.

Initialize parameters in the `begin()` function

The `begin()` function is automatically called **only once at the beginning** of the sketch (just like the `setup()` function in Arduino). It is a good place to initialize parameters that cannot be set through constructor arguments.

For example, to set our wave's frequency instead of its period:

```
void begin() {  
  myWave.frequency(10); // 10 Hz = 10 times per second  
}
```

Or, if you prefer to set the frequency in beats-per-minute (BPM):

```
myWave.bpm(120); // 120 cycles per minute
```

Note: Setting the frequency will override the period (and vice versa), since period is simply the inverse of frequency.

Change parameters of a unit during runtime

What if we wanted to change the parameters of the oscillator during runtime rather than just at the beginning? The Wave unit type allows real-time modification of its **parameters** by calling one of its functions using the `.` (*dot*) operator.

For example, to change the frequency while the program is running, we would simply need call `frequency(value)` function inside the `step()` function rather than the `begin()` function:

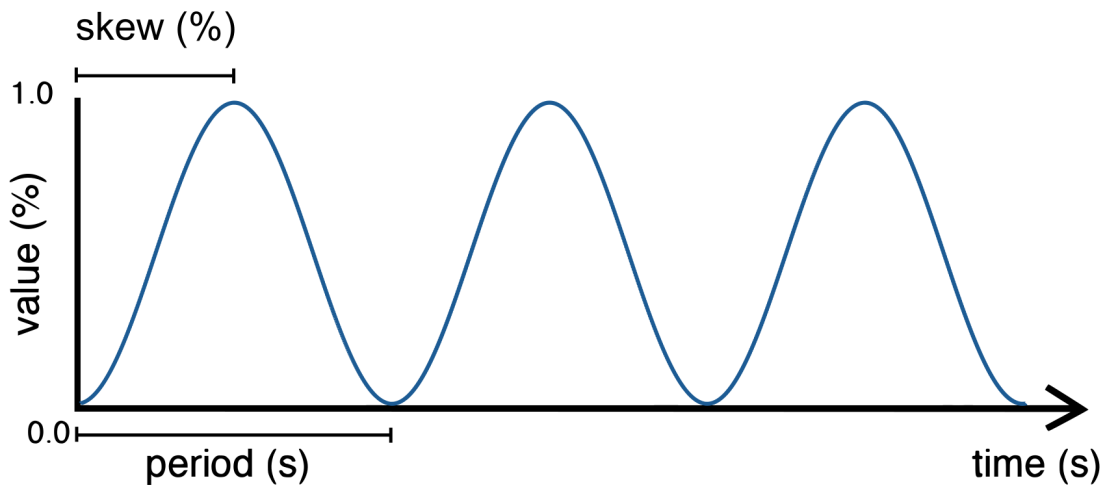
```
void step() {  
  myWave.frequency(newFrequency); // change the frequency  
  myWave >> myLed;  
}
```

Of course, to accomplish our goal, we need a way to *change* the value `newFrequency` during runtime. We can accomplish this in many different ways, but let's try something simple: we will use another wave to *modulate* our wave's frequency.

For this, we will be using another type of wave called a *sine wave* and will use its outputs to change the period of `myWave`.

```
Wave myModulator(SINE, 20.0);
```

This wave will oscillate smoothly from 0 to 1 every 20 seconds.



```
void step() {
  myWave.frequency(myModulator); // change the frequency of myWave using myModulator
  myWave >> myLed;
}
```

Upload the sketch and you should see the LED blinking as before, with the difference that the blinking speed will now change from blinking very fast (in fact, infinitely fast, with a period of zero seconds!) to very slow (period of 20 seconds).

You can take advantage of the >> operator to change a unit's parameter with a more expressive syntax:

```
myModulator >> myWave.Frequency(); // equivalent to: myWave.frequency(myModulator);
```

Important: Notice the use of the **capitalized first letter** in the above call: `myWave.Frequency()`. This tells Plaquette that we are flowing a value to a *parameter slot* rather than directly reading or setting a parameter.

In Plaquette, most functions are written in lowercase (such as `frequency()`) and are used to **read or set a value directly**. Functions starting with a capital letter (such as `Frequency()`) instead provide a **connection point** for that parameter, making it possible to connect it seamlessly using the >> operator, without confusing parameter slots with normal function calls.

Upload the sketch and you should see the LED blinking as before, with the difference that the blinking speed will now change from blinking very slowly (in fact, infinitely slowly, with a frequency of zero-times-per-seconds!) to one time per second.

Tip: You can try modulating other parameters of the wave instead of its frequency:

```
myModulator >> myWave.Skew();
myModulator >> myWave.Period();
```

This table summarizes the different ways one can access and modify a unit's parameters:

Name	Format	Preferred use	Example use
Read parameter	<code>unit.parameter()</code>	Read the current value of a parameter (monitoring, logic, debugging).	<code>wave.frequency() >> plotter;</code>
Set parameter	<code>unit.parameter(value)</code>	Initialize or directly update a parameter from a value or expression.	<code>wave.frequency(2.0);</code>
Parameter flow	<code>source >> unit.Parameter()</code>	Continuously control a parameter (modulation, mapping).	<code>modulator >> wave.Frequency();</code>

Caution: You *cannot* use the `>>` operator with any objects or functions: only Plaquette units and parameters are supported. Not all functions provided by a unit are parameters. Read the unit's documentation for more details.

Tip: If you want to visualize the values of both waves on your computer, you can print them on the serial port. For this you need to create a **Plotter**. You can do so by adding the following line at the top of your sketch:

```
Plotter plotter(115200);
```

The **Plotter** unit type accepts an argument to define baud rate, or the speed of message transmission. In this guide we use 115200, which is a standard speed for serial communication.

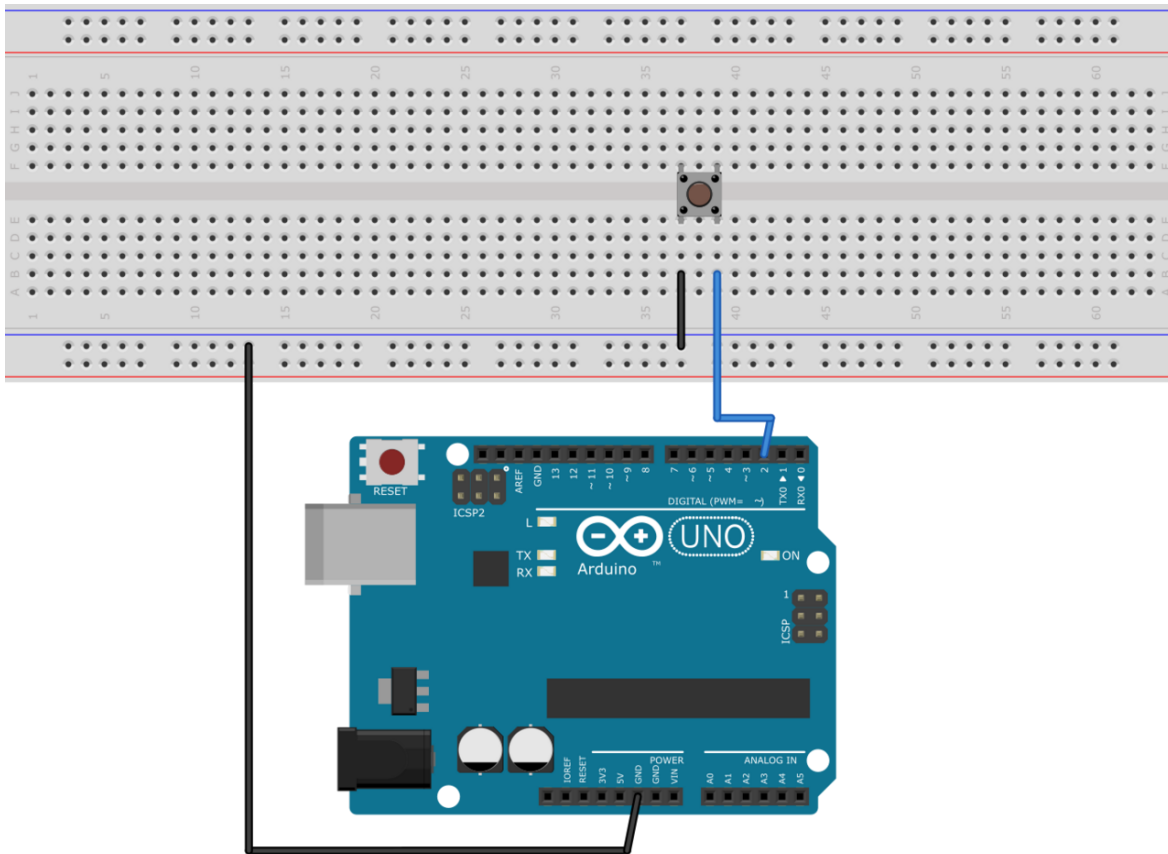
Then, add the following code to your `step()` function to send the values to the plotter:

```
myWave >> plotter;
myModulator >> plotter;
```

Finally, launch the Arduino **Serial Plotter** by selecting **Tools > Serial Plotter**. Make sure to select the same baud rate (115200). You should see a live visualization of the two waves.

Use a button

Now let's try to do some very simple interactivity by using a simple switch or button. For this we will be using the internal pull-up resistor available on Arduino boards for a very simple circuit. One leg of the button should be connected to ground (GND) while the other should be connected to digital pin 2.



Tip: If you do not have a button or switch, you can just use two electric wires: one connected to ground (GND) and the other one to digital pin 2. When you want to press the button, simply touch the wires together to close the circuit.

Declare the button unit with the other units at the top of your sketch:

```
DigitalIn myButton(2, INTERNAL_PULLUP);
```

You will notice that the type of this unit (*DigitalIn*) resembles that of our LED-controlling unit (*DigitalOut*). This is because both units have something in common: they have only two states: either on or off, high or low, true or false, one or zero, hence the adjective *Digital*. However, while the LED is considered an output or actuator (*Out*) our button is rather an input or sensor (*In*).

Note: If you are curious, you might also want to know that there is an *AnalogIn* and an *AnalogOut* types which support sensors and actuators that work with continuous values between 0 and 1 (0% to 100%).

Now, let's use this button as a way to control whether the LED blinks or not. For this, we will need to use the value of the button as part of a **condition** for an *if...else* statement.

```
if (myButton)
  myWave >> myLed;
else
  0 >> myLed;
```

Full code

```
#include <Plaquette.h>

DigitalOut myLed(13);

Wave myWave(2.0);

Wave myModulator(SINE, 20.0);

DigitalIn myButton(2, INTERNAL_PULLUP);

void step() {
    myModulator >> myWave.Period();

    if (myButton)
        myWave >> myLed;
    else
        0 >> myLed;
}
```

2.1.4 Learning More

Built-in Examples

You will find more examples [here](#) or directly from the Arduino software in **File > Examples > Plaquette** including:

- Using analog inputs such as a photocells or potentiometers
- Using analog outputs
- Serial input and output
- Using wave generators
- Time management
- Ramps
- Basic filtering (smoothing, re-scaling)
- Peak detection
- Event-driven programming
- Controlling servomotors

The Plaquette Reference

The online reference can be accessed [here](#) or directly from the sidebar of the [Plaquette website](#). It provides detailed technical documentation for every available unit and function in Plaquette. This reference serves as a go-to resource for understanding the specifics of each component, including their parameters, methods, and behavior.

Here are the key sections of the reference:

- *Base Units*: Introduces foundational units like *DigitalIn*, *DigitalOut*, *AnalogIn*, and *AnalogOut*. These are the building blocks for interfacing with hardware pins.
- *Generators*: Covers the signal generators *Wave* and *Ramp*. These are used to create various types of regular signals.
- *Timing*: Focuses on units related to time management such as *Metronome* for periodic events and *Alarm* for duration-based functionality.
- *Filters*: Discusses tools for *smoothing*, *scaling*, or *normalizing* signals, as well as *detecting peaks*.
- *Communication*: Presents units used for *monitoring* and *plotting* data, allowing information to flow between embedded device and computer.
- *Functions*: Explains helper functions for tasks like value mapping, signal transformations, and conversions.
- *Structure*: Describes core structural functions and operators.
- *Extra*: Contains miscellaneous units and features.

What's Next?

With the basics covered, you are now ready to dive deeper into Plaquette's capabilities. Explore the rest of the guide to learn about specific features and advanced techniques:

- *Inputs and Outputs*: Learn how to use Plaquette to handle a variety of inputs and outputs, including analog, digital, and specialized sensors or actuators.
- *Generating Waveforms*: Understand the different types of wave generators available and how they can be used for oscillatory or periodic behavior.
- *Working with Time*: Delve into Plaquette's timing management units to handle scheduling and time-based logic in your projects.
- *Regularizing Signals*: Discover methods for automatically scaling and normalizing signals and respond to peaks.
- *Managing Events*: Trigger actions, schedule events and manage parallel loops using event-driven programming.

2.2 Inputs and Outputs

When working with Plaquette, **inputs** and **outputs** allow you to interact with the physical world. Whether you are reading a sensor's value or controlling an actuator, Plaquette makes this process intuitive and efficient. This section introduces the basic concepts of digital and analog inputs and outputs, presents Plaquette's unique syntax for efficient and expressive code, explains how to clean noisy data, shows how to make decisions based on input values, and describes the different configuration modes of input and output units.

Let's explore these ideas step by step.

2.2.1 Digital vs Analog

Before diving into code, let's first clarify the difference between **digital** and **analog** signals.

- **Digital** signals represent binary states: ON vs OFF, HIGH vs LOW, 1 vs 0, true vs false. Examples of digital inputs include buttons and presence sensors, while digital outputs might control LEDs or relays.
- **Analog** signals represent a continuous range of values. Think of a dimmer switch (potentiometer) or a light sensor that measures brightness levels. Analog outputs control devices such as LEDs with variable brightness and DC motors where the speed can vary.

Warning: On many microcontrollers, analog outputs are generated using **Pulse Width Modulation (PWM)** rather than a true **Digital-to-Analog Converter (DAC)**. PWM rapidly switches the output pin between HIGH and LOW, creating the illusion of a continuous analog voltage when averaged over time. While this works for controlling brightness in LEDs or speed in motors, it may not be suitable for applications requiring a steady, smooth signal.

For more information, visit the official Arduino documentation on [Pulse Width Modulation](#).

2.2.2 Digital Inputs and Outputs

Let's look at the basics of digital signals by working with LEDs and buttons.

Digital Outputs

A *DigitalOut* unit controls devices that can be turned ON or OFF, such as LEDs or relays. Here's an example of how to control an LED:

```
#include <Plaquette.h>

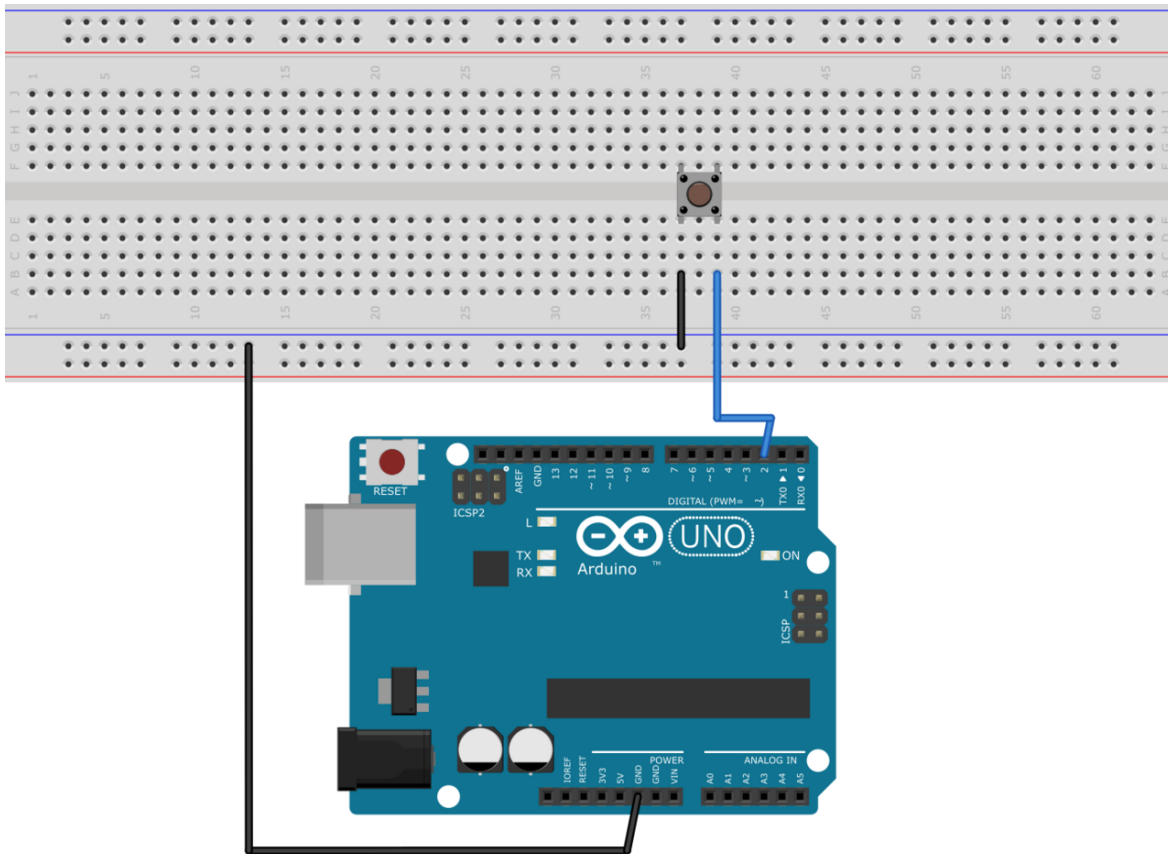
DigitalOut led(13); // LED connected to pin 13

void begin() {
    led.on(); // Turn the LED on initially
}

void step() {
    if (seconds() > 10.0)
        led.off(); // Turn the LED off after 10 seconds
}
```

Digital Inputs

A *DigitalIn* unit reads binary states from devices like buttons or switches. The easiest way to connect a button or switch is to use the **internal pull-up** resistor on Arduino boards. One leg of the button should be connected to ground (GND) while the other should be connected to a digital pin.



```
#include <Plaquette.h>

DigitalIn button(2, INTERNAL_PULLUP); // Button connected to pin 2
DigitalOut led(13);                  // LED connected to pin 13

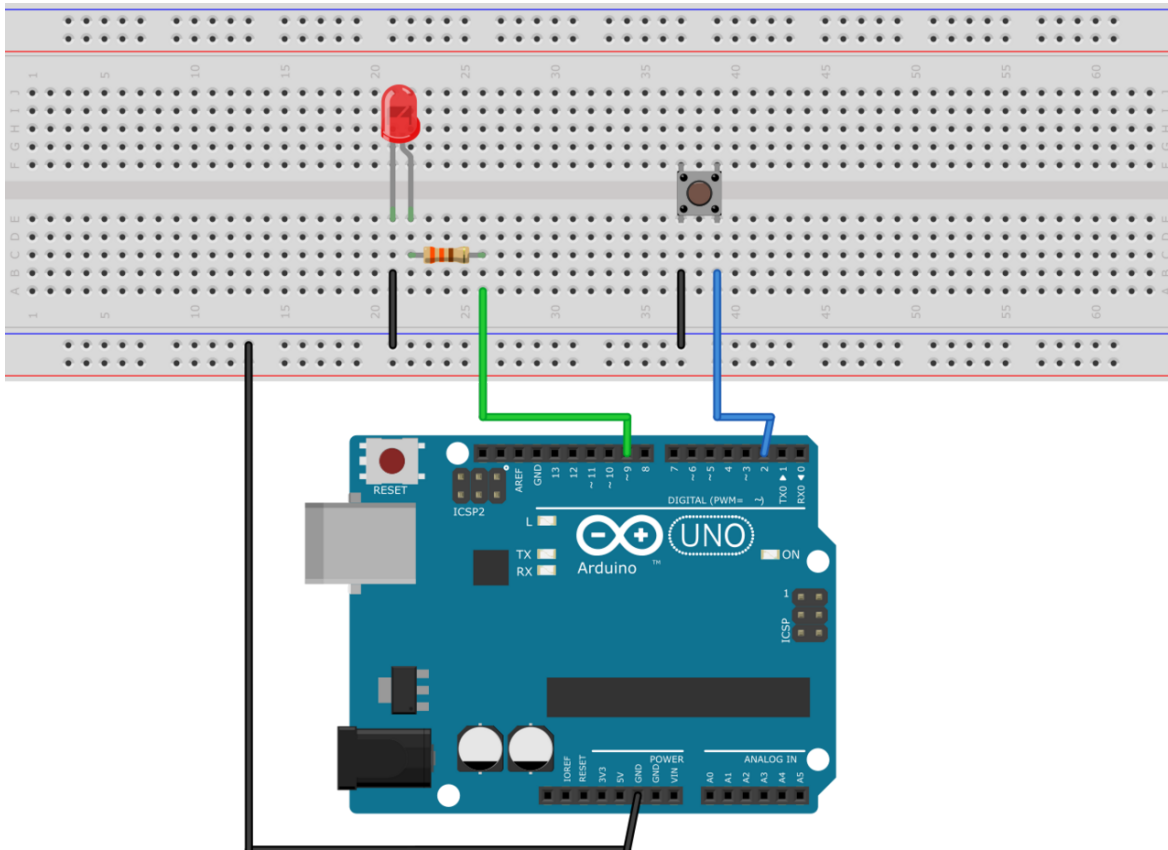
void step() {
  if (button.isOn()) { // Check if the button is pressed
    led.on();
  } else {
    led.off();
  }
}
```

2.2.3 Analog Outputs and Inputs

Next, we will explore analog signals, which allow for finer control and more detailed readings.

Analog Outputs

An *AnalogOut* unit controls devices like LEDs or motors with continuous values. Here's an example of dimming an LED. The cathode (short leg) of the LED should be connected to ground, while the anode (long leg) should be connected to a 300 Ω resistor, which in turn should be connected to an analog / PWM pin (eg. pin 9).



```
#include <Plaquette.h>

AnalogOut led(9); // LED connected to pin 9

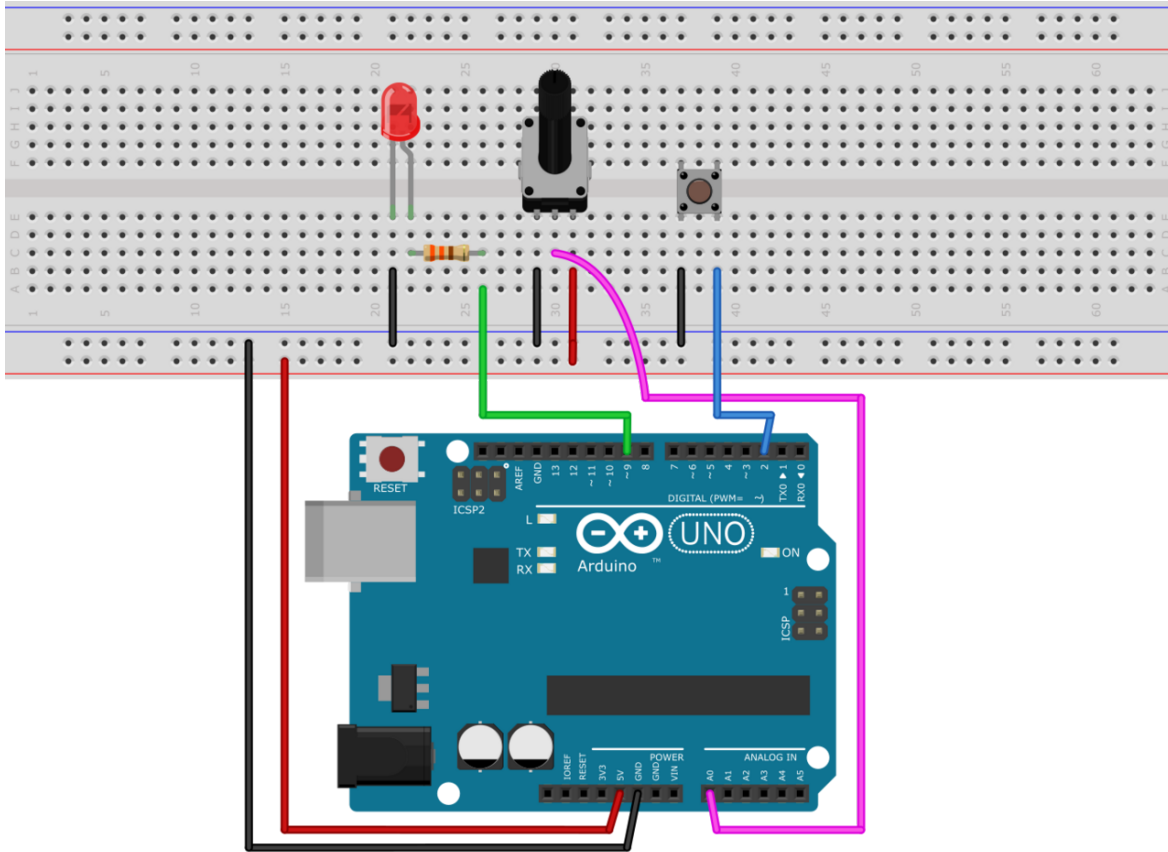
void begin() {
  led.put(0); // Set initial LED brightness to 0%
}

void step() {
  led.put( seconds() / 10 ); // Will reach 100% after 10 seconds
}
```

Analog Inputs

An *AnalogIn* unit reads continuous values from sensors, such as potentiometers, light, or temperature sensors.

Let's use a potentiometer to control an LED's brightness. For this circuit, the center pin of the potentiometer should be connected to analog input pin (A0), the left pin to ground (GND) and the right pin to +5V (Vcc).



```
#include <Plaquette.h>

AnalogIn dimmer(A0); // Potentiometer on analog pin A0
AnalogOut led(9);    // LED on pin 9

void step() {
  led.put(dimmer.get()); // Map the potentiometer value directly to LED brightness
}
```

2.2.4 Using Units as Their Own Values

Plaquette offers an elegant shortcut: you don't need to explicitly call `isOn()` or `get()` for digital or analog inputs. Instead, you can use the input or output unit itself in lieu of the value it contains. This makes your code cleaner and easier to read.

Here's the same LED and button example, rewritten with this feature:

```
#include <Plaquette.h>

DigitalIn button(2, INTERNAL_PULLUP);
DigitalOut led(13);

void step() {
  if (button) { // No need for button.isOn() : just use button as its own value
    led.on();
  } else {
    led.off();
  }
}
```

For analog inputs, this works similarly. Instead of calling `dimmer.get()`, you can use the `dimmer` unit directly:

```
#include <Plaquette.h>

AnalogIn dimmer(A0);
AnalogOut led(9);

void step() {
  led.put(dimmer); // No need for dimmer.get() : just use dimmer as its own value
}
```

These simplifications make your code more expressive and emphasize the logic over the syntax.

2.2.5 The Flow Operator (>>)

In Plaquette, the `>>` operator allows you to directly send the value of one unit to another. This makes it incredibly simple to map inputs to outputs without extra variables or function calls.

Let's revisit the potentiometer and LED example using the piping operator:

```
#include <Plaquette.h>

AnalogIn dimmer(A0);
AnalogOut led(9);

void step() {
  dimmer >> led; // Directly pipe the potentiometer value to the LED
}
```

This operator improves code readability and emphasizes the relationship between inputs and outputs.

Note: The flow operator (`>>`) allows to expressively connect input, output, and filtering units in a similar fashion to data-flow environments such as [Max](#), [Pure Data](#), and [TouchDesigner](#). The operator is directly inspired from the [Chuck](#)

operator (`=>`) in programming language [ChucK](#).

2.2.6 Dealing with Noisy Signals: Debouncing and Smoothing

In real-world applications, signals can be messy. Buttons can produce electrical noise when pressed, and analog sensors might give fluctuating readings. Plaquette provides tools to handle these issues: **debouncing** for digital signals and **smoothing** for analog ones.

Debouncing

Debouncing ensures that a button press is recorded cleanly, ignoring any noise. Here's how to debounce a button:

```
#include <Plaquette.h>

DigitalIn button(2, INTERNAL_PULLUP); // Button with pull-up resistor
DigitalOut led(13);                    // LED on pin 13

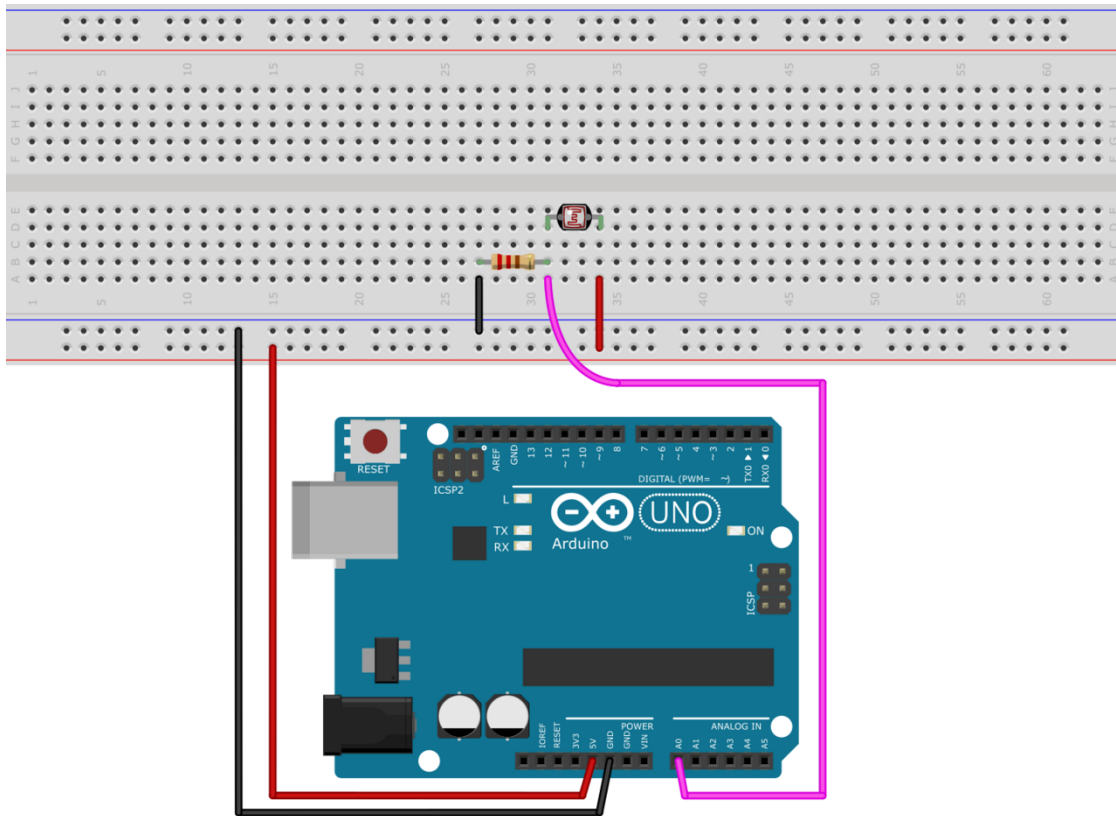
void begin() {
    button.debounce(); // Debounce the button
}

void step() {
    if (button.rose()) { // Detect a clean press
        led.toggle();   // Toggle the LED state
    }
}
```

Smoothing

For analog signals, smoothing helps stabilize noisy data.

Here's how you can smooth a light sensor (photoresistor). For this circuit, you will need to create a simple [voltage divider circuit](#). Connect the photoresistor between the ground (GND) and the analog input pin (A0). Then connect a fixed resistor with value matching your photoresistor between analog input pin and +5V (Vcc). For example, for a 1k Ω - 10k Ω photoresistor you could use a fixed resistor of about 5.5k Ω .



```
#include <Plaquette.h>

AnalogIn lightSensor(A0);
AnalogOut led(9);

void begin() {
  lightSensor.smooth(); // Apply default smoothing
}

void step() {
  lightSensor >> led;
}
```

You can adjust the level of smoothing and debouncing by indicating a parameter representing the time window (in seconds) over which the value is averaged. Experiment with different smoothing values to see the result:

- `lightSensor.smooth()` : Default smoothing window (100ms)
- `lightSensor.smooth(1.0)` : Smooth over one second
- `lightSensor.smooth(10.0)` : Smooth over 10 seconds
- `lightSensor.smooth(0.01)` : Smooth over 10ms

2.2.7 Mapping Values to Different Ranges

Sometimes, the output of a sensor doesn't match the range needed for an actuator. Plaquette provides a simple **mapping function** `mapTo(low, high)` which maps the analog input value to a specified range which is very useful for scaling sensor readings.

Example: Controlling the blinking frequency of an LED based on the value of a light sensor.

```
#include <Plaquette.h>

AnalogIn lightSensor(A0);
DigitalOut led(13);
Wave wave(1.0);

void step() {
  // Map sensor value to frequency in range 1-10 Hz
  wave.frequency( lightSensor.mapTo(1, 10) );
  // Control LED with wave.
  wave >> led;
}
```

2.2.8 Making Decisions with Conditions

Interactive systems often need to respond to changes in input. Plaquette provides convenient methods like `rose()`, `fell()`, and `changed()` for detecting transitions in digital signals.

Digital Conditions

Here's an example of toggling an LED when a button is pressed:

```
#include <Plaquette.h>

DigitalIn button(2, INTERNAL_PULLUP);
DigitalOut led(13);

void step() {
  if (button.rose()) { // Detect the moment the button is pressed
    led.toggle();      // Toggle the LED state
  }
}
```

Analog Conditions

Analog conditions are useful when you want to trigger actions based on a threshold. For instance, turning on an LED when the light level drops below 30% (0.3):

```
#include <Plaquette.h>

AnalogIn lightSensor(A0);
DigitalOut led(13);
```

(continues on next page)

(continued from previous page)

```

void step() {
  if (lightSensor < 0.3) {
    led.on(); // Turn on LED in low light
  } else {
    led.off(); // Turn off LED in bright light
  }
}

```

2.2.9 Modes for Inputs and Outputs

All input and output units in Plaquette support different modes, which allow you to adapt to various circuit configurations. You may already be familiar with the `INTERNAL_PULLUP` mode from *DigitalIn*, which provides a simple way to connect a button input. Let's explore how modes affect *DigitalIn*, *AnalogIn*, *DigitalOut*, and *AnalogOut* units.

Understanding these modes helps you design stable and efficient circuits, whether you're reading inputs or driving outputs. Choose the mode that best fits your hardware setup and application requirements.

DigitalIn Modes: DIRECT, INVERTED, and INTERNAL_PULLUP

The *DigitalIn* unit supports three primary modes:

- **DIRECT** (default): The unit is ON when the input pin is HIGH (e.g., 5V). This mode is used for buttons with pull-down resistors, which keep the pin LOW (OFF) when the button is not pressed and allow it to go HIGH (ON) when the button is pressed. Pull-down resistors typically have values around 10k Ω .

Example: Button connected between pin 2 and 5V with a pull-down resistor to ground:

```

DigitalIn button(2, DIRECT);
DigitalOut led(13);

void step() {
  if (button) {
    led.on();
  } else {
    led.off();
  }
}

```

- **INVERTED**: The unit is ON when the input pin is LOW (e.g., GND). This is useful for buttons with pull-up resistors, which keep the pin HIGH when the button is not pressed and allow it to go LOW when the button is pressed. The `INTERNAL_PULLUP` mode activates an internal pull-up resistor, simplifying the circuit.

Example: Button connected between pin 2 and ground with a pull-down resistor to +5V (Vcc):

```
DigitalIn button(2, INVERTED);
```

- **INTERNAL_PULLUP**: As in mode `INVERTED` the unit is ON when the input pin is LOW (e.g., GND). Makes use of the internal pull-up resistor on the board, therefore removing the need to add a pull-up resistor.

Example: Button connected between pin 2 and ground (no need for an extra pull-up resistor):

```
DigitalIn button(2, INTERNAL_PULLUP);
```

AnalogIn Modes: DIRECT and INVERTED

The *AnalogIn* unit also supports DIRECT and INVERTED modes, which determine how the sensor's voltage is interpreted:

- **DIRECT** (default): Reads the raw analog value, normalized to a range of [0.0, 1.0]. This mode is suitable for sensors like photoresistors, where increasing light decreases resistance, resulting in higher voltage and a higher normalized value.

Example: Using a photoresistor in direct mode:

```
AnalogIn lightSensor(A0, DIRECT);
AnalogOut led(9);

void step() {
  lightSensor >> led;
}
```

- **INVERTED:** Flips the normalized value, so high input voltage results in a low output value and vice versa. This is useful when you want the sensor to behave oppositely without changing your logic.

Example: Inverted photoresistor reading:

```
AnalogIn lightSensor(A0, INVERTED);
```

DigitalOut and AnalogOut Modes: DIRECT and INVERTED

The *DigitalOut* and *AnalogOut* units control the flow of current and can operate in two modes:

- **DIRECT** (default): The pin provides current when ON, suitable for devices like LEDs connected between the pin and ground.

Example: LED in direct mode. Connect the LED anode (long leg) to pin 9 and the cathode (short leg) to ground, with a 330 Ω in series.

```
AnalogOut led(9, DIRECT);
Wave wave(SINE, 1.0);

void step() {
  wave >> led;
}
```

- **INVERTED:** The pin emits zero volts (GND) when ON so the current “sinks” to the pin. Suitable for digital outputs connected between a positive voltage and the pin.

Example: LED in inverted mode. Connect the LED anode (long leg) to +5V (Vcc) and the cathode to pin 9, with a 330 Ω resistor in series.

```
AnalogOut led(9, INVERTED);
```

2.2.10 Conclusion

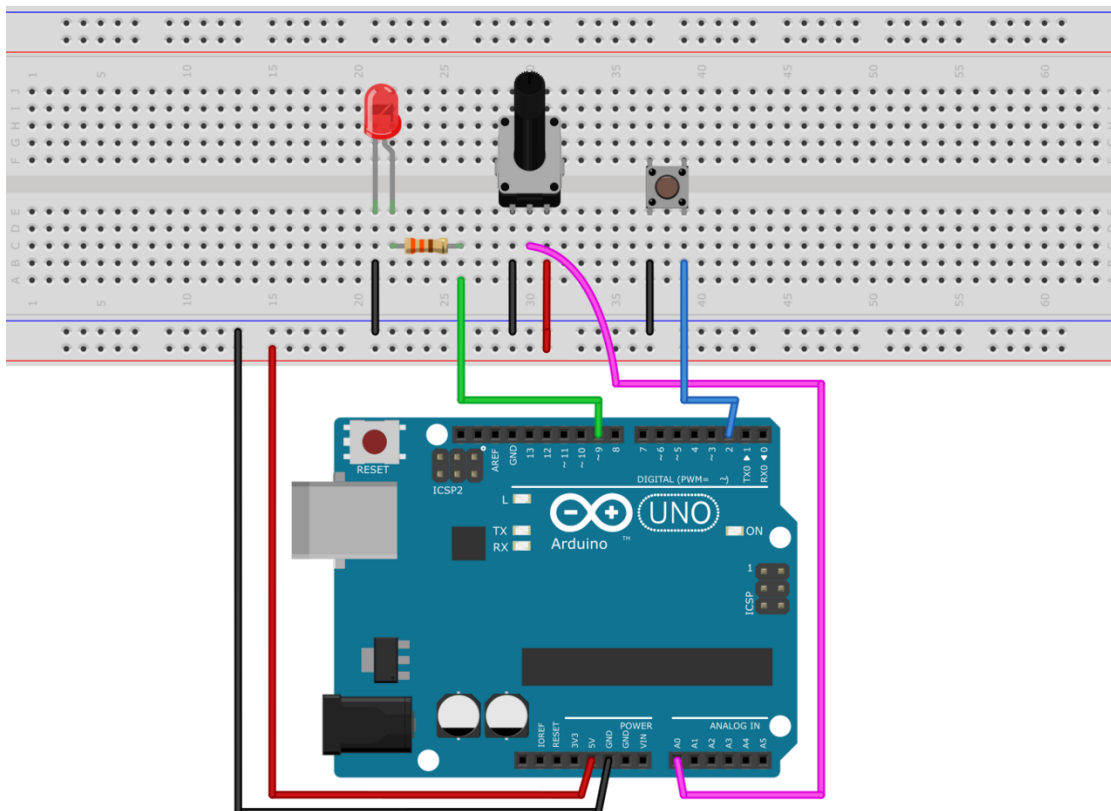
Understanding inputs and outputs is crucial for building interactive projects. With Plaqueette's simplified syntax, tools for handling noisy signals, and powerful mapping and conditional features, you can quickly create dynamic and engaging systems. Next, we'll explore how to use Plaqueette's timing and signal generation features to add even more complexity and creativity to your projects.

2.3 Generating Waveforms

In this section, we will explore waves (also called oscillators), essential tools for creating dynamic and expressive media. Oscillators generate repeating waveforms, which can control various outputs such as LEDs or motors. We will also learn how to visualize signals and shape different kinds of waveforms. We will then introduce combining different waves together, either by adding them or through modulation. Finally, we will look at how to use randomness to generate noisy waveforms that feel more natural.

Note: To follow along with the examples, set up a simple circuit:

- A **potentiometer** connected to A0 to control properties dynamically.
- A **button** connected to pin 2 with an internal pull-up resistor to trigger actions.
- An **LED** connected to pin 9 (PWM capable) through a 330 Ω resistor.



2.3.1 Visualizing Waves with the Serial Plotter

In this section, we will use **serial communication** to send data from our Arduino board to our PC so as to visualize the waves in real time. After instantiating a `Plotter` object, you can use `print()` and `println()` functions to send data to the plotter, which is invaluable for debugging and visualizing data. The plotter provides a way to graphically observe how wave properties like amplitude, phase, or frequency affect the output.

Single Signal

To visualize the data, open the [Serial Plotter](#) in the Arduino IDE. The Serial Plotter can graphically display waveforms by interpreting each printed value as a separate line on the graph, making it an invaluable tool to visualize signals such as sensor values and waveforms.

Example: Print the value of the potentiometer:

```
#include <Plaquette.h>

AnalogIn pot(A0); // The potentiometer
Plotter plotter(115200); // Create a plotter object and set a baudrate

void step() {
  pot >> plotter; // send the pot value directly to the plotter using a flow operator
}
```

Multiple Signals

You can use the same method to send multiple signals.

Example: Print the value of the potentiometer and a sine wave:

```
#include <Plaquette.h>

AnalogIn pot(A0); // Potentiometer input
Wave wave(SINE, 2.0); // Sine wave with period of 2 seconds

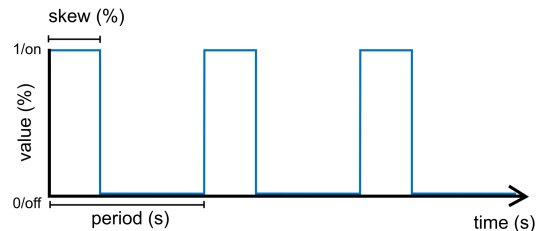
// Create plotter with baud and optional labels for multiple signals.
Plotter plotter(115200, "wave,pot"); // First value sent to plotter labeled as "wave",
→ second as "pot"

void step() {
  wave >> plotter;
  pot >> plotter;
}
```

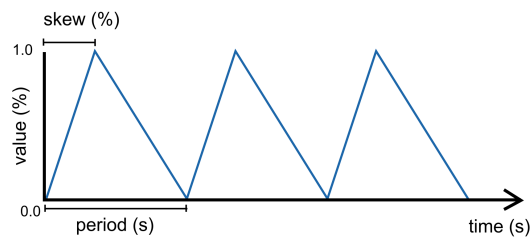
2.3.2 Types of Waves

Plaquette provides 3 shapes of waves:

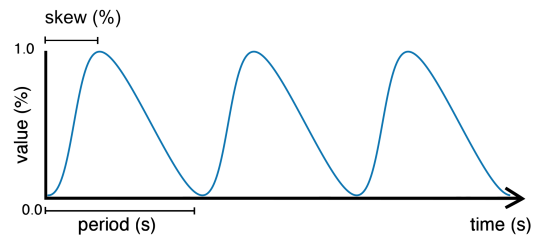
- **SQUARE** (default): Default wave shape. Alternates between two levels with sharp transitions. Useful for creating rhythmic on-off patterns such as blinking LEDs or simple tone generators for buzzers. Possesses some properties of digital units.



- **TRIANGLE**: Smoothly transitions between two levels in a linear fashion. By varying the skew of the wave, you can create a **sawtooth wave** (skew = 0) or an **inverted sawtooth wave** (skew = 1). This is ideal for simulating ramping motions or gradual changes in brightness.



- **SINE**: Produces a sinusoidal waveform for smoother modulation. Commonly used for creating natural, flowing transitions, such as smooth dimming or speed control.



You can visualize these waves on the Serial Plotter by streaming their values.

Example: Display different waves for comparison:

```
#include <Plaquette.h>

// Three wave types.
```

(continues on next page)

(continued from previous page)

```

Wave squareWave(SQUARE, 1.0);
Wave triangleWave(TRIANGLE, 1.0);
Wave sineWave(SINE, 1.0);

// Create plotter with baud and optional labels for multiple signals.
Plotter plotter(115200, "square,triangle,sine"); // First value sent to plotter labeled
↳ as "square", second as "triangle", etc.

void step() {
    // Print all wave values separated by spaces
    squarewave >> plotter;
    trianglewave >> plotter;
    sinewave >> plotter;
}

```

2.3.3 Wave Properties

Oscillators are defined by their **shape**, **period**, **skew**, **frequency**, **amplitude**, and **phase**. Let us explore these properties and their corresponding functions:

- **shape()**: Sets the shape of the wave (SQUARE, TRIANGLE or SINE).
- **period()**: Sets the duration of one cycle in seconds.
- **skew()**: Controls the balance between the rising and falling portions of the wave cycle (in range [0, 1]). For each wave type, this property has a specific effect:
 - For the SQUARE wave, it adjusts the duty cycle (the ratio of ON to OFF time).
 - For the TRIANGLE wave, it skews the wave towards a sawtooth (skew = 0) or inverted sawtooth (skew = 1).
 - For the SINE wave, it shifts the inflection points of the wave, altering its symmetry.
- **frequency()**: Inverse of period; sets the cycles per second (Hz).
- **bpm()**: Alternative way to set the frequency using beats per minute (BPM).
- **phase()**: Sets the initial point in the wave cycle (as % of period) (in range [0, 1]).
- **amplitude()**: Sets the peak level of the wave (as % of max) (in range [0, 1]);

Initializing Properties

The period and skew of a waveform can be initialized when the unit is created.

Example: Assign period and skew when creating the unit:

```

#include <Plaquette.h>

Wave wave1;           // Default (square wave, 1 second period, 50% skew)
Wave wave1(TRIANGLE); // Triangle wave with default period and skew
Wave wave2(SINE, 2.0); // Sine wave with 2 seconds period and default skew
Wave wave3(SQUARE, 3.0, 0.1); // Square wave, 3 seconds period, 10% skew

```

Other properties are typically initialized in the `begin()` to build a specific waveform. It is also common to initialize period and skew in the same way for more expressive code.

Example: Assign some properties of a wave at program startup:

```
#include <Plaquette.h>

Wave wave;
Plotter plotter(115200);

void begin() {
  wave.shape(TRIANGLE); // triangle wave
  wave.frequency(2);    // 2 Hz
  wave.skew(0.9);       // skew 90%
  wave.phaseShift(0.1); // dephased by 10% of period
  wave.amplitude(0.5);  // 50% amplitude
}

void step() {
  wave >> plotter; // send wave value to plotter
}
```

Changing Properties During Runtime

Properties can also be changed in real-time in the `step()` function to create interactive or evolutive effects.

Example: Control the skew of the waves using the potentiometer:

```
#include <Plaquette.h>

AnalogIn pot(A0); // Potentiometer input

Wave square(SQUARE, 1.0);
Wave triangle(TRIANGLE, 1.0);
Wave sine(SINE, 1.0);

// Create plotter with baud and optional labels for multiple signals.
Plotter plotter(115200, "square,triangle,sine"); // First value sent to plotter labeled
↳ as "square", second as "triangle", etc.

void step() {
  // Assign new skew value.
  square.skew(pot);
  triangle.skew(pot);
  sine.skew(pot);
  // Send the wave values to the plotter for visualization
  square >> plotter;
  triangle >> plotter;
  sine >> plotter;
}
```

Example: Control the period of the waves using the potentiometer. Necessitates remapping potentiometer value to appropriate ranges.

```

#include <Plaquette.h>

AnalogIn pot(A0); // Potentiometer input

Wave square(SQUARE, 1.0);
Wave triangle(TRIANGLE, 1.0);
Wave sine(SINE, 1.0);

// Create plotter with baud and optional labels for multiple signals.
Plotter plotter(115200, "square,triangle,sine"); // First value sent to plotter labeled
↳ as "square", second as "triangle", etc.

void step() {
    // Read new period value.
    float newPeriod = pot.mapTo(0.5, 5); // Map to 0.5-5 seconds period
    // Assign new period value.
    square.period(newPeriod);
    triangle.period(newPeriod);
    sine.period(newPeriod);
    // Send wave values to the plotter for visualization
    square >> plotter;
    triangle >> plotter;
    sine >> plotter;
}

```

Try using the potentiometer to control different wave properties and visualize the result using the Serial Plotter.

Accessors and Mutators

All properties in wave units have two variants:

- A **mutator** variant allowing to change the value of the property. Example: `wave.period(3.0);`.
- An **accessor** read-only variant that returns the current value of the property. Example: `float x = wave.period();`

Tip: This naming convention is a standard in Plaquette and you will find it in other units as well.

Example: Increase the wave's period by one second each time the button is pressed:

```

#include <Plaquette.h>

DigitalIn button(2, INTERNAL_PULLUP); // Button input

Wave wave(TRIANGLE, 1.0); // Wave with initial 1 second period

Plotter plotter(115200); // Plotter with 115200 baud rate

void step() {
    if (button.rose()) {
        wave.period( wave.period() + 1 ); // Set period to current period plus one
    }
}

```

(continues on next page)

(continued from previous page)

```

    wave >> plotter // send wave to plotter
}

```

2.3.4 Wave Addition

Adding waves together allows for the creation of complex and dynamic waveforms. By superimposing multiple signals, you can simulate natural phenomena, generate rhythmic patterns, or create rich textures for artistic applications. In Plaqueette, wave addition is as simple as computing the average value of different waves.

One compelling example of wave addition is simulating a **heartbeat**. A heartbeat typically has two peaks: a stronger primary beat followed by a softer secondary beat. This can be achieved by adding two waves with different amplitudes and timings.

Example: Heartbeat simulation. This example uses two **SINE wave** units: one for the primary beat, and one for the secondary beat. The `bpm()` function sets the frequency of the waves in beats per minute.

```

#include <Plaqueette.h>

Wave primary(SINE); // Main heartbeat wave
Wave secondary(SINE); // Secondary beat
AnalogOut led(9); // LED for visualizing the heartbeat

Plotter plotter(115200); // plotter with 115200 baudrate

void begin() {
    primary.bpm(80); // Set primary beat to 80 beats per minute
    secondary.bpm(2*primary.bpm()); // Set secondary beat to twice primary BPM
    secondary.amplitude(0.8); // Secondary beat is less strong
}

void step() {
    float heartBeat = (primary + secondary) / 2; // Combine and normalize waves
    heartBeat >> led; // Drive LED with combined signal
    heartBeat >> plotter; // Stream the combined wave for visualization
}

```

In this simulation, the `primary` sine wave provides the dominant rhythm, while the `secondary` sine wave introduces a softer, complementary pulse. The resulting waveform mimics the double-thump pattern of a human heartbeat.

Try experimenting with different wave types, amplitudes, and frequencies to see how the combined waveform changes. Try adding a third wave, making sure you divide the result by 3 instead of 2. Wave addition opens up endless possibilities for creating expressive and engaging outputs.

2.3.5 Modulation

Modulation involves using one oscillator to influence the properties of another, creating rich and dynamic effects. For example, a slower wave (also called a **Low-Frequency Oscillator (LFO)**) can modulate the frequency, phase, period, amplitude, or skew of a faster wave.

Example: Modulate the frequency of a sine wave with a triangle wave:

```
#include <Plaquette.h>

Wave modulator(TRIANGLE, 10.0); // LFO (10 seconds period)
Wave sine(SINE); // Main wave
AnalogOut led(9); // LED output

Plotter plotter(115200); // plotter object for visualizaiton

void step() {
  sine.frequency(modulator.mapTo(1.0, 10.0)); // Modulate frequency between 1 and 10 Hz
  sine >> led; // Drive LED with modulated sine wave
  sine >> plotter; // Stream the modulated wave
}
```

2.3.6 Adding Noise with randomFloat()

While oscillators are incredibly useful for generating regular and predictable waveforms, there are times when you may want to introduce randomness to add a sense of natural variation or lifelike behavior. Plaquette provides the `randomFloat()` function, which is a powerful tool for generating random values.

Warning: Avoid using Arduino's `random()` function as it returns integer numbers instead of floating-point numbers.

The `randomFloat()` function can be used in several ways:

- `randomFloat()` generates a random float between 0.0 and 1.0.
- `randomFloat(max)` generates a random float between 0.0 and `max`.
- `randomFloat(min, max)` generates a random float between `min` and `max`.

These random values can be used to add noise directly to a signal.

Example: Add noise to a sine wave.

```
#include <Plaquette.h>

Wave wave(SINE, 1.0); // Base waveform
AnalogOut led(9); // LED output

Plotter plotter(115200); // plotter object for visualization

void step() {
  float noise = randomFloat(-0.1, 0.1); // Generate noise value in [-0.1, 0.1]
  float noisyWave = wave + noise; // Compute sine value + noise
  noisyWave >> led; // Drive LED with noisy sine wave
}
```

(continues on next page)

(continued from previous page)

```
noisyWave >> plotter; // Stream the noisy sine wave
}
```

These random values can also be used to modify properties such as amplitude, frequency, skew, or phase.

Example: Update the wave's period according to a random walk. The potentiometer controls the amount of noise.

```
#include <Plaqueette.h>

AnalogIn pot(A0); // Potentiometer input
Wave wave(SINE, 1.0); // Wave with initial period of 1 second
AnalogOut led(9); // LED output

Plotter plotter(115200);

void step() {
    float noise = randomFloat(-pot, pot); // Generate noise according to potentiometer
    ↪value
    wave.period( wave.period() + noise ); // Add noise to period
    wave >> led; // Drive LED with noisy sine wave
    wave >> plotter // Stream the sine wave to plotter for visualization
}
```

Example: Introduce randomness to the frequency of a triangle wave. Frequency updated on each push of the button.

```
#include <Plaqueette.h>

DigitalIn button(2, INTERNAL_PULLUP); // Button input
Wave wave(TRIANGLE); // Wave with default properties
AnalogOut led(9); // LED output

Plotter plotter(115200); // plotter object for visualization

void begin() {
    button.debounce(); // Debounce button
    wave.frequency(5.0); // Start at 5 Hz
}

void step() {
    if (button.rose()) {
        wave.frequency(randomFloat(4.0, 6.0)); // Random frequency between 4 and 6 Hz
    }
    wave >> plotter; // Stream the wave for visualization
}
```

Randomness can also be combined with modulation to create highly dynamic and expressive behaviors. Experiment with adding random noise to various properties and observe the effects using the Serial Plotter. Try to simulate a natural phenomena like a flickering flame or a lightning bolt.

2.3.7 Timing Functions

Oscillators offer various timing functions to control their behavior:

- **start()**: Starts/restarts the oscillator.
- **stop()**: Stops it and resets it.
- **pause()**: Pauses the wave at its current point.
- **resume()**: Resumes from the paused point.
- **togglePause()**: Toggles between paused and running states.
- **isRunning()**: Returns whether the oscillator is active.
- **setTime()**: Sets the current phase of the oscillator based on absolute time (in seconds).

Example: Use the button to start and stop the wave:

```
#include <Plaquette.h>

DigitalIn button(2, INTERNAL_PULLUP); // Button input
Wave sine(SINE); // Wave with default properties
AnalogOut led(9); // LED output

Plotter plotter(115200); // Plotter for signal visualization

void begin() {
    sine.frequency(2.0); // Initialize frequency to 2 Hz
}

void step() {
    if (button.rose()) {
        sine.togglePause(); // Pause or resume the wave
    }
    sine >> led; // Drive LED with sine wave
    sine >> plotter; // Stream the wave for visualization
}
```

2.3.8 Phase Shifting with shiftBy()

The `shiftBy()` function allows you to offset the phase of an oscillator relative to its current position and returns the value of the dephased wave. This is useful for creating complex, synchronized patterns.

Example: Shift the phase of a sine wave:

```
#include <Plaquette.h>

Wave wave(SINE, 5.0); // Sine wave with 5 seconds period
Plotter plotter(115200); // plotter for signal visualization

void step() {
    // Print shifted values separated by white spaces.
    print(wave); print(" "); // 0% shift
    print(wave.shiftBy(0.25)); print(" "); // 25% shift
}
```

(continues on next page)

(continued from previous page)

```
print(wave.shiftBy(0.5)); print(" "); // 50% shift
println(wave.shiftBy(0.75)); // 75% shift
}
```

2.3.9 Conclusion

Oscillators are powerful tools for creating dynamic, expressive systems. By combining their waveforms, timing functions, and phase-shifting capabilities, you can achieve intricate and synchronized behaviors. Modulation and randomness add another layer of complexity, enabling you to create engaging and responsive media systems. Explore these features in Plaqueette and see how waves can bring your projects to life.

2.4 Working with Time

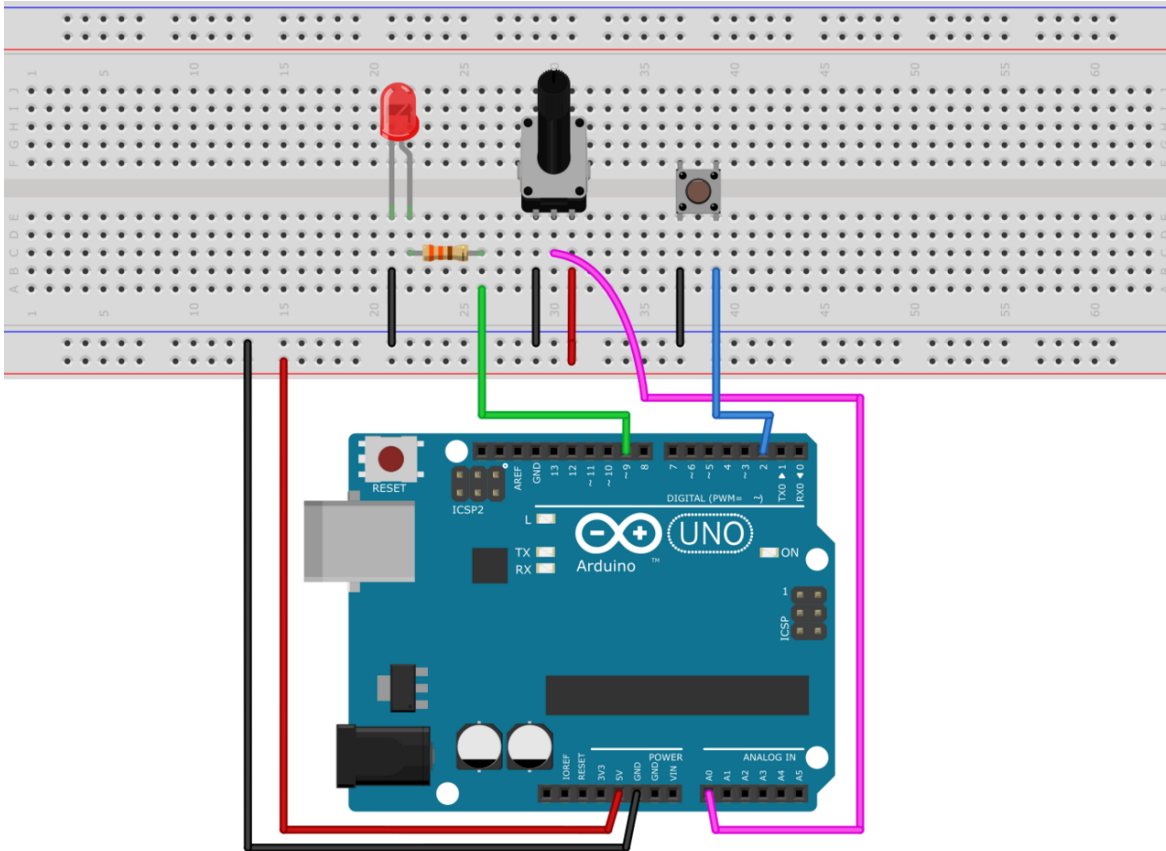
In interactive systems, **timing** plays a crucial role in controlling the flow of events. Whether you are counting time, triggering events, or generating periodic signals, Plaqueette provides powerful tools to manage time effectively. In this section, we will explore timing functions and units, such as *seconds()*, *Chronometer*, *Alarm*, *Metronome*, and *Ramp*.

Timing is the backbone of interactive systems. By understanding and leveraging Plaqueette's timing tools, you can create precise, dynamic, creative projects that respond in real-time to various inputs and conditions.

Note: To follow along with the examples, set up a simple circuit:

- A **potentiometer** connected to A0 to control properties dynamically.
- A **button** connected to pin 2 with an internal pull-up resistor to trigger actions.
- An **LED** connected to pin 9 (PWM capable) through a 330 Ω resistor.

It is strongly advised to use the *Serial Plotter* to visualize the signals.



Warning: The value returned by timing units and functions are approximations. They are good enough for most creative applications. They should, however, not be used as a substitute for a real-time clock in applications requiring high precision, especially over long periods of time. For example, on an Arduino Uno, a drift may be experienced of up to 10 seconds per hour, or 5 minutes per day.

2.4.1 Measuring Absolute Time with seconds()

The most fundamental timing functionality in Plaquette is the `seconds()` function. As its name suggests, it simply returns the elapsed time in seconds *since the program started running*. This is very useful for measuring durations or triggering time-based events.

Example: Turn LED off after 3 seconds, then on again after 10 seconds.

```
#include <Plaquette.h>

DigitalOut led(LED_BUILTIN);

void begin() {
    led.on(); // Initialize LED as "on"
}
```

(continues on next page)

(continued from previous page)

```
void step() {
  if (seconds() >= 3) { // After 3 seconds: turn LED off
    led.off();
  } else if (seconds() >= 10) { // After 10 seconds: turn LED on
    led.on();
  }
}
```

While `seconds()` provides a simple and effective way to measure time, it is inherently limited in scope. It measures only the elapsed time since the program started, functioning as a continuously increasing global counter that cannot be reset or adapted for specific events. This makes it inflexible when we need to measure time between arbitrary points or manage multiple independent timing events. For precise control and event-specific timing operations such as starting, stopping, resetting, or tracking multiple durations simultaneously, we need more refined timing instruments.

2.4.2 Timing Units

Plaqueette offers a core set of specialized units to simplify common timing tasks:

- *Chronometer*: Measures elapsed time between events
- *Alarm*: Activates after a specific duration
- *Metronome*: Generates periodic pulses
- *Ramp*: Creates smooth transitions

Danger: Timing units deal with time and events without interrupting the main processing loop. Users should avoid blocking processes such as `delay()` and `delayMicroseconds()` and when using Plaqueette.

Let us dive into these units and see what each one of them has to offer.

2.4.3 Keeping Track of Time with Chronometer

While `seconds()` can only give you the time since the start of the program, the *Chronometer* unit allows you to measure the time elapsed since it was started, like a real-life stopwatch. It is your basic building block for creating responsive systems where timing matters.

Chronometers are particularly useful for scenarios where the duration of an action determines its outcome. For instance, measuring how long a button is pressed can enable a system to interpret short and long presses differently.

Example: Changes LED intensity depending on how long button was pressed.

```
#include <Plaqueette.h>

DigitalIn button(2, INTERNAL_PULLUP); // Button input
AnalogOut led(9); // LED output
Monitor monitor(115200); // open a serial monitor
Chronometer chrono; // Chronometer measuring button press duration

void begin() {
  button.debounce(); // Debounce button
  led.off(); // Initialize led as "off"
```

(continues on next page)

(continued from previous page)

```

}

void step() {
  if (button.rose()) {
    chrono.start(); // Start the timer when button is pressed
  }

  else if (button.fell()) {
    // Converts chronometer time to LED intensity over a range of 10 seconds
    float ledValue = mapTo01(chrono, 0, 10); // Maps from 0-10 seconds to [0, 1] range
    ledValue >> led;
    chrono.stop(); // Stops/resets the timer when button is released
  }

  println(chrono); // Prints value of chrono to the serial monitor.
}

```

The *Chronometer* is great for counting time. In many scenarios, however, you want to know whether you waited for a certain amount of time. The *Alarm* unit provides a convenient way to do so.

2.4.4 Scheduling with Alarm

Like a real-world alarm-clock, the *Alarm* unit starts “buzzing” after a predefined time. This **digital unit** is initialized with a certain duration. It outputs 0/false until it reaches its timeout; then, it starts “ringing” and outputs 1/true until it is stopped or restarted.

Once triggered, it can be stopped by calling its `stop()` function, or restarted by calling `start()`, making the unit ideal for implementing delayed responses or timed sequences.

Alarms can help manage actions that require specific timing, such as turning off a light after a certain duration or triggering an animation. Their flexibility makes them a powerful tool in time- based designs.

Example: Starts blinking an LED when we reach the alarm’s timeout. Pushing the button restarts the alarm, increasing its duration by 50% each time.

```

#include <Plaqueette.h>

DigitalOut led(LED_BUILTIN); // LED on built-in pin
DigitalIn button(2, INTERNAL_PULLUP); // Button input

Monitor monitor(115200); // Open a serial monitor

Wave blink(0.5); // Wave to blink LED when alarm is buzzing

Alarm alarm(2.0); // Alarm with 2s duration

void begin() {
  button.debounce(); // Debounce button
}

void step() {
  // Button: restart.
  if (button.rose()) { // Button pressed event

```

(continues on next page)

(continued from previous page)

```

    led.off();           // Turn off LED
    alarm.duration( alarm.duration() * 1.5 ); // Increase duration by 50%.
    alarm.start();       // Start alarm
}

// Alarm buzzing: blink LED.
if (alarm) {           // Check if alarm is buzzing
    blink >> led; // Send the squarewave to the LED to make it blink
}

println(alarm.progress()); // % progress of the alarm
}

```

2.4.5 Triggering Periodic Events with Metronome

While the *Alarm* unit is great for dealing with one-time events, there are many cases where an action needs to be triggered periodically. For such use cases, Plaquette provides the *Metronome* unit which sends a periodic pulse or “bang”. In other words, it acts like an *Alarm* that gets restarted as soon as it starts buzzing. It also bears some resemblance with *wave units*.

Periodic actions are at the core of interactive systems, whether you are blinking an LED or synchronizing motor movements. The *Metronome* provides a straightforward way to create these kinds of repetitions.

Example: Blink an LED using a Metronome:

```

#include <Plaquette.h>

DigitalOut led(LED_BUILTIN); // LED on built-in pin
Metronome metro(1.0); // Metronome with period of 1 second

void step() {
    if (metro) { // The unit will be true for a single frame every time it triggers
        led.toggle(); // Toggle LED on each pulse
    }
}

```

Metronome units can be used as a way to trigger different actions in parallel.

Example: Use multiple *Metronome* units to control different actions. One metronome toggles LED visibility, while another slower metronome accelerates blinking speed at each tick.

```

#include <Plaquette.h>

DigitalOut led(LED_BUILTIN); // LED on built-in pin
Wave blink(1.0); // Wave to blink the LED
Metronome metroToggle(2.0); // Metronome to toggle visibility
Metronome metroAccelerate(10.0); // Metronome to accelerate blink

boolean visible = true; // Flag to keep track of visibility

void step() {
    // Toggle visibility.
}

```

(continues on next page)

(continued from previous page)

```

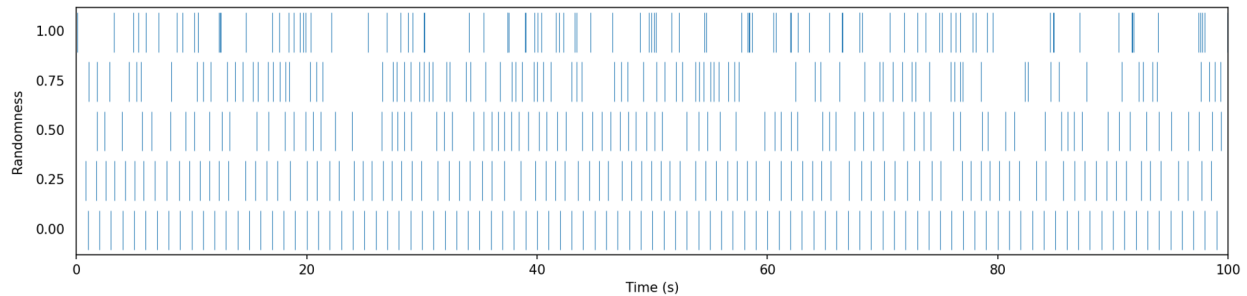
if (metroToggle) {
  visible = !visible; // Invert boolean value
}

// Accelerate blink.
if (metroAccelerate) {
  blink.frequency( blink.frequency() * 2 ); // Double frequency
}

// Activate LED depending on visibility status.
if (visible)
  blink >> led;
else
  led.off();
}

```

Tip: You can make a metronome feel more natural by adding randomness. Simply call `jitter(amount)` where `amount` is between 0 (perfectly regular) and 1 (highly irregular). This is useful when you want a rhythm that feels organic rather than mechanical, like breathing, footsteps, or blinking lights in nature. See the *Metronome* unit documentation for more information.



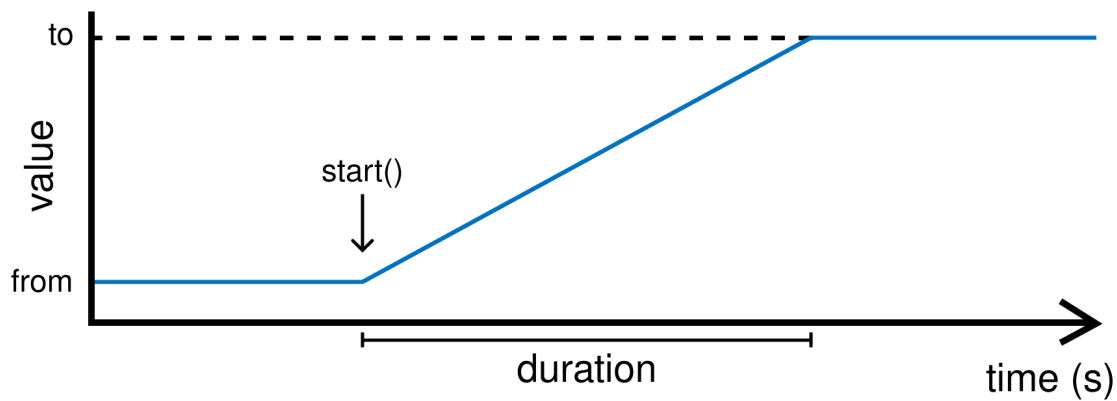
2.4.6 Creating Smooth Transitions with Ramp

Ramps are a cornerstone of creative expression. Unlike *Wave* units, which generate periodic signals, ramps interpolate from one value to another over a specific duration or at a specific speed. The *Ramp* unit in Plaquette provides a flexible and powerful way to animate visual elements such as LEDs or physical components such as motors in a natural manner, allowing the creation of rich, dynamic, evolving experiences.

Tip: We strongly recommend to use the Serial Plotter to visualize the ramp values in the following examples.

Basic Usage

Like *Alarm* units, ramps can be restarted by calling their `start()` function. By default, they will ramp between 0 and 1.



Example: Gradually increases an LED brightness over 5 seconds every time a button is pressed.

```
#include <Plaquette.h>

DigitalIn button(2, INTERNAL_PULLUP); // Button input
AnalogOut led(9); // LED output
Ramp ramp(5.0); // Ramp with 5 seconds duration
Plotter plotter(115200); // Initialize a serial plotter

void begin() {
  button.debounce(); // Debounce the button
  ramp.start(); // Initial ramp startup
}

void step() {
  if (button.rose()) {
    ramp.start(); // Restart ramp
  }

  ramp >> led; // Use ramp value to control LED brightness
  ramp >> plotter; // Visualize ramp value with the Serial Plotter
}
```

Try changing the behavior of the ramp to rather go from 1 to 0 by calling the `fromTo()` function and see how that changes the behavior of the ramp:

```
void begin() {
  ramp.fromTo(1.0, 0.0); // Ramp from one to zero
  ramp.start();
}
```

Flexible Ranges

Ramps are not restricted to the range [0, 1]. You can define any starting and ending values, making ramps very useful for various applications such as changing properties of waves, controlling the angle of a servo motor, adjusting the color of a RGB LED, etc.

Example: Gradually increases an LED brightness over a 5 seconds period every time a button is pressed. The potentiometer sets the maximum LED value to attain.

```
#include <Plaquette.h>

DigitalIn button(2, INTERNAL_PULLUP); // Button input
AnalogIn pot(A0); // Potentiometer input
AnalogOut led(9); // LED output
Ramp ramp(5.0); // Ramp with 5 seconds duration

Plotter plotter(115200); // Open a serial plotter

void begin() {
  button.debounce(); // Debounce the button
}

void step() {
  if (button.rose()) {
    ramp.to(pot); // Set ramp goal to value of potentiometer
    ramp.start(); // Restart ramp
  }

  ramp >> led; // Use ramp value to control LED brightness
  ramp >> plotter; // Visualize ramp value
}
```

Try adjusting the potentiometer to different positions and then pressing the button to see the effect.

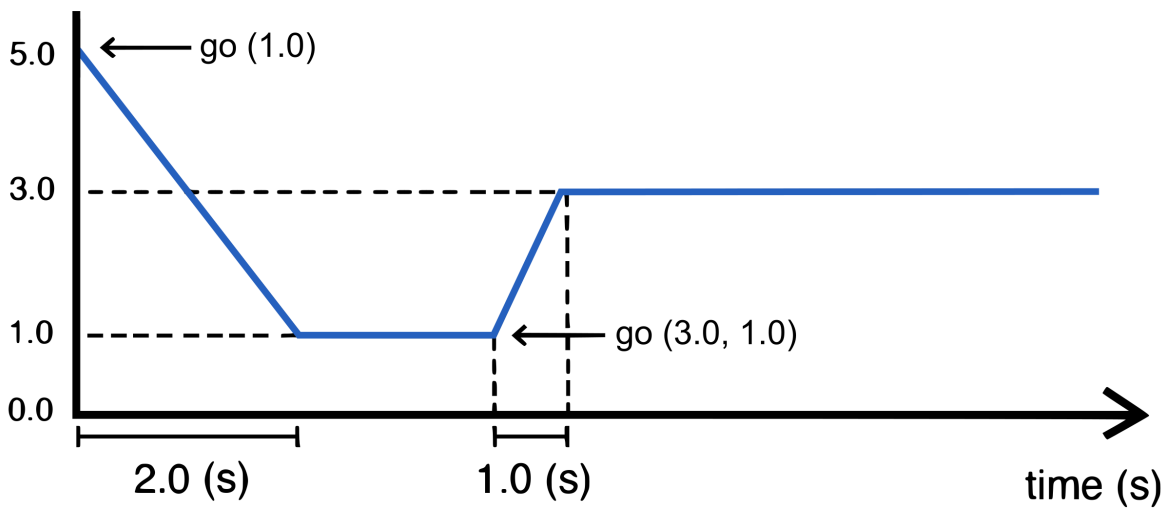
Notice how we are using function `to()` to set the goal of the ramp. The starting value is left unchanged at zero (default value). To change the starting value while preserving the goal value, use function `from()` instead. See what happens if you change the call `ramp.to(pot)` to use `from()` instead:

```
ramp.from(pot); // Set ramp goal to value of potentiometer
```

Dynamic Control with go()

A common scenario in creative applications is to respond to events by changing a value such as the position of a servomotor, the color of a RGB LED, or the volume of a sound. Ramps are often used in these cases to create smooth transitions instead of abrupt changes.

The `go()` function provides a simple way to immediately launch a ramp from the current value to another, or simply from the current value towards a new goal.



Example: Control blinking frequency using a button. Each time the button is pushed, a new frequency is chosen randomly and the ramp smoothly goes to the new frequency.

```
#include <Plaquette.h>

DigitalIn button(2, INTERNAL_PULLUP); // Button input
AnalogOut led(9); // LED output
Ramp ramp(5.0); // Ramp with 5 seconds duration
Wave wave(TRIANGLE, 1.0); // Oscillator

Plotter plotter(115200); // Open a serial plotter

void begin() {
    wave.skew(1.0); // Sawtooth wave
    wave.bpm(100); // Initial BPM
    button.debounce(); // Debounce button
}

void step() {
    if (button.rose()) {
        // Set target BPM to random value
        float targetBpm = randomFloat(60, 200);
        ramp.go(targetBpm); // Launch ramp
    }
}
```

(continues on next page)

(continued from previous page)

```

}

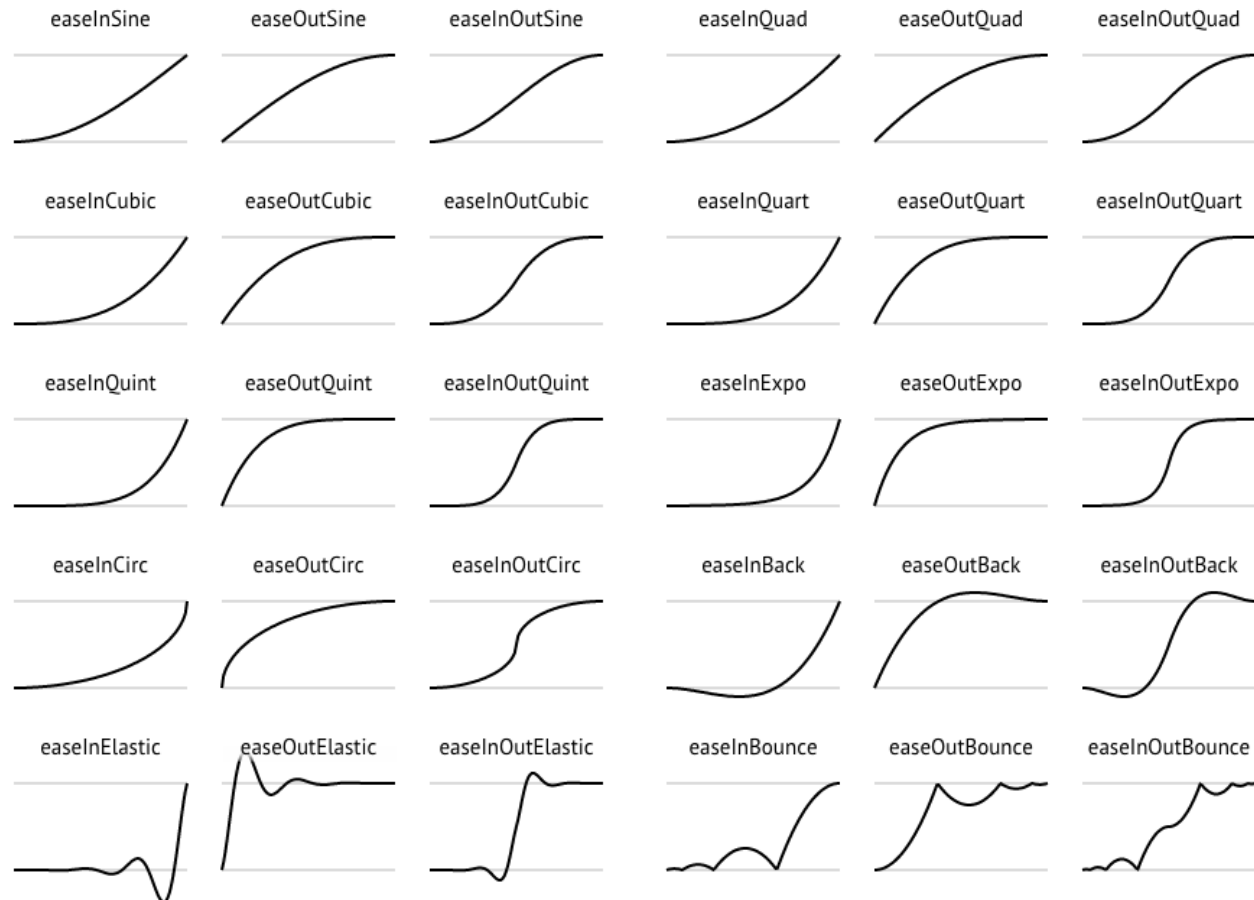
wave.bpm(ramp); // Use ramp value to adjust BPM of wave

wave >> led;    // Oscillate LED
ramp >> plotter; // Visualize ramp value with the Serial Plotter
}
    
```

Note: Ramps provide multiple ways to call `go()` depending on the desired behavior, including specifying starting value and duration on the spot. For more details, please consult the [Ramp unit's reference](#).

Generating Expressive Effects with Easing Functions

Ramp supports *easing function*, providing many different ways to generate expressive effects. Easing functions add acceleration or deceleration effects to ramp transitions, making them feel more natural and lifelike.



Example: Use easing to create a smooth LED fade repeatedly:

```

#include <Plaquette.h>

AnalogOut led(9); // LED output
    
```

(continues on next page)

(continued from previous page)

```
Ramp ramp(3.0); // Ramp with 3 seconds duration

Plotter plotter(115200); // Open a serial plotter

void begin() {
    ramp.easing(easeInOutQuad); // Apply an easing function
    ramp.start();
}

void step() {
    if (ramp.isFinished())
        ramp.start(); // Restart the ramp with the easing effect
}

ramp >> led; // Use the ramp's value to control the LED brightness
ramp >> plotter; // Visualize ramp value with the Serial Plotter
}
```

Try experimenting with different easing functions and observe the results on the LED and using the Serial Plotter. Easing can transform mechanical transitions into expressive animations, giving your projects character.

Operational Modes: Duration vs Speed

By default, ramps transition between two values over a definite duration. However, there are many scenarios where this is not the appropriate behavior. For example, one might want to move a servomotor at a specific angular speed: ramping over 10 degrees should take much less time than a 90 degrees transition.

Ramps accomodate these different use cases by providing two modes of operation:

- In **duration mode** (default) the ramp transitions between values over a fixed number of seconds.
- In **speed mode** the ramp moves at a constant rate, defined in value change per second.

Example: Compare duration and speed modes. Ramp values can be visualized using the Serial Plotter.

```
#include <Plaqueette.h>

DigitalIn button(2, INTERNAL_PULLUP); // Button input
Ramp rampDuration; // Ramp operating in duration mode
Ramp rampSpeed; // Ramp operating in speed mode

Plotter plotter(115200); // Open a serial plotter

void begin() {
    rampDuration.duration(5.0); // Duration: 5 seconds
    rampSpeed.speed(5.0); // Rate of change: 5 per second
    button.debounce(); // Debounce button
}

void step() {
    if (button.rose()) {
        // Both ramps go to random target value.
        float targetValue = randomFloat(-20, 20);
    }
}
```

(continues on next page)

(continued from previous page)

```

    rampDuration.go(targetValue);
    rampSpeed.go(targetValue);
}

// Visualize and compare ramps with the Serial Plotter
rampWithDuration >> plotter;
rampWithSpeed >> plotter;
}

```

Tip: To switch between modes, you can simply call the `duration(value)` or `speed(value)` functions with a target duration or speed (recommended). Alternatively, you can change mode by calling `mode(RAMP_DURATION)` or `mode(RAMP_SPEED)`, in which case the duration or speed will be computed based on the ramp's current properties (ie. duration/speed, starting, and target values).

2.4.7 Combining Timing Units

Plaquette allows you to combine different timing units to achieve complex behaviors while keeping your workflow clear and intuitive. For instance, you can use a *Metronome* to repeatedly trigger a *Ramp* or synchronize multiple timing units.

Example: Use a Metronome to trigger a Ramp at regular intervals:

```

#include <Plaquette.h>

Metronome metro(10.0); // Trigger every 10 seconds
Ramp ramp(3.0); // Ramp with 3 seconds duration
AnalogOut led(9); // LED output

Plotter plotter(115200); // Open a serial monitor

void step() {
  if (metro) {
    ramp.start(); // Start the ramp each time the metronome triggers
  }

  ramp >> led; // Use the ramp's value to control the LED brightness
  ramp >> plotter; // Stream the ramp's value for visualization
}

```

Combining timing units unlocks an even greater range of creative possibilities. Use these tools to design intricate behaviors, smooth transitions, and expressive animations in your projects.

2.4.8 Conclusion

Timing is an essential aspect of creating interactive and dynamic systems, and Plaquette provides an intuitive set of tools to make this process seamless. From measuring durations with the *Chronometer*, to triggering events with the *Alarm*, generating rhythmic patterns with the *Metronome*, and creating smooth transitions with the *Ramp*, each timing unit offers unique possibilities.

The flexibility of these tools allows for countless creative applications, whether you are developing reactive systems, synchronizing events, or designing natural and expressive transitions. By combining these units, you can build intricate behaviors that bring your projects to life.

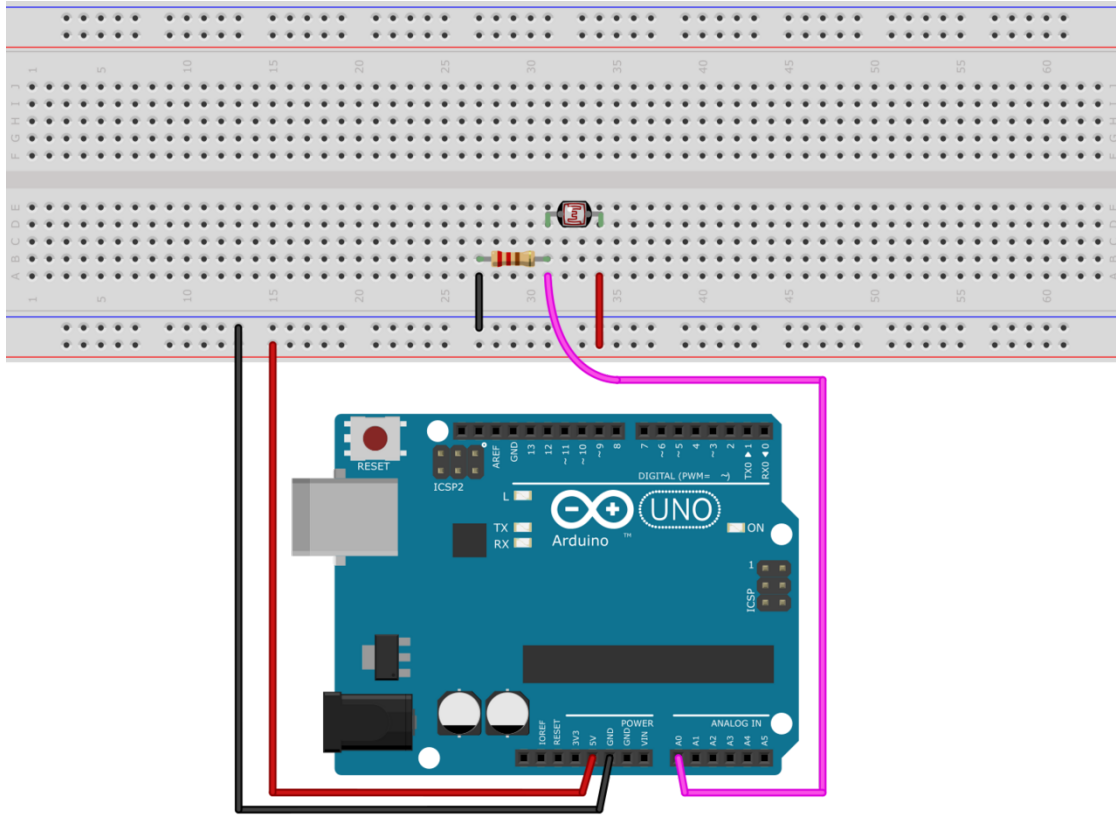
2.5 Regularizing Signals

Plaquette provides expressive, automated, and robust ways to deal with signals for interactive design using **regularization filters** such as smoothing, min-max scaling, and normalization.

2.5.1 Direct Input-to-Output

Let's review briefly how to handle raw *input and output* signals in Plaquette. We will be using an analog sensor such as a photoresistor for this example.

Note: In order to build this circuit, you will need to create a simple *voltage divider circuit*. Connect the photoresistor between the ground (GND) and the analog input pin (A0). Then connect a fixed resistor with value matching your photoresistor between analog input pin and +5V (Vcc). For example, for a 1k Ω - 10k Ω photoresistor you could use a fixed resistor of about 5.5k Ω .



Here is a basic Arduino sketch that allows changing the value of an output LED using an input photocell:

```
// The photocell analog pin.
int photoCellPin = A0;

// The output analog LED pin.
int ledPin = 9;

void setup() {
  // Initialize pins.
  pinMode(photoCellPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Read value from photocell (between 0 and 1023).
  int value = analogRead(photoCellPin);

  // Write value to LED (between 0 and 255).
  analogWrite(ledPin, value / 4);
}
```

As explained in *Why Plaqueette?* section, this code forces the programmer needs to remember low-level information concerning the ranges of raw number values (1023, 255, ...) Furthermore, it fails to adapt to changing conditions such as the range of the ambient light, which might evolve over the course of the day.

Let's see how Plaquette can help us to create more expressive code by using inputs and outputs signals rather than meaningless raw numbers.

To begin, we will re-implement the example above using Plaquette units.

First, let's define our input photocell on pin A0 using an *AnalogIn* unit:

```
AnalogIn photoCell(A0);
```

Then, let's add an output analog LED on pin 9 using an *AnalogOut* unit:

```
AnalogOut led(9);
```

If we want to directly control the value of the LED from the value of the photocell, all we need to do is to send the photocell's value to the led. The easiest way to do so is by using the `>>` operator:

```
photoCell >> led;
```

The complete Plaquette code looks like this:

```
#include <Plaquette.h> // include the Plaquette library

// Create input unit for photocell.
AnalogIn photoCell(A0);

// Create output unit for LED.
AnalogOut led(9);

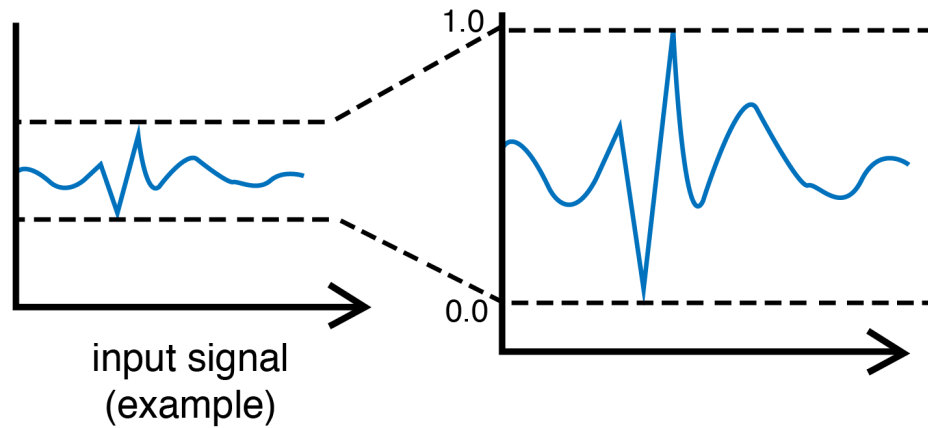
// NOTE: Since in this case there is nothing to do in begin(), we don't need to include it

void step() {
    // Send photocell value directly to the LED.
    photoCell >> led;
}
```

2.5.2 Getting the Full Range of a Signal

If we run this program, we will likely notice that the LED brightness will not span the full range from 0% to 100%. That's because depending on ambient lighting conditions, the photocell's values will not move across the full spectrum of possibility. For instance, in the dark, the photocell might range from 10% to 50%, while in full daylight, it might range between 70% and 95%.

In order to resolve this issue, we need to **regularize** the photocell's signal. We can do so using a filtering unit such as a *MinMaxScaler*. This unit automatically keeps track of the minimum and maximum values of the incoming signal over time (for example, 10% and 50%) and remaps them into a new interval of [0, 1] (ie., 0% to 100%).



To use this approach, create the unit:

```
MinMaxScaler regularizer;
```

... and then *insert it* in the pipeline between the incoming photocell signal and the output LED:

```
photoCell >> regularizer >> led;
```

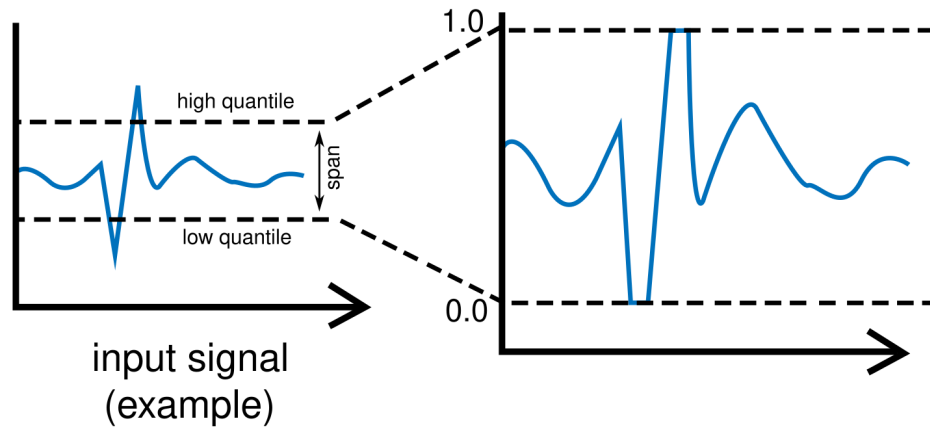
The above expression will do the following, in order:

1. Read the raw photocell value using the `photoCell` unit.
2. Send that raw value from the `photoCell` unit to the `regularizer` unit.
3. The `regularizer` unit updates itself if the value is a new extreme value (minimum or maximum).
4. The `regularizer` then remaps the raw photocell value to the full range of $[0, 1]$ and sends it to the `led` unit.
5. The `led` unit takes the input value in $[0, 1]$ and applies it to the intensity of the LED.

2.5.3 Handling Noisy or Unpredictable Signals

The *MinMaxScaler* is great when your signal behaves in a fairly stable way. It learns the smallest and largest values it has ever seen, then stretches everything into a clean $[0, 1]$ range. However, some sensors behave in a much more irregular way. They might produce short spikes, sudden jumps, or occasional “glitches.” These rare events can throw off the *MinMaxScaler* by forcing it to update its minimum or maximum too aggressively.

For such situations, Plaquette provides the *RobustScaler*. Instead of focusing on *extreme* values, the *RobustScaler* concentrates on what the signal is doing *most of the time*. It keeps track of a “typical low” and a “typical high” value, ignoring rare spikes, and maps the signal between 0 and 1 based on this typical span.



You can create a RobustScaler the same way as a MinMaxScaler:

```
RobustScaler regularizer;
```

Then:

```
void step() {
  photoCell >> regularizer >> led;
}
```

The output still stays within $[0, 1]$, but the regularizer behaves more calmly and is less affected by sudden unexpected events.

If needed, you can adjust how tolerant it is to spikes by setting its `span()`. A larger span makes it more robust but also clamps extreme values more strongly. A smaller span makes it more sensitive but less stable.

2.5.4 Reacting to Signal Changes

Remember our example from *earlier*, where we were trying to detect high-valued signals using arbitrary numbers?

```
if (value > 716)
  // do something
```

Suppose that instead of directly controlling the LED value based on the photocell's value, we instead want to use sudden changes in the photocell's value to trigger the on/off state of the LED? In other words, we would like to work with the **peaks** in the incoming signal (such as when someone points a light source towards the photocell).

One way to do so would be to pick a threshold in the regularized signal above which we would react to the light source. Let's say that we will react when the signal goes above 70%. The code of the `step()` function now becomes:


```
void step() {
  photoCell >> regularizer;
  if (regularizer > 0.7)
    1 >> led;
  else
    0 >> led;
}
```

... which can be more compactly rewritten by sending directly the conditional expression (`regularizer > 0.7`) to the output LED:

```
void step() {
  photoCell >> regularizer;
  (regularizer > 0.7) >> led;
}
```

2.5.5 Adapting to Changing Conditions

So far so good. The number 0.7 is still a bit of an arbitrary, hand-picked number, but it makes more sense than 716 because it refers to a more human-understandable concept (70% instead of 716 / 1023). However, this approach will still be sensitive to changes in the ambient light, and behave differently under different light conditions (for example, it might work as expected in the morning, but work less well in the late afternoon when the sun starts to go down.)

One thing we could do would be to make sure that our regularization unit adapts to changing conditions. In order to do this, rather than having our MinMaxScaler remap values depending on every single incoming value, we can have it adapt over a **time window**. This will allow our regularizer to slowly forget what it has learned, and reprogram itself after a certain amount of time has passed.

This can be accomplished by calling the `timeWindow(seconds)` function inside the `begin()` function:

```
void begin() {
  // Allow regularizer to adapt over an approximate period of 1 hour (3600 s).
  regularizer.timeWindow(3600.0f);
}
```

2.5.6 Normalizing Signals to Spot Extreme Values

The MinMaxScaler is a very useful unit for making sure signals stay within a [0, 1] range. However, it is not always the best for signal detection since it only accounts for extreme values (minimum and maximum), which makes it sensitive to rare events. Someone switching the lights on and off again rapidly might completely ruin the show.

A better alternative is the *Normalizer* unit, which regularizes incoming signals by normalizing them around a target **mean** by taking into account **standard deviation**. Once the data is normalized, extreme **outlier** values can be more easily and robustly detected based on how much they diverge from the mean.

Let's replace our MinMaxScaler by a Normalizer unit:

```
Normalizer regularizer;
```

... and use the `isHighOutlier()` function to find values that are higher than usual:

```
void step() {  
  photoCell >> regularizer;  
  regularizer.isHighOutlier(photoCell) >> led;  
}
```

Tip: By default, the `isHighOutlier()` function detects values that are more than 1.5 deviations from the mean. The function can be made more or less sensitive by adjusting the number of deviations (typically between 1.0 and 3.0). For example, `isHighOutlier(value, 1.2)` will be more sensitive, `isHighOutlier(value, 2.5)` will be less sensitive, and `isHighOutlier(value, 3.0)` will only respond to rarely-occurring extremes. While these numbers (1.2, 1.5, 2.5, etc.) still need to be hand-picked, they are much more robust than our 716 and even to our 0.7 number from earlier.

Here is a complete version of the code:

```
#include <Plaqueette.h> // include the Plaqueette library  
  
// Create input unit for photocell.  
AnalogIn photoCell(A0);  
  
// Create output unit for LED.  
AnalogOut led(9);  
  
// Create regularization object.  
Normalizer regularizer;  
  
// Initialize everything.  
void begin() {  
  // Allow regularizer to adapt over an approximate period of 1 hour (3600 s).  
  regularizer.timeWindow(3600.0f);  
}  
  
// Define frame-by-frame operations.  
void step() {  
  // Update regularizer with raw signal value.  
  photoCell >> regularizer;  
  
  // Detect outliers and send the value (1=true=outlier, 0=false=no outlier)  
  // directly to the LED.  
  regularizer.isHighOutlier(photoCell) >> led;  
}
```

2.5.7 Choosing the Right Regularizer for the Job

In creative media, each regularizer offers a different way of interpreting the same signal. A simple way to choose between them is to think about the kind of behaviour you want:

- Use **MinMaxScaler** when you want to **use the full expressive range** of the signal exactly as it occurs. It is a good choice when the signal's lowest and highest values are *meaningful* (not accidental glitches), and when you want outliers to have a noticeable effect. This makes it ideal for direct mappings where the natural span of the sensor should translate into the full motion of a parameter such as brightness, speed, position, or size.

- Use **RobustScaler** when you want **stable behaviour** even in the presence of noise, glitches, or unpredictable input. It focuses on the “typical” range of the signal and ignores rare spikes, making it suitable for real-world sensors with variability, jitter, or unpredictable environmental conditions.
- Use **Normalizer** when you want to understand **how far the signal deviates from its typical behaviour**, rather than how big or small it is in absolute terms. This is useful for detecting unusual events, bursts of activity, or expressive gestures that stand out relative to the usual pattern of the sensor.

Thinking about your signal in terms of **expressive range**, **stability**, or **deviation** will help you select the regularizer that best supports the interaction you are designing.

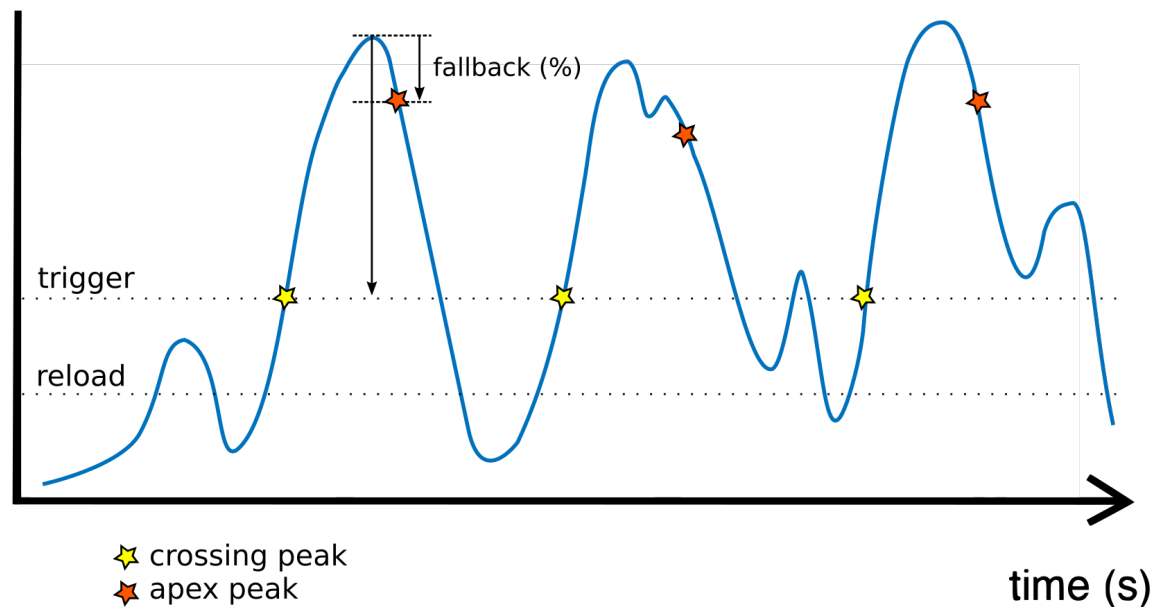
The table below summarizes when and why you would use each unit, and gives examples of typical signals where each is a good fit.

Regu- larizer	Best Used When ...	Pros & Cons	Example Signals
<i>Min- MaxS- caler</i>	Your signal has clear, stable limits and you want to remap it cleanly into the full [0, 1] range, often for direct control (sensor → brightness, speed, position, etc.).	Pros: • Very simple and intuitive • Great for direct, continuous mappings • Easy to reason about visually (“full sweep = 0% to 100%”) Cons: • Very sensitive to spikes and rare extreme values (outliers) • Sudden glitches can distort the usable range	• Photocells in controlled lighting • Distance sensors with bounded ranges • Knobs, sliders, faders • Flex sensors over limited movement arcs
<i>Ro- bustScaler</i>	Your signal is noisy, erratic, or unpredictable , and you want a stable result even when rare spikes occur. You care more about the “typical” range than about extremes.	Pros: • Tolerant to outliers • Ignores occasional glitches • Produces a stable [0, 1] range • <code>span()</code> adjusts robustness vs. sensitivity Cons: • Very extreme values may be clamped • Adapts more slowly than MinMaxScaler	• Physiological / biosignals (heart-rate, EMG, GSR, EEG amplitude) • Noisy microphones or piezo discs • Jittery accelerometers • Environmental sensors in public spaces
<i>Normal- izer</i>	You want to understand a signal in terms of how much it differs from its own average behaviour . Works best when the signal naturally clusters around a single central value (a “bell-shaped” or roughly normal distribution).	Pros: • Provides a clear measure of relative deviation from the mean • Tolerant to outliers • Very effective for identifying outliers or unusual events Cons: • Output is not limited to [0, 1] • Less suited for direct sensor → output mappings • Can be misleading for signals with multiple distinct operating states (e.g., empty room vs. crowded room)	• Photocells where the goal is to detect departures from normal light levels • Microphones used to detect unusual loudness relative to ambient noise • Motion sensors where gestures stand out against small baseline movement

2.5.8 Detecting Peaks

The outlier detection method is useful to find extreme values. However, it also comes with an important limitation. The `isHighOutlier()` and `isOutlierLow()` methods return `true` *as long as* the received value is considered to be an outlier, making these methods unsuitable for triggering instantaneous events, such as toggling the status of an LED, starting a sound event, activating a motor, etc.

The *PeakDetector* unit addresses this limitation. It is best used in combination with a Normalizer unit. We will use the default mode of the PeakDetector (PEAK_MAX): for a peak to be detected. In this mode, the signal will need to (1) cross a *trigger threshold* value (`triggerThreshold`); (2) reach its *apex* (max); and (3) *fall back* by a certain proportion (%) between the threshold and the apex (controlled by the `fallbackTolerance` parameter).



Building on the previous section for outlier detection, we will assign the PeakDetector's `triggerThreshold` to the value above which a value is considered to be a high outlier, which can be obtained by calling the Normalizer's function `highOutlierThreshold()`:

```
PeakDetector detector(normalizer.highOutlierThreshold());
```

Tip: As for the `isHighOutlier()` function, the `highOutlierThreshold()` function is set to return, by default, a threshold that is 1.5 standard deviations from the mean. The function can be made more or less sensitive by adjusting the number of deviations. For example, `highOutlierThreshold(1.2)` will be more sensitive, while `highOutlierThreshold(2.5)` will be less sensitive.

Finally, let's rewrite the `step()` function with our new peak detector, so that only when a **peak** is detected will the LED change state:

```
void step() {
  // Signal is normalized and sent to peak detector.
  sensor >> normalizer >> detector;
```

(continues on next page)

(continued from previous page)

```
// Toggle LED when peak detector triggers.
if (detector)
  led.toggle();
}
```

The PeakDetector unit offers many options to fine-tune the peak detection process. Please read the [full documentation of the unit](#) for details.

2.5.9 Conclusion

The Plaquette library simplifies signal processing for interactive design by abstracting low-level details and offering intuitive regularization tools like [MinMaxScaler](#) and [Normalizer](#). Combined with [PeakDetector](#) opens the way to deploy precise event-driven behaviors.

Plaquette’s ability to adapt to changing conditions ensures dynamic, robust systems while keeping code concise and expressive. By leveraging its modular architecture, users can streamline signal handling, improve scalability, and focus on innovation in signal-driven creative applications.

2.6 Managing Events

In interactive systems, many interesting actions happen at specific moments in time: a performer presses a button, a sensor crosses a threshold, a timer runs out, a metronome ticks. These fleeting moments are distinct from continuous signals flows like the brightness of a light or the position of a knob: they happen in an instant and are then gone.

In Plaquette, an **event** is an instantaneous condition that occurs on a unit for exactly one step: the push of a button, the tick of a metronome, the end of a timer, or a peak detection. Plaquette offers two different and complementary ways to work with events: **trigger conditions** and **callback functions**.

Trigger conditions are true for the single step when the event occurs. They are typically polled continuously inside the `step()` function using `if` statements. We have already been working with some of them in the [Inputs and Outputs](#) and [Working with Time](#) sections:

```
void step() {
  if (button.rose()) { ... }    // true only on the step when button is
  ↪ pressed
  if (metro) { ... }           // true only on the step when the metronome
  ↪ ticks
}
```

Callback functions are functions that you register once in `begin()`. They are called automatically when the event occurs, without needing to write `step()` logic.

```
void doSomething() { ... }    // user-defined function

void begin() {
  button.onRise(doSomething); // function doSomething() will be called on
  ↪ button press
  metro.onBang(doSomething);  // same, but triggered by metronome tick
}
```

Note: Registering a callback function is like giving someone instructions on what to do when a specific event happens. For example, imagine you are baking cookies and you set a timer. Instead of constantly watching the oven, you set the timer to “call you back” (in other words, alert you) when time is up, so you can take the cookies out.

Use **trigger conditions** when you have few events to manage and want to keep things simple: all your logic stays in `step()`, easy to read top-to-bottom, and you can combine events with other conditions (e.g. `if (button.rose() && sensor > 0.7)`).

Use **callback functions** when you have many events to manage or want to reuse the same action across different events: register them once in `begin()` and they will be called automatically so you don’t have to think about polling them.

2.6.1 Supported Events

This table lists all supported events, their trigger condition and callback form, and which units support them.

When	Trigger	Callback	Units
Unit emits a pulse	unit itself	<code>onBang(c)</code>	<i>Metronome, PeakDetector</i>
Digital value toggles (on/off)	<code>changed()</code>	<code>onChange(c)</code>	<i>Alarm, DigitalIn</i>
Digital value goes from on to off	<code>fell()</code>	<code>onFall(c)</code>	<i>Alarm, DigitalIn</i>
Timed process completes	<code>finished()</code>	<code>onFinish(c)</code>	<i>Alarm, Ramp</i>
Wave completes a full cycle	<code>passedPeriod()</code>	<code>onPassPeriod(c)</code>	<i>Wave</i>
Wave passes its skew point	<code>passedSkew()</code>	<code>onPassSkew(c)</code>	<i>Wave</i>
Digital value goes from off to on	<code>rose()</code>	<code>onRise(c)</code>	<i>Alarm, DigitalIn</i>
New data is available	<code>updated()</code>	<code>onUpdate(c)</code>	<i>TimeSliceField</i>

2.6.2 Using Trigger Conditions

Trigger functions are the simplest way to react to events. Call them inside `step()` with an `if` statement:

```
void step() {
  if (button.rose())           // button was just pressed
    led.toggle();

  if (alarm.finished())        // alarm just timed out
    println("Time is up!");

  if (metro)                   // metronome ticked
    println("Tick...");
}
```

Since triggers are only true for one step, they naturally detect transitions rather than ongoing states. You can combine them with logical operators and other conditions to create more complex behaviors:

```
void step() {
  // Only react to button press when sensor value is high enough.
  if (button.rose() && sensor > 0.7)
    led.toggle();

  // React to either button press or metronome tick.
  if (button.rose() || metro)
```

(continues on next page)

(continued from previous page)

```
println("Something happened!");
}
```

Warning: Since triggers are true for only one step, **they must be checked at every call to ``step()``** or they will be missed. A common mistake comes when nesting a trigger inside another trigger's `if` block:

```
void step() {
    // WRONG: button.rose() is only checked when metro fires,
    // so most button presses will be missed
    if (metro) {
        if (button.rose())
            led.toggle();
    }
}
```

In this example, `button.rose()` is only evaluated on the rare step when `metro` is true. On all other steps, the button press goes undetected. Instead, check both triggers independently.

2.6.3 Using Callback Functions

As your project grows and you start managing more events, the `step()` function can become cluttered with `if` statements, making event management confusing and error-prone. Callback functions offer a different approach: instead of checking for events yourself, you tell each unit what to do when an event happens, and Plaqueette automatically takes care of the rest. This keeps your code organized and ensures no event is ever missed.

Let us take an example where we want to react to the push of a button by switching an LED on and off.

First, let us create the units we will be working with:

```
#include <Plaqueette.h>

DigitalOut led(LED_BUILTIN); // LED connected to built-in pin
DigitalIn button(2, INTERNAL_PULLUP); // Button connected to pin 2
```

In order to react to an event, we first need to create a callback function which will be called when the event will happen:

```
// Callback function to toggle the LED.
void toggleLed() {
    led.toggle();
}
```

Then, we need to register our callback to an event. In this case, we will register our function `toggleLed()` to the `onRise()` event of our button unit, which will trigger at the instant the button is pressed.

```
void begin() {
    button.debounce(); // Enable debouncing to avoid multiple events

    // Register callbacks for button events.
    button.onRise(toggleLed); // Toggle the LED on button press
}
```

In this case, since the callback will take care of all the logic, we do not even need to declare a `step()` function!

Here is the final code for this example:

```
#include <Plaquette.h>

DigitalOut led(LED_BUILTIN); // LED connected to built-in pin
DigitalIn button(2, INTERNAL_PULLUP); // Button connected to pin 2

// Callback function to toggle the LED.
void toggleLed() {
    led.toggle();
}

void begin() {
    button.debounce(); // Enable debouncing to avoid multiple events

    // Register callbacks for button events.
    button.onRise(toggleLed); // Toggle the LED on button press
}

// Notice that we haven't invoked step(){} because it isn't necessary
```

Now, try changing `onRise()` to `onFall()` or to `onChange()`. How does that affect the interaction between the button and the LED?

2.6.4 Managing Multiple Events

It is possible to register multiple callbacks with the same event. Likewise, a single callback can be registered with many events.

Example: Launch both `toggleLed()` and `printButton()` on button press, registering `printButton()` to both press and release events.

```
#include <Plaquette.h>

DigitalOut led(LED_BUILTIN); // LED connected to built-in pin
DigitalIn button(2, INTERNAL_PULLUP); // Button connected to pin 2

Monitor monitor(115200); // Serial monitor.

// Callback function to toggle the LED.
void toggleLed() {
    led.toggle();
}

// Callback function to print button state.
void printButton() {
    print("Button ");
    println(button ? "pressed" : "released");
}

void begin() {
    button.debounce(); // Enable debouncing to avoid multiple events
```

(continues on next page)

(continued from previous page)

```
// Register callbacks for button events.
button.onRise(toggleLed); // Toggle the LED on button press

button.onRise(printButton); // Print button state
button.onFall(printButton); // Same here
}
```

2.6.5 Coordinating Parallel Events with Metronomes

There are many applications for which things happen concurrently at different pace, making one wish there could be multiple looping functions similar to `step()` running in parallel at different rates. This is easy to achieve in Plaquette using event-driven coding. Metronomes tick at a specific period, generating “bang” events which can trigger callbacks by registering them to the `onBang()` event.

In this example, two metronomes control two LEDs, one digital and one analog, each at a different interval. A ramp is used to fade the analog LED.

```
#include <Plaquette.h>

DigitalOut led1(LED_BUILTIN); // First LED (digital) connected to built-in pin
AnalogOut led2(9);           // Second LED (PWM) connected to pin 9
Metronome metro1(1.0);       // Metronome with a 1 second period
Metronome metro2(2.0);       // Metronome with a 2 seconds period
Ramp rampLed(0.5);           // Short ramp to control LED 2

// Function to toggle the first LED.
void pingLed1() {
    led1.toggle();
}

// Function to start the ramp on second LED.
void pingLed2() {
    rampLed.start();
}

void begin() {
    // Register callbacks for the metronomes.
    metro1.onBang(pingLed1); // Toggle LED 1 every second
    metro2.onBang(pingLed2); // Fade in LED 2 every 2 seconds
}

void step() {
    rampLed >> led2; // Ramp second LED from 100% to 0%
}
```

2.6.6 Creating On-the-fly Callbacks

For simple, localized actions, you can define callback functions directly inline using an **anonymous function** (also called **lambda function**) which can be created with the following syntax:

```
[ ]() {  
  // Function content goes here.  
}
```

It allows you to write concise code without defining separate named functions and is thus especially useful for short, self-contained actions, keeping the code clean and readable.

For example, we could rewrite the callback registration from the example above in a shorter way, like this:

```
void begin() {  
  // Register callbacks for the metronomes.  
  metro1.onBang([ ]() { led1.toggle(); }); // Toggle LED 1 every second  
  metro2.onBang([ ]() { rampLed.start(); }); // Fade in LED 2 every 2 seconds  
}
```

2.6.7 Conclusion

Event-driven programming in Plaquette simplifies the process of reacting to changes and scheduling actions, allowing you to write modular, expressive, and efficient code. By using callbacks and event sources like buttons and metronomes, you can manage complex behaviors that happen concurrently and at different rates.

2.7 Advanced Usage

2.7.1 Smoothing Arbitrary Signals

While the *AnalogIn* unit in Plaquette provide a convenient built-in `smooth()` function for removing noise, there are many cases where you may need to smooth signals coming from other sources such as specialized sensors. The *Smoother* unit provides a highly flexible smoothing solution for such use cases, allowing seamless integration into any signal pipeline. It works using an exponential moving average, acting as a low-pass filter to stabilize fast variations.

Here is an example of using the *Smoother* to smoothen a DHT 22 temperature and humidity sensor using the external DHT sensor library:

```
#include <Plaquette.h>  
#include <DHT.h> // External specialized library  
  
DHT dht(2, DHT22); // DHT 22 sensor connected to pin 2  
  
// Create a Smoother with a 10-second time window.  
Smoother temperatureSmoother(10.0);  
  
// Stream out for debugging (e.g., to Serial Monitor).  
Plotter serialOut(115200);  
  
void begin() {  
  dht.begin(); // Initialize the DHT sensor
```

(continues on next page)

(continued from previous page)

```

}

void step() {
    // Read temperature in Celsius.
    float rawTemperature = dht.readTemperature();

    // Smooth the temperature and send it to the Serial.
    rawTemperature >> temperatureSmoother >> serialOut;
}
    
```

2.7.2 Vanilla Coding Style

You can avoid Plaquette's >> operator or auto-conversion of units to values (eg., `if (input), input >> output`) in favor of a more conventional programming style by simply using the `get()` and `put()` functions of Plaquette units.

The `get()` method returns the current value of the unit:

```
float get()
```

The `put()` method sends a value to the unit and then returns the current value of the unit (the same that would be returned by `get()`):

```
float put(float value)
```

Additionally, digital input units such as *DigitalIn*, *Metronome* have a boolean `isOn()` method that works for boolean true/false values, while digital output units such as *DigitalOut* have a boolean `putOn(boolean value)` method.

Here are some examples of how to adopt a classic object-oriented functions style instead of the Plaquette style.

Plaquette Style	Object-Oriented Style
<code>input >> output;</code>	<code>output.put(input.get());</code>
<code>digitalInput >> digitalOutput;</code>	<code>digitalOutput.putOn(digitalInput.isOn());</code>
<code>(2 * input) >> output;</code>	<code>output.put(2 * input.get());</code>
<code>!digitalInput >> digitalOutput;</code>	<code>digitalOutput.putOn(!digitalInput.isOn());</code>
<code>if (digitalInput)</code>	<code>if (digitalInput.isOn())</code>
<code>if (input < 0.4)</code>	<code>if (input.get() < 0.4)</code>
<code>input >> filter >> output;</code>	<code>output.put(filter.put(input.get()));</code>

2.7.3 Using Plaquette as an External Library

Seasoned Arduino coders might want to avoid rewriting their code using Plaquette's builtin `begin()` and `step()` functions, or they may want to include Plaquette's self-updating loop in a timer interrupt function. It is possible to do so by including the file `PlaquetteLib.h` instead of `Plaquette.h`.

After this step, you are responsible for calling `Plaquette.begin()` at the beginning of the `setup()` function, and also to call `Plaquette.step()` at the beginning of the `loop()` function, or inside the interrupt.

Here is an example of our blinking code rewritten by using this feature:

```
#include <PlaquetteLib.h>
```

(continues on next page)

(continued from previous page)

```
using namespace pq;

DigitalOut myLed(13);

Wave myWave(2.0, 0.5);

void setup() {
    Plaquette.begin();
}

void loop() {
    Plaquette.step();
    myWave >> myLed;
}
```

Warning: `Plaquette.step()` computes timings and performs background update operations on units. When setting a specific (target) sample rate using `Plaquette.sampleRate(rate)`, the `Plaquette.step()` function returns `true` if the step has been fully performed and `false` if it is still waiting for the next “tick”. As an example, if one sets a 100 Hz sample rate by calling `Plaquette.sampleRate(100)`, the `Plaquette.step()` function should return `true` about 100 times per second.

It is thus highly recommended to use this feature as a **guard condition** in the main stepping loop to avoid performing unnecessary operations on units:

```
void loop() {
    if (Plaquette.step()) {
        myWave >> myLed;
    }
}
```

Alternative syntax that performs early exit if not ready:

```
void loop() {
    if (!Plaquette.step())
        return; // exit
    myWave >> myLed;
}
```

2.7.4 Synchronizing Groups of Units with Secondary Engines

Have you wondered how units such as waves, inputs, outputs, or ramps are automatically initialized and updated in Plaquette? This is done thanks to an *Engine*, a control structure that acts like the **conductor of an orchestra**. It contains an ensemble of **units** and manages their initialization and updates. Every time the engine “ticks”, it updates all of its units, making sure they stay synchronized.

By default, all units are added to a built-in engine called the **primary engine**. This engine is simply called Plaquette and is mainly used when working with *Plaquette as an external library*. In the default mode, when one declares the `void begin()` and `void step()` functions, the primary engine’s `Plaquette.begin()` and `Plaquette.step()` functions are automatically called.

There are many contexts where more than one engine are necessary:

- **Multi-tasking** Engines allow you to take full advantage of **timer interrupts**, **threads** and/or **multiple processor cores** to run different unit ensembles in parallel, possibly running with different frequencies and priorities.
- **Grouping** Engines can be used to better organize your code by creating **groups of units** and possibly run them at different frequencies.
- **Switching** On computationally-intensive applications with lots of units, you may want to **switch between multiple ensembles of units** to avoid running them all at the same time.
- **Saving Energy** Lowering the update frequency of units using engines allows for more energy-efficient applications.

In these cases, you can create **secondary engines** that each control their own group of units at their own refresh rate. You can step them in a timer interrupt, in a task running on another core, or even from a *Metronome* unit.

To create an engine, simply declare it:

```
Engine secondaryEngine;
```

When you create a unit, you can now add it to your new engine by adding the engine as the last argument of the unit's constructor:

```
// Ramp starting at default value.
Ramp ramp(secondaryEngine);

// Alarm unit with 10s duration.
Alarm alarm(10.0, secondaryEngine);

// Sine wave unit with period of 1s and 20% skew.
Wave wave(SINE, 1.0, 0.2, secondaryEngine);
```

Example: Fast vs Slow Control

In this example we will control a blinking LED with a pushbutton: every time we push the button, the LED will blink faster and faster. The button should be polled quite frequently to ensure it is debounced properly. However, there is no need to update the LED as fast as possible, so we can save some precious computation steps by updating it less often (about 25 frames per second would be plenty for such visual feedback).

- The **slow engine** (running at 25 fps) will control the LED with the square wave.
- The **fast engine** (running at 1000 Hz) will monitor the button.
- The **primary engine** (running as fast as possible) will use *Metronome* units to trigger the slow and fast engines.

```
#include <Plaquette.h>

// The engines.
Engine slowEngine;
Engine fastEngine;

// Metronomes (belong to primary engine).
Metronome slowMetro(0.04); // 25 fps
Metronome fastMetro(0.001); // 1000 Hz

DigitalIn button(2, INTERNAL_PULLUP, fastEngine); // Button (operates on fast engine).

// Oscillator and LED (can operate more slowly to save on computation).
```

(continues on next page)

(continued from previous page)

```

Wave squareWave(1.0, slowEngine);
DigitalOut led(LED_BUILTIN, slowEngine);

float ledFrequency = 1.0; // Oscillation frequency.

void begin() {
    // Initialize engines.
    slowEngine.begin();
    fastEngine.begin();
    button.debounce(); // Debounce button.
    // Attach metronomes to engine step functions.
    slowMetro.onBang(slowEngineStep);
    fastMetro.onBang(fastEngineStep);
}

void step() {}

void slowEngineStep() {
    slowEngine.step(); // Update slow engine.
    // Adjust frequency and send to LED.
    squareWave.frequency(ledFrequency);
    squareWave >> led;
}

void fastEngineStep() {
    fastEngine.step(); // Update fast engine.
    // On button press, increase square wave frequency.
    if (button.rose())
        ledFrequency++;
}

```

Example: ESP32 Dual Core

In this example, we will take full advantage of the ESP32's dual core architecture:

- **Core 1 (default)** will run the **primary engine** to update a simple indicator LED.
- **Core 0** will run a **secondary engine** to animate Neopixel LEDs using a waveform.

```

#include <Plaquette.h>
#include <Adafruit_NeoPixel.h>

// Pin where the Neopixel strip is connected
#define STRIP_PIN 5
#define NUM_LEDS 16

// The NeoPixel LED strip.
Adafruit_NeoPixel strip(NUM_LEDS, STRIP_PIN, NEO_GRB + NEO_KHZ800); // Neopixels.

Wave indicatorLedBlink(1.0, 0.2); // Blinking oscillator.
DigitalOut indicatorLed(LED_BUILTIN); // Indicator LED.

```

(continues on next page)

(continued from previous page)

```

Engine ledEngine; // Secondary engine for controlling NeoPixels.
Wave ledWave(SINE, 1.0, ledEngine); // Waveform for NeoPixels.

void begin() {
  xTaskCreatePinnedToCore( // Launch LED engine on Core 0.
    [] (void* param) {
      ledEngine.begin(); // Start LED engine.

      // Initialize LED strip.
      strip.begin();
      strip.show();

      // Main loop.
      while (true) {
        ledEngine.step(); // Step engine.
        stepLeds();       // Update LEDs.
        vTaskDelay(1);    // Important! Prevents IDLE on Core 0 (1 tick = ~1ms).
      }
    },
    "LED Engine", 2048, nullptr, 1, nullptr,
    0 // Core 0
  );
}

// Primary engine step function.
void step() {
  indicatorLedBlink >> indicatorLed; // Blink.
}

// Step function for the LED engine.
void stepLeds() {
  // Update LED strip according to LED wave.
  for (int i = 0; i < NUM_LEDS; i++) {
    float phaseShift = mapTo01(i, 0, NUM_LEDS-1); // LED position to % phase shift.
    float shiftedLedWave = ledWave.shiftBy(phaseShift); // Shifted wave value in [0, 1].
    int level = int(mapFrom01(shiftedLedWave, 0, 255)); // Brightness level in [0, 255].
    strip.setPixelColor(i, strip.Color(level, 0, 0)); // Red channel only.
  }
  strip.show(); // Display LEDs.
}

```

Multiple engines give you more control and better performance, especially on multi-core platforms or in time-sensitive applications like LED control, audio, or robotics.

2.8 Extending Plaquette

This guide provides comprehensive instructions for developers who want to extend Plaquette by creating custom units. Whether you're building a specialized sensor interface, a custom filter, or an advanced signal generator, this document covers everything you need to know.

2.8.1 Architecture Overview

Plaquette is built around a **signal-centric** programming model where data flows between components. The framework provides:

- **Automatic unit registration:** Units register themselves with their Engine upon construction
- **Synchronized updates:** All units are updated together in each execution step
- **Flow operators:** The `>>` operator chains components for intuitive data routing

Core Components

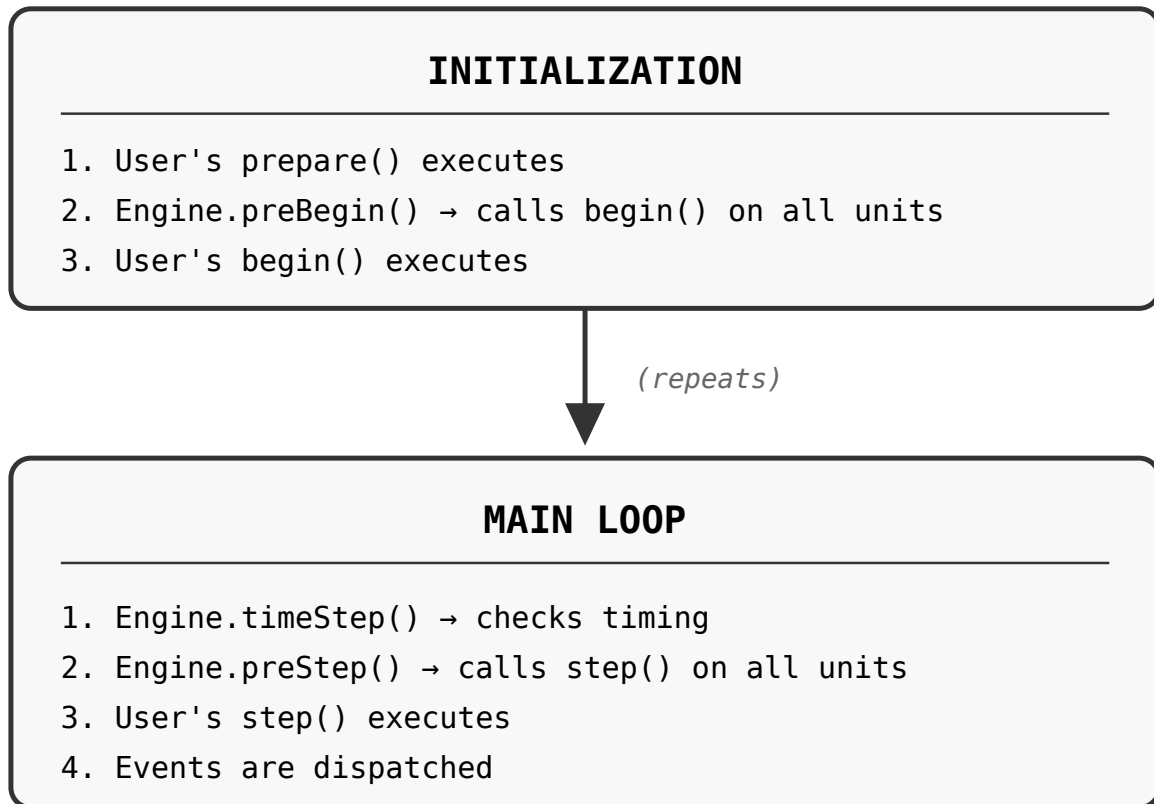
Component	Description
Engine	Central orchestrator that manages all units, timing, and execution
Flowable	Base interface for all signal components. Provides <code>get()</code> , <code>put()</code> , <code>>></code> operators, and conversion to <code>float/bool</code> so components can be used directly as values
Unit	Base class for engine-managed components with lifecycle methods

Note: All Flowables (including Units) can be used directly as values in expressions:

```
float v = sensor;           // get() called implicitly
if (button) { ... }        // true when isOn()
Serial.println(filter);    // prints current value
```

Execution Model

Plaquette replaces Arduino's `setup()/loop()` with `begin()/step()`:



The pq Namespace

All Plaquette classes and functions are defined within the `pq` namespace. This prevents naming conflicts with other libraries and the Arduino core. When writing extension code (header and implementation files), you should place your code inside the `pq` namespace:

```
// MyUnit.h
#include "PqCore.h"

namespace pq {

class MyUnit : public Unit {
    // ...
};

} // namespace pq
```

Note: If you're writing a standalone library that extends Plaquette but don't want to pollute the global namespace, you can use `pq::` prefixes explicitly or add `using namespace pq;` only where needed.

2.8.2 Flowable vs Unit: Choosing Your Base Class

This is the **most important decision** when creating a custom component. Plaquette provides two fundamentally different base classes:

Flowable: Non-Engine-Managed Components

Subclass Flowable when:

- Your component does NOT need to be managed by an engine
- You do NOT need per-frame updates (`step()`)
- You do NOT need initialization hooks (`begin()`)
- You do NOT need access to engine timing (sample rate, elapsed time)
- Your component can operate independently of the engine lifecycle

Characteristics of Flowable subclasses:

- **NOT registered with any Engine**
- **NO `begin()` or `step()` methods**
- Implement `get()` and `put()` for flow operator compatibility
- Compatible with Plaquette syntax
- Lightweight with no engine management overhead
- Examples: `Float`, `Boolean`, `Integer` (value wrappers), custom transformers

Example: Clamped Value

```
#include "PqCore.h"

namespace pq {

/// A value that stays within [min, max] bounds.
class ClampedValue : public Flowable {
public:
    ClampedValue(float min = 0.0f, float max = 1.0f)
        : _value(0), _min(min), _max(max) {}

    float get() override {
        return _value;
    }

    float put(float value) override {
        _value = constrain(value, _min, _max);
        return _value;
    }

private:
    float _value;
    float _min;
    float _max;
};
```

(continues on next page)

(continued from previous page)

```
} // namespace pq
```

Usage:

```
ClampedValue clamped(0.0, 100.0);
sensor >> clamped >> output; // Values automatically clamped

// Also works with Plaquette syntax:
float v = clamped;           // Uses operator float()
if (clamped) { ... }        // Uses operator bool()
```

Unit: Engine-Managed Components

Subclass Unit (or its descendants) when:

- Your component needs to be managed by an engine
- You need per-frame updates via `step()`
- You need initialization via `begin()`
- You need access to engine timing (`seconds()`, `sampleRate()`, `samplePeriod()`)
- You interact with hardware (sensors, actuators)
- You implement time-based behavior (oscillators, timers, filters)
- You need event callbacks

Characteristics of Unit subclasses:

- **Automatically registered with an Engine** upon construction
- **MUST accept an Engine& parameter** in ALL constructors (defaulting to primary engine)
- Have `begin()` and `step()` lifecycle methods
- Access to synchronized timing information via engine
- Can register event callbacks

Decision Flowchart

Does your component need...

- Per-frame updates (`step`)?
 - └ YES → Use Unit
- Initialization (`begin`)?
 - └ YES → Use Unit
- Access to synchronized time (`seconds`, `sampleRate`)?
 - └ YES → Use Unit
- Event callbacks?

(continues on next page)

(continued from previous page)

- └ YES → Use Unit
- └ None of the above, just flow compatibility?
 - └ YES → Use Flowable

Comparison Table

Feature	Flowable	Unit
Engine registration	×	✓ (automatic)
begin() method	×	✓
step() method	×	✓
Synchronized timing	×	✓
Event callbacks	×	✓
Flow operators (>>)	✓	✓
operator float()	✓	✓
operator bool()	✓	✓
Memory overhead	Minimal	Engine tracking
Use case	Standalone components	Engine-managed components

2.8.3 Class Hierarchy

Complete Hierarchy

```

Flowable (interface - NOT engine-managed)
├── Value<T> (Float, Boolean, Integer - NOT engine-managed)
├── Unit (base class - engine-managed)
│   ├── DigitalUnit (boolean on/off units)
│   │   └── DigitalSource (with change detection)
│   └── AnalogSource (values in [0, 1] range)
    
```

Flowable

The foundational interface for all chainable components:

```

class Flowable {
public:
    virtual float get();           // Read current value
    virtual float put(float value); // Write value (default: read-only)
    virtual float mapTo(float toLow, float toHigh); // Map value to range

    operator float();             // Implicit conversion to float
    
```

(continues on next page)

(continued from previous page)

```
explicit operator bool();           // Explicit conversion to bool
};
```

Unit

The base class for all engine-managed components:

```
class Unit : public Flowable {
protected:
    virtual void begin();    // Called once after all units are registered
    virtual void step();     // Called every frame in the main loop

public:
    // IMPORTANT: Constructor MUST accept Engine reference
    Unit(Engine& engine = Engine::primary());

    // Access engine-synchronized timing
    float seconds() const;
    uint32_t milliseconds() const;
    uint64_t microseconds() const;
    unsigned long nSteps() const;
    float sampleRate() const;
    float samplePeriod() const;

    // Event support
    virtual void onEvent(EventCallback callback, EventType eventType);
};
```

DigitalUnit

Base for boolean (on/off) units:

```
class DigitalUnit : public Unit {
public:
    DigitalUnit(Engine& engine = Engine::primary());

    virtual bool isOn();
    virtual bool isOff();
    virtual int getInt();           // Returns 0 or 1
    virtual float get();            // Returns 0.0 or 1.0

    virtual bool on();              // Turn on
    virtual bool off();             // Turn off
    virtual bool putOn(bool value); // Set state (override for writable units)
};
```

DigitalSource

Digital unit with change detection and events:

```
class DigitalSource : public DigitalUnit {
public:
    DigitalSource(Engine& engine = Engine::primary());

    virtual bool rose();      // True if just changed from off to on
    virtual bool fell();      // True if just changed from on to off
    virtual bool changed();   // True if state changed this step

    // Event callbacks
    virtual void onRise(EventCallback callback);
    virtual void onFall(EventCallback callback);
    virtual void onChange(EventCallback callback);
};
```

AnalogSource

Base for units outputting normalized values in [0, 1]:

```
class AnalogSource : public Unit {
public:
    AnalogSource(Engine& engine = Engine::primary());
    AnalogSource(float initialValue, Engine& engine = Engine::primary());

    virtual float get(); // Returns _value (always in [0, 1])

protected:
    float _value;        // Must stay in [0, 1] range
};
```

2.8.4 Unit Lifecycle

Every unit follows a predictable lifecycle managed by the Engine.

Construction and Engine Registration

Warning: All Unit subclasses MUST accept an Engine& parameter in every constructor, defaulting to the primary engine.

Header file (.h): Declare the default value here:

```
class MyUnit : public AnalogSource {
public:
    MyUnit(Engine& engine = Engine::primary());
    MyUnit(float param, Engine& engine = Engine::primary());
    MyUnit(float param1, float param2, Engine& engine = Engine::primary());
```

(continues on next page)

(continued from previous page)

```
// ...
};
```

Implementation file (.cpp):

```
MyUnit::MyUnit(Engine& engine)
    : AnalogSource(engine)
{ }

MyUnit::MyUnit(float param, Engine& engine)
    : AnalogSource(engine), _param(param)
{ }

MyUnit::MyUnit(float param1, float param2, Engine& engine)
    : AnalogSource(engine), _param1(param1), _param2(param2)
{ }
```

Caution: Never provide constructors without an Engine& parameter.

In header:

```
// WRONG: Missing engine parameter
MyUnit(float param);
```

Core abstract classes Unit, DigitalUnit, DigitalSource and AnalogSource deliberately do NOT provide a default engine at construction. This causes a compilation error if you forget the engine parameter - a safety feature that catches mistakes early.

Caution: Always pass the engine to the parent class.

When subclassing units whose constructors provide a default engine, forgetting to pass the engine breaks multi-engine setups silently:

```
// WRONG: Missing parent initializer - defaults to primary engine, ignoring parameter
MyUnitSubClass::MyUnitSubClass(Engine& engine)
{ }

// WRONG: Same problem - engine parameter is ignored
MyUnitSubClass::MyUnitSubClass(float param, Engine& engine)
    : MyUnit(param)
{ }

// CORRECT: Always pass the engine to the parent class
MyUnitSubClass::MyUnitSubClass(float param, Engine& engine)
    : MyUnit(param, engine)
{ }
```

Why is this important?

- Users may want to assign units to secondary engines
- The engine parameter must ALWAYS be the LAST parameter

- Forgetting to pass the engine breaks multi-engine setups silently

```
// User can assign to different engines:
Engine slowEngine;
MyUnit unit1;           // Uses primary engine (default)
MyUnit unit2(0.5);      // Uses primary engine (default)
MyUnit unit3(0.5, slowEngine); // Uses secondary engine
```

begin()

Called once after all units are registered, before the main loop:

```
void MyUnit::begin() {
    // Initialize state
    // Read initial sensor values
    // Set up hardware
}
```

step()

Called once per frame in the main loop:

```
void MyUnit::step() {
    // Update internal state
    // Read sensors / write actuators
    // Cache computed values for get()
}
```

Timing Access

Inside any unit method, you have access to engine-synchronized timing information.

Warning: Always use the unit's timing methods instead of global Arduino functions like `millis()` or `micros()`. The unit's timing methods are synchronized with its engine, ensuring consistent behavior especially when using multiple engines with different sample rates.

```
void MyUnit::step() {
    // CORRECT: Use engine-synchronized timing
    float t = seconds();      // Time since engine start (seconds)
    float dt = samplePeriod(); // Time since last step (seconds)
    float rate = sampleRate(); // Steps per second
    unsigned long n = nSteps(); // Number of steps executed

    // WRONG: Don't use global Arduino timing functions
    // unsigned long t = millis(); // Not synchronized with engine!
    // unsigned long t = micros(); // Not synchronized with engine!
}
```

Why use engine timing?

- Engine timing is synchronized with the unit's step cycle
- Different engines can run at different sample rates
- Consistent timing even when using secondary engines
- Proper handling of sample rate changes

2.8.5 Step-Invariance Principle

The step-invariance principle applies to **source units** (units whose `put(float)` is inactive or not overridden). For these units, `get()` should return a constant value throughout a step.

Sink and filter units (units that accept input via `put(float)`) are different: their value *can* change during a step when `put()` is called. This is expected and correct behavior.

When Step-Invariance Applies

Unit Type	put() Behavior	Step-Invariance
Source (sensor, generator)	Inactive / not overridden	<code>get()</code> constant throughout step
Sink (output, actuator)	Accepts input, stores for <code>step()</code>	<code>get()</code> may change when <code>put()</code> called
Filter (transformer)	Accepts input, transforms immediately	<code>get()</code> changes when <code>put()</code> called

For source units, the principle ensures:

1. **get() returns a constant value throughout a step**
2. **Hardware reads only occur in step()** (never in `get()`)
3. **Values are cached** and returned consistently until next step

2.8.6 Flow Operators

The `>>` operator enables intuitive signal chaining:

```
sensor >> filter >> output;
// Equivalent to: output.put(filter.put(sensor.get()));
```

How It Works

```
// Float to flowable
inline float operator>>(float value, Flowable& unit) {
    return unit.put(value);
}

// Flowable to flowable
inline float operator>>(Flowable& source, Flowable& sink) {
    return sink.put(source.get());
}
```

Making Your Component Flow-Compatible

Both Flowable and Unit subclasses are automatically flow-compatible if they implement `get()` and/or `put()`:

```
// Flowable subclass (no engine)
class MyTransformer : public Flowable {
public:
    float get() override { return _value; }
    float put(float value) override {
        _value = transform(value);
        return _value;
    }
private:
    float _value;
    float transform(float v) { return v * v; }
};

// Unit subclass (with engine)
class MyUnit : public AnalogSource {
public:
    MyUnit(Engine& engine = Engine::primary()) : AnalogSource(engine) {}

    float get() override { return _value; }
    float put(float value) override {
        _value = constrain01(value);
        return _value;
    }
};

// Both work with flow operators:
sensor >> myTransformer >> myUnit >> output;
```

2.8.7 Creating Your First Unit

Let's create a simple unit step by step.

Step 1: Choose Your Base Class

If your component...	Inherit from
Doesn't need engine management (no timing, no step)	Flowable
Outputs values in [0, 1] (sensors, oscillators, filters)	AnalogSource
Outputs boolean on/off states	DigitalUnit
Outputs boolean with change detection (buttons, triggers)	DigitalSource
Has custom value range or complex behavior	Unit

Step 2: Create the Header File

```
// RandomWalk.h
#ifndef RANDOM_WALK_H_
#define RANDOM_WALK_H_

#include "PqCore.h"

namespace pq {

/**
 * Generates a smooth random walk in [0, 1].
 *
 * The value drifts randomly at each step, scaled by the rate
 * parameter and the engine's sample period. Use put() to hard
 * reset to a specific value.
 */
class RandomWalk : public AnalogSource {
public:
    /**
     * Constructor with default rate.
     * @param engine the engine running this unit
     */
    RandomWalk(Engine& engine = Engine::primary());

    /**
     * Constructor with specified rate.
     * @param rate maximum change per second (default: 0.5)
     * @param engine the engine running this unit
     */
    RandomWalk(float rate, Engine& engine = Engine::primary());

    virtual ~RandomWalk() {}

    /// Sets the rate of change per second.
    void rate(float rate);

    /// Returns the current rate.
    float rate() const { return _rate; }

    /// Returns a ParameterSlot for dynamic rate modulation.
    ParameterSlot<RandomWalk> Rate() {
        return ParameterSlot<RandomWalk>(this, &RandomWalk::rate, &RandomWalk::rate);
    }

    /**
     * Hard resets the random walk to a specific value.
     * @param value the value to reset to (clamped to [0, 1])
     * @return the new value
     */
    virtual float put(float value) override;
protected:
```

(continues on next page)

(continued from previous page)

```

virtual void begin() override;
virtual void step() override;

private:
    float _rate;
};

} // namespace pq

#endif // RANDOM_WALK_H

```

Step 3: Create the Implementation File

```

// RandomWalk.cpp
#include "RandomWalk.h"

namespace pq {

RandomWalk::RandomWalk(Engine& engine)
    : AnalogSource(engine), // Pass engine to parent
      _rate(0.5f)           // Default rate: 0.5 units/second
{
    // Unit is automatically registered with engine
}

RandomWalk::RandomWalk(float rate, Engine& engine)
    : AnalogSource(engine), // Pass engine to parent
      _rate(rate)
{
    // Unit is automatically registered with engine
}

void RandomWalk::begin() {
    // Initialize to center position
    _value = 0.5f;
}

void RandomWalk::step() {
    // Compute random delta scaled by rate and sample period.
    // randomFloat(-1, 1) gives uniform random in [-1, 1].
    // Multiplying by rate and samplePeriod() makes the walk
    // time-consistent regardless of sample rate.
    float delta = randomFloat(-1.0f, 1.0f) * _rate * samplePeriod();

    // Apply delta and constrain to [0, 1]
    _value = constrain01(_value + delta);
}

void RandomWalk::rate(float rate) {
    _rate = max(rate, 0.0f); // Rate must be non-negative
}

```

(continues on next page)

(continued from previous page)

```
}

float RandomWalk::put(float value) {
    // Hard reset: immediately set value (bypass random walk)
    _value = constrain01(value);
    return _value;
}

} // namespace pq
```

Step 4: Use Your Unit

```
#include <Plaquette.h>
#include "RandomWalk.h"

using namespace pq;

RandomWalk wanderer(0.3); // Slow drift: 0.3 units/second max
AnalogOut led(LED_BUILTIN);
DigitalIn button(2);
SineWave lfo(20.0); // Slow LFO for rate modulation

void step() {
    // Modulate rate dynamically via ParameterSlot
    lfo >> wanderer.Rate();

    // LED brightness follows random walk
    wanderer >> led;

    // Button press resets walk to minimum
    if (button.rose())
        0 >> wanderer;
}

// Or with a secondary engine:
Engine slowEngine;
RandomWalk slowWanderer(0.1, slowEngine); // Assigned to secondary engine
```

2.8.8 Unit Templates

The following templates show common patterns. All Unit subclasses must include `Engine& engine = Engine::primary()` as the last parameter in every constructor.

Source Unit (Generator)

For units that generate values (step-invariant):

```
class MyGenerator : public AnalogSource {
public:
    MyGenerator(float frequency, Engine& engine = Engine::primary())
        : AnalogSource(engine), _frequency(frequency) {}

protected:
    void step() override {
        float t = seconds(); // Use engine timing, not millis()!
        _value = /* compute value in [0, 1] */;
    }

private:
    float _frequency;
};
```

Filter Unit

For units that transform incoming signals:

```
class MyFilter : public AnalogSource {
public:
    MyFilter(float strength, Engine& engine = Engine::primary())
        : AnalogSource(engine), _strength(strength) {}

    float put(float value) override {
        _value = constrain01(applyFilter(value));
        return _value;
    }

private:
    float _strength;
    float applyFilter(float v) { return v * _strength; }
};
```

Digital Trigger Unit

For units that detect events (inherits `rose()`, `fell()`, `changed()`):

```
class MyTrigger : public DigitalSource {
public:
    MyTrigger(float threshold, Engine& engine = Engine::primary())
        : DigitalSource(engine), _threshold(threshold) {}

    float put(float value) override {
        _currentValue = value;
        return get();
    }

protected:
    void step() override {
        _onValue = (_currentValue >= _threshold);
        _updateChangeState(); // Updates rose/fell/changed
    }

private:
    float _threshold;
    float _currentValue;
};
```

2.8.9 Parameter Slots

Parameter slots allow dynamic control of unit properties via flow operators:

```
Wave lfo(10.0);
Wave carrier(0.5);

void step() {
    lfo >> carrier.Frequency(); // LFO modulates carrier frequency
}
```

Implementing Parameter Slots

Use the `ParameterSlot` template to expose parameters:

```
// In your header
#include "PqCore.h"

class MyOscillator : public AnalogSource {
public:
    MyOscillator(Engine& engine = Engine::primary())
        : AnalogSource(engine), _frequency(1.0f) {}

    // Getter and setter for frequency
    void frequency(float freq) { _frequency = max(0.0f, freq); }
    float frequency() const { return _frequency; }
```

(continues on next page)

(continued from previous page)

```
// Parameter slot accessor
ParameterSlot<MyOscillator> Frequency() {
    return ParameterSlot<MyOscillator>(
        this,
        &MyOscillator::frequency, // setter
        &MyOscillator::frequency // getter (same name, different signature)
    );
}

private:
    float _frequency;
};
```

2.8.10 Event System

Units can trigger callbacks when specific events occur. Plaquette provides several event types for different use cases.

Event Types Reference

Event Type	When to Use	Boolean Check	Callback Registration
EVENT_BANG	Trigger/pulse units where <code>get()</code> returns 1 momentarily during <code>step()</code> (e.g., metronome tick, peak detected) and 0 otherwise	n/a	<code>onBang()</code>
EVENT_RISE	Digital units that transition from off to on	<code>rose()</code>	<code>onRise()</code>
EVENT_FALL	Digital units that transition from on to off	<code>fell()</code>	<code>onFall()</code>
EVENT_CHANGE	Digital units that change state (either direction)	<code>changed()</code>	<code>onChange()</code>
EVENT_FINISH	Timers or processes that complete (e.g., Ramp, Alarm)	<code>finished()</code>	<code>onFinish()</code>
EVENT_UPDATE	Units that get updated (eg. received data)	<code>updated()</code>	<code>onUpdate()</code>
EVENT_CUSTOM_1 to EVENT_CUSTOM_4	Custom events for specialized units	(user-defined)	(user-defined)

Event Naming Convention

Important: Plaquette follows a consistent naming convention for events:

- **Boolean check:** Use **past tense** (e.g., `rose()`, `fell()`, `finished()`)
 - **Callback registration:** Use **present tense** with `on` prefix (e.g., `onRise()`, `onFall()`, `onFinish()`)
-

This convention makes code read naturally:

```
if (button.rose()) { ... }           // "if button rose" (past tense check)
button.onRise(myCallback);           // "on rise, call myCallback" (present tense action)
```

Adding Event Support

```
class MyDetector : public DigitalUnit {
public:
    MyDetector(Engine& engine = Engine::primary())
        : DigitalUnit(engine), _detected(false) {}

    /// Returns true if detection occurred this step (past tense).
    bool detected() const { return _detected; }

    /// Register callback for detection events (present tense with "on" prefix).
    void onDetect(EventCallback callback) {
        onEvent(callback, EVENT_BANG);
    }

protected:
    bool eventTriggered(EventType eventType) override {
        switch (eventType) {
            case EVENT_BANG: return _detected;
            default: return DigitalUnit::eventTriggered(eventType);
        }
    }

    void step() override {
        _detected = checkCondition();
        /// Events are dispatched automatically after step()
    }

private:
    bool _detected;

    bool checkCondition() {
        /// Your detection logic
        return false;
    }
};
```

Usage

```
MyDetector detector;

void begin() {
  detector.onDetect([]() {
    Serial.println("Detected!");
  });
}

void step() {
  // Can also check the boolean directly
  if (detector.detected()) {
    // Do something
  }
}
```

2.8.11 Testing Your Unit

Plaquette uses AUnit for testing, running natively on Linux/macOS via EpoxyDuino.

Test File Structure

Create a test directory and .ino file:

```
tests/
  myunit/
    myunit.ino
    Makefile
```

Example Test File

```
// tests/myunit/myunit.ino
#include <Arduino.h>
#include <PlaquetteLib.h>
#include <AUnit.h>

using namespace pq;

MyGain gain(2.0);

test(gain_doubles_value) {
  Plaquette.step();

  gain.put(0.25);
  assertNear(gain.get(), 0.5f, 0.001f);

  Plaquette.step();
}
```

(continues on next page)

(continued from previous page)

```
test(gain_constrains_to_01) {
    Plaquette.step();

    gain.put(0.75); // 0.75 * 2 = 1.5, should clamp to 1.0
    assertEquals(gain.get(), 1.0f);

    Plaquette.step();
}

test(gain_flow_operator) {
    Plaquette.step();

    0.3f >> gain;
    assertNear(gain.get(), 0.6f, 0.001f);

    Plaquette.step();
}

test(gain_with_secondary_engine) {
    Engine secondaryEngine;
    MyGain secondaryGain(3.0, secondaryEngine);

    secondaryEngine.begin();
    secondaryEngine.step();

    secondaryGain.put(0.2);
    assertNear(secondaryGain.get(), 0.6f, 0.001f);

    secondaryEngine.step();
}

void setup() {
    Plaquette.begin();
}

void loop() {
    aunit::TestRunner::run();
}
```

Makefile

```
APP_NAME := myunit
ARDUINO_LIBS := AUnit Plaquette

include ../../../../libraries/EpoxyDuino/EpoxyDuino.mk
```

Running Tests

```
cd tests/myunit
make clean
make
./myunit.out
```

Testing Tips

1. **Always call** `Plaquette.step()` between test operations to simulate the main loop
2. **Use** `assertNear()` for floating-point comparisons
3. **Test edge cases:** zero values, maximum values, negative inputs
4. **Test flow operators:** ensure `>>` works correctly
5. **Test step-invariance:** verify `get()` returns the same value within a step
6. **Test with secondary engines:** verify the engine parameter works correctly

2.8.12 Coding Conventions

Follow these conventions for consistency with the Plaquette codebase.

Naming

Element	Convention	Example
Classes	PascalCase	PeakDetector
Public methods	camelCase()	frequency()
Parameter methods	PascalCase()	Frequency()
Public variables	camelCase	value
Private/protected methods	<code>_camelCase()</code>	<code>_step()</code>
Private/protected variables	<code>_camelCase</code>	<code>_value</code>
Enums/constants	UPPER_SNAKE	PEAK_MAX
Event boolean check	past tense	<code>rose()</code> , <code>fell()</code> , <code>sampled()</code>
Event callback registration	on + present tense	<code>onRise()</code> , <code>onFall()</code> , <code>onSample()</code>

Constructor Pattern

Important: All Unit subclasses must follow this pattern:

```
class MyUnit : public AnalogSource {
public:
    // Constructor with just engine (always provide this one)
    MyUnit(Engine& engine = Engine::primary());

    // Constructor with parameters - engine is ALWAYS LAST with default
```

(continues on next page)

(continued from previous page)

```
MyUnit(float param1, Engine& engine = Engine::primary());

// Multiple parameters - engine is ALWAYS LAST with default
MyUnit(float param1, float param2, Engine& engine = Engine::primary());
};
```

Getter/Setter Pattern

Use the same name for getter and setter:

```
// Getter
float frequency() const { return _frequency; }

// Setter
void frequency(float freq) { _frequency = freq; }
```

Formatting

- **Indentation:** 2 spaces (no tabs)
- **Braces:** Opening on same line, closing on own line
- **Line length:** 100 characters or fewer

```
void MyUnit::step() {
    if (condition) {
        doSomething();
    } else {
        doSomethingElse();
    }
}
```

Documentation

Use Doxygen-style comments:

```
/**
 * Sets the frequency of oscillation.
 * @param frequency the frequency in Hz (must be >= 0)
 */
void frequency(float frequency);
```

Include Guards

Use `#define` style (not `#pragma once`):

```
#ifndef MY_UNIT_H_
#define MY_UNIT_H_

// ... content ...

#endif // MY_UNIT_H_
```

License Header

All files must include the GPLv3+ header:

```
/*
 * MyUnit.h
 *
 * (c) 2025 Your Name :: your@email.com
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program. If not, see <http://www.gnu.org/licenses/>.
 */
```

Memory Considerations

- **Avoid heap allocations** - use stack or pre-allocated buffers
- **Use bit-packing** for boolean flags: `bool _flag : 1;`
- **Prefer const references** to avoid copies
- **Target Arduino Uno/Nano** memory constraints

2.8.13 Summary

Key Decisions

1. **Flowable vs Unit:** Use `Flowable` for components that don't need engine management; use `Unit` for components requiring `begin()/step()` lifecycle and synchronized timing
2. **Engine Parameter:** ALL `Unit` constructors MUST accept `Engine& engine = Engine::primary()` as the LAST parameter
3. **Timing Functions:** Always use the unit's timing methods (`seconds()`, `samplePeriod()`, etc.) instead of global Arduino functions (`millis()`, `micros()`) to ensure synchronization with the engine
4. **Event Naming:** Use past tense for boolean checks (`sampld()`) and present tense with `on` prefix for callbacks (`onSample()`)
5. **Base Class Selection:**
 - `Flowable` - Non-engine-managed components
 - `AnalogSource` - Values in [0, 1]
 - `DigitalUnit` - Boolean on/off
 - `DigitalSource` - Boolean with change detection
 - `Unit` - Custom behavior

Implementation Checklist

- Choose appropriate base class (`Flowable` or `Unit` subclass)
- If `Unit`: Add `Engine&` parameter to ALL constructors (last, with default)
- If `Unit`: Use engine timing methods, not global Arduino functions
- Implement `begin()` for initialization (`Unit` only)
- Implement `step()` for per-frame updates (`Unit` only)
- Implement `get()` to return cached values
- Implement `put()` to accept input values
- Follow step-invariance principle (optional but recommended)
- For events: past tense boolean, present tense callback
- Add parameter slots for dynamic control (optional)
- Write tests using `AUnit`
- Follow coding conventions
- Add GPLv3+ license header

For questions or contributions, see the [CONTRIBUTING.md](#) file or open an issue on GitHub.

REFERENCE

3.1 Base Units

Basic input-output units.

3.1.1 AnalogIn

An analog (ie. continuous) input unit that returns values between 0 and 1 (ie. 0% and 100%).

The unit is assigned to a specific pin on the board.

The mode specifies the behavior of the component attached to the pin:

- in DIRECT mode (default) the value is expressed as a percentage of the reference voltage (Vref, typically 5V)
- in INVERTED mode the value is inverted (ie. 0V corresponds to 100% while 2.5V corresponds to 50%).

Warning: If the analog input pin is **not connected** to anything, the value returned by `get()` will fluctuate based on a number of factors (e.g. the values of the other analog inputs, how close your hand is to the board, etc.).

Example

Control an LED using a potentiometer.

```
#include <Plaquette.h>

AnalogIn potentiometer(A0);

AnalogOut led(9);

Wave oscillator(SINE);

void step() {
    // The analog input controls the frequency of the LED's oscillation.
    oscillator.frequency(potentiometer.mapTo(2.0, 10.0));
    oscillator >> led;
}
```

Reference

class **AnalogIn** : public pq::Unit, public pq::PinConfig, public pq::Smoothable

A generic class representing a simple analog input.

Public Functions

AnalogIn(uint8_t pin, *Engine* &engine = *Engine::primary*())

Constructor.

Parameters

- **pin** – the pin number
- **mode** – the mode (DIRECT or INVERTED)

inline virtual float **get**()

Returns value in [0, 1].

virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

float **read**() const

Directly reads value from the pin (bypasses mode, smoothing, and engine).

int **rawRead**() const

Directly reads raw value from the pin (bypasses mode, smoothing, and engine).

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliSeconds**() const

Returns engine time in milliseconds.

inline uint64_t **microSeconds**() const

Returns engine time in microseconds.

inline unsigned long **nSteps**() const

Returns number of engine steps.

inline float **sampleRate**() const

Returns engine sample rate.

inline float **samplePeriod**() const

Returns enginesample period.

inline **operator float**()

Object can be used directly to access its value.

inline virtual float **put**(float value)

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline explicit **operator bool()**

Operator that allows usage in conditional expressions.

inline uint8_t **pin()** const

Returns the pin this component is attached to.

inline uint8_t **mode()** const

Returns the mode of the component.

inline virtual void **mode**(uint8_t mode)

Changes the mode of the component.

inline virtual void **smooth**(float smoothTime = PLAQUETTE_DEFAULT_SMOOTH_WINDOW)

Apply smoothing to object.

inline virtual void **noSmooth()**

Remove smoothing.

virtual void **infiniteTimeWindow()**

Sets time window to infinite.

virtual void **noTimeWindow()**

Sets time window to no time window.

virtual void **timeWindow**(float seconds)

Changes the time window (expressed in seconds).

inline virtual float **timeWindow()** const

Returns the time window (expressed in seconds).

inline virtual bool **timeWindowIsInfinite()** const

Returns true if time window is infinite.

virtual void **cutoff**(float hz)

Changes the time window cutoff frequency (expressed in Hz).

virtual float **cutoff()** const

Returns the time window cutoff frequency (expressed in Hz).

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- [AnalogOut](#)
- [DigitalIn](#)

3.1.2 AnalogOut

An analog (ie. continuous) output unit that converts a value between 0 and 1 (ie. 0% and 100%) into an analog voltage on one of the analog output pins.

The unit is assigned to a specific `pin` on the board.

The mode specifies the behavior of the component attached to the pin:

- in `DIRECT` mode (default) the pin acts as the source of current and the value is expressed as a percentage of the maximum voltage (V_{cc} , typically 5V)
- in `INVERTED` mode the source of current is external (V_{cc})

Example

```
AnalogOut led(9);

void begin() {
  // Initialize the brightness of the LED at half-strength.
  led.put(0.5);
}

void step() {
  // The LED value is changed randomly by a tiny amount (random walk).
  // Multiplying by samplePeriod() makes sure the rate of change stays stable.
  (led + randomFloat(-0.1, 0.1) * samplePeriod()) >> led;
}
```

Important: On most Arduino boards analog outputs rely on [Pulse Width Modulation \(PWM\)](#). After a call to `put(value)`, the pin will generate a steady square wave of the specified duty cycle until the next call to `put()` on the same pin. The frequency of the PWM signal on most pins is approximately 490 Hz. On the Uno and similar boards, pins 5 and 6 have a frequency of approximately 980 Hz.

Note: On most Arduino boards (those with the ATmega168 or ATmega328P), this functionality works on pins 3, 5, 6, 9, 10, and 11. On the Arduino Mega, it works on pins 2 - 13 and 44 - 46. Older Arduino boards with an ATmega8 only support `AnalogOut` on pins 9, 10, and 11. The Arduino DUE supports analog output on pins 2 through 13, plus pins DAC0 and DAC1. Unlike the PWM pins, DAC0 and DAC1 are Digital to Analog converters, and act as true analog outputs.

Reference

class **AnalogOut** : public pq::AnalogSource, public pq::PinConfig

A generic class representing a simple PWM output.

Public Functions

AnalogOut(uint8_t pin, *Engine* &engine = *Engine::primary()*)

Constructor with default mode DIRECT.

Parameters

pin – the pin number

AnalogOut(uint8_t pin, uint8_t mode, *Engine* &engine = *Engine::primary()*)

Constructor.

Parameters

- **pin** – the pin number
- **mode** – the mode (DIRECT or INVERTED)

virtual float **put**(float value)

Pushes value into the component and returns its (possibly filtered) value.

inline virtual void **invert**()

Inverts value by calling `put(1-get())` (eg. 0.2 becomes 0.8).

void **write**(float value)

Directly writes value in [0, 1] to the pin (bypasses mode and engine).

void **rawWrite**(int value)

Directly writes raw value to the pin (bypasses mode and engine).

inline virtual float **get**()

Returns value in [0, 1].

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliSeconds**() const

Returns engine time in milliseconds.

inline uint64_t **microSeconds**() const

Returns engine time in microseconds.

inline unsigned long **nSteps**() const

Returns number of engine steps.

inline float **sampleRate**() const

Returns engine sample rate.

inline float **samplePeriod**() const

Returns enginesample period.

inline **operator float()**

Object can be used directly to access its value.

inline explicit **operator bool()**

Operator that allows usage in conditional expressions.

inline uint8_t **pin()** const

Returns the pin this component is attached to.

inline uint8_t **mode()** const

Returns the mode of the component.

inline virtual void **mode**(uint8_t mode)

Changes the mode of the component.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- *AnalogIn*
- *DigitalOut*

3.1.3 DigitalIn

A digital (ie. binary) input unit that can be either “on” or “off”.

The unit is assigned to a specific **pin** on the board.

The **mode** specifies the behavior of the component attached to the pin:

- in **DIRECT** mode (default) the unit will be “on” when the voltage on the pin is high (V_{ref} , typically 5V)
- in **INVERTED** mode the unit will be “on” when the voltage on the pin is low (GND)
- in **INTERNAL_PULLUP** mode the internal **pullup resistor** is used, simplifying usage of switches and buttons

Debouncing

Some digital inputs such as **push-buttons** often generate spurious open/close transitions when pressed, due to mechanical and physical issues: these transitions called “bouncing” may be read as multiple presses in a very short time, fooling the program.

The **DigitalIn** object features debouncing capabilities which can prevent this kind of problems. Debouncing can be achieved using different modes: stable (default) (**DEBOUNCE_STABLE**), lock-out (**DEBOUNCE_LOCK_OUT**) and prompt-detect (**DEBOUNCE_PROMPT_DETECT**). For more information please refer to the documentation of the [Bounce2 Arduino Library](#).

Events

When	Trigger	Callback
Digital value toggles (on/off)	<code>changed()</code>	<code>onChange(c)</code>
Digital value goes from on to off	<code>fell()</code>	<code>onFall(c)</code>
Digital value goes from off to on	<code>rose()</code>	<code>onRise(c)</code>

Example

Turns on and off a light emitting diode (LED) connected to digital pin 13, when pressing a pushbutton attached to digital pin 2. Pushbutton should be wired by connecting one side to pin 2 and the other to ground.

```
#include <Plaquette.h>

DigitalIn button(2, INTERNAL_PULLUP);

DigitalOut led(13);

void begin() {
    button.debounce(); // debounce button
}

void step() {
    // Toggle the LED each time the button is pressed.
    if (button.rose())
        led.toggle();
}
```

Reference

class **DigitalIn** : public pq::DigitalSource, public pq::PinConfig, public pq::Debounceable

A generic class representing a simple digital input.

Public Functions

DigitalIn(uint8_t pin, *Engine* &engine = *Engine::primary()*)

Constructor.

Parameters

- **pin** – the pin number
- **mode** – the mode (DIRECT, INVERTED, or INTERNAL_PULLUP)

virtual void **mode**(uint8_t mode)

Changes the mode of the component.

float **read**() const

Directly reads value from the pin as 1 or 0 (bypasses mode, debounce, and engine).

int **rawRead()** const

Directly reads raw value from the pin as HIGH or LOW (bypasses mode, debounce, and engine).

inline virtual bool **isOn()**

Returns true iff the input is “on”.

inline virtual bool **putOn**(bool value)

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline virtual bool **rose()**

Returns true if the value rose.

inline virtual bool **fell()**

Returns true if the value fell.

inline virtual bool **changed()**

Returns true if the value changed.

inline virtual bool **toggle()**

Switches between on and off.

inline virtual int8_t **changeState()**

Difference between current and previous value of the unit.

inline virtual void **onRise**(EventCallback callback)

Registers event callback on rise event.

inline virtual void **onFall**(EventCallback callback)

Registers event callback on fall event.

inline virtual void **onChange**(EventCallback callback)

Registers event callback on change event.

inline virtual bool **isOff()**

Returns true iff the input is “off”.

inline virtual int **getInt()**

Returns value as integer (0 or 1).

inline virtual float **get()**

Returns value as float (either 0.0 or 1.0).

inline virtual bool **on()**

Sets output to “on” (ie. true, 1).

inline virtual bool **off()**

Sets output to “off” (ie. false, 0).

inline virtual float **put**(float value)

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

inline **operator bool**()

Operator that allows usage in conditional expressions.

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliSeconds**() const

Returns engine time in milliseconds.

inline uint64_t **microSeconds**() const

Returns engine time in microseconds.

inline unsigned long **nSteps**() const

Returns number of engine steps.

inline float **sampleRate**() const

Returns engine sample rate.

inline float **samplePeriod**() const

Returns enginesample period.

inline uint8_t **pin**() const

Returns the pin this component is attached to.

inline uint8_t **mode**() const

Returns the mode of the component.

inline virtual void **debounce**(float debounceTime = PLAQUETTE_DEFAULT_DEBOUNCE_WINDOW)

Apply smoothing to object.

inline virtual void **noDebounce**()

Remove smoothing.

inline virtual void **smooth**(float smoothTime = PLAQUETTE_DEFAULT_DEBOUNCE_WINDOW)

Deprecated.

Left for backwards compatibility.

Deprecated:

inline virtual void **noSmooth**()

Remove smoothing.

Deprecated:

inline uint8_t **debounceMode**() const

Returns the debounce mode.

inline void **debounceMode**(uint8_t mode)

Sets debounce mode.

Parameters

mode – the debounce mode (DEBOUNCE_DEFAULT, DEBOUNCE_LOCK_OUT or DEBOUNCE_PROMPT_DETECT)

virtual void **infiniteTimeWindow()**

Sets time window to infinite.

virtual void **noTimeWindow()**

Sets time window to no time window.

virtual void **timeWindow**(float seconds)

Changes the time window (expressed in seconds).

inline virtual float **timeWindow()** const

Returns the time window (expressed in seconds).

inline virtual bool **timeWindowIsInfinite()** const

Returns true if time window is infinite.

virtual void **cutoff**(float hz)

Changes the time window cutoff frequency (expressed in Hz).

virtual float **cutoff()** const

Returns the time window cutoff frequency (expressed in Hz).

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- [AnalogIn](#)
- [DigitalOut](#)
- [Bounce2 Arduino Library](#)

3.1.4 DigitalOut

A digital (ie. binary) output unit that can be switched “on” or “off”.

The unit is assigned to a specific **pin** on the board.

The mode specifies the behavior of the component attached to the pin:

- in **DIRECT** mode (default) the pin acts as the source of current and the component is “on” when the pin is “high” (Vcc, typically 5V)
- in **INVERTED** mode the source of current is external (Vcc) and the component is “on” when the pin is “low” (GND)

Example

Switches off an LED connected in “sink” mode after a timeout.

```
#include <Plaquette.h>

DigitalOut led(13, INVERTED);

void begin() {
    led.on();
}

void step() {
    // Switch the LED off after 5 seconds.
    if (seconds() > 5)
        led.off();
}
```

Reference

class **DigitalOut** : public pq::DigitalSource, public pq::PinConfig

A generic class representing a simple digital output.

Public Functions

DigitalOut(uint8_t pin, *Engine* &engine = *Engine::primary*())

Constructor with default mode DIRECT.

Parameters

pin – the pin number

DigitalOut(uint8_t pin, uint8_t mode, *Engine* &engine = *Engine::primary*())

Constructor.

Parameters

- **pin** – the pin number
- **mode** – the mode (DIRECT or INVERTED)

virtual void **mode**(uint8_t mode)

Changes the mode of the component.

void **write**(bool value)

Directly writes value to the pin (bypasses mode and engine).

void **write**(float value)

Directly writes value in [0, 1] to the pin (bypasses mode and engine).

void **rawWrite**(int value)

Directly writes HIGH or LOW value to the pin (bypasses mode and engine).

inline virtual bool **isOn**()

Returns true iff the input is “on”.

inline virtual bool **putOn**(bool value)

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline virtual bool **rose**()

Returns true if the value rose.

inline virtual bool **fell**()

Returns true if the value fell.

inline virtual bool **changed**()

Returns true if the value changed.

inline virtual bool **toggle**()

Switches between on and off.

inline virtual int8_t **changeState**()

Difference between current and previous value of the unit.

inline virtual void **onRise**(EventCallback callback)

Registers event callback on rise event.

inline virtual void **onFall**(EventCallback callback)

Registers event callback on fall event.

inline virtual void **onChange**(EventCallback callback)

Registers event callback on change event.

inline virtual bool **isOff**()

Returns true iff the input is “off”.

inline virtual int **getInt**()

Returns value as integer (0 or 1).

inline virtual float **get**()

Returns value as float (either 0.0 or 1.0).

inline virtual bool **on**()

Sets output to “on” (ie. true, 1).

inline virtual bool **off**()

Sets output to “off” (ie. false, 0).

inline virtual float **put**(float value)

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

inline **operator bool**()

Operator that allows usage in conditional expressions.

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliSeconds**() const

Returns engine time in milliseconds.

inline uint64_t **microSeconds**() const

Returns engine time in microseconds.

inline unsigned long **nSteps**() const

Returns number of engine steps.

inline float **sampleRate**() const

Returns engine sample rate.

inline float **samplePeriod**() const

Returns enginesample period.

inline uint8_t **pin**() const

Returns the pin this component is attached to.

inline uint8_t **mode**() const

Returns the mode of the component.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- *AnalogOut*
- *DigitalIn*

3.2 Generators

Source units that generate different kinds of signals.

3.2.1 Ramp

A source unit that generates a smooth transition between a starting value and an ending value over a given duration. The transition begins when `start()` is called, and the ramp's current value can be read or piped to an output at any time.

Parameters

Name	Description	Range	Setter	Getter	Flow
duration	Duration of the ramp transition (in seconds).	> 0	<code>duration(v)</code>	<code>duration()</code>	<code>Duration()</code>
speed	Rate of change (change-per-second).	> 0	<code>speed(v)</code>	<code>speed()</code>	<code>Speed()</code>

Events

When	Trigger	Callback
Timed process completes	<code>finished()</code>	<code>onFinish(c)</code>

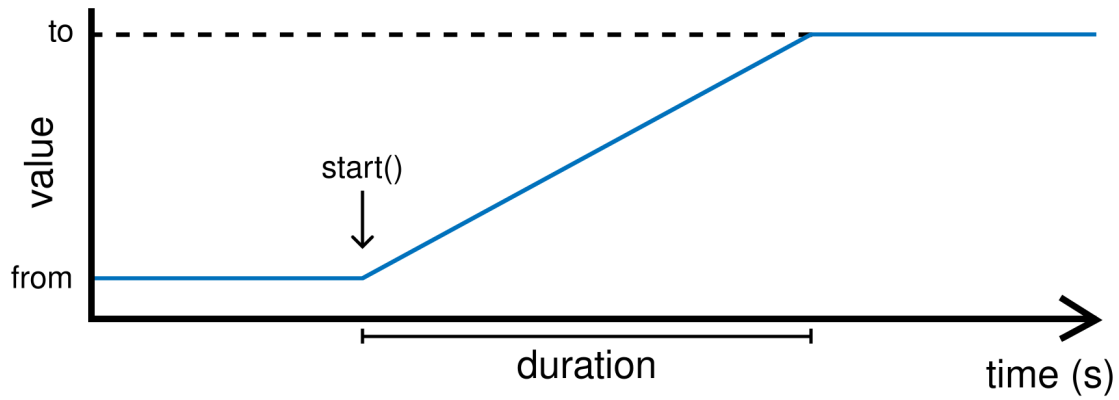
Basic Usage

A ramp is defined by three values:

- `from()` is the value at which the ramp starts (default: 0)
- `to()` is the value where the ramp ends (default: 1)
- `duration()` is the duration the ramp takes to go from `from` to `to`

It can then be started by calling the `start()` function.

Tip: As an alternative to `duration`, one can use the `speed` parameter instead to specify transition time in values-per-seconds. This is useful when we want to maintain a stable rate of change when the `from` and `to` values change, such as when controlling servo-motors.



Example

Pressing a button below triggers a two-second fade from off to full brightness:

```
#include <Plaquette.h>

Ramp fader(2.0); // 2-second ramp, from 0 to 1 by default

AnalogOut led(9);
DigitalIn button(2, INTERNAL_PULLUP);

void step() {
  // Press button to trigger the fade.
  if (button.rose())
    fader.start();

  // Flow ramp value to LED.
  fader >> led;
}
```

Easing

Ramps support easing functions to shape the transition curve for more expressive effects. Passing an easing function to `easing()` applies it to all subsequent transitions.

Please refer to [this page](#) for a full list of available easing functions.

Starting from the example above, add the following `begin()` function to specify an easing function. Easing `easeInSine` makes the fade start slowly and accelerate.

```
void begin() {  
    fader.easing(easeInSine); // slow start, accelerating towards the end  
}
```

Changing End Points

The starting value (`from`) and ending value (`to`) can be changed to produce different transitions. Setting `from` to 1.0 and `to` to 0.0 reverses the direction, fading the LED from full brightness down to off:

```
void begin() {  
    fader.fromTo(1.0, 0.0); // changes both from (1.0) and to (0.0)  
}
```

Values do not need to stay within [0, 1], they can take any values.

Example

This example uses a metronome to trigger a ramp. The ramp controls the frequency of a blinking LED.

```
#include <Plaquette.h>  
  
Ramp fader(5.0); // 5-second ramp  
  
Wave wave(SQUARE); // square wave  
DigitalOut led(13); // blinking LED  
  
Metronome metro(7.5); // 7.5-second metro  
  
void begin() {  
    fader.fromTo(2, 10); // ramp from 2 Hz to 10 Hz  
}  
  
void step() {  
    // Metronome triggers the ramp.  
    if (metro)  
        fader.start();  
  
    // Flow ramp value wave frequency.  
    fader >> wave.Frequency();  
  
    // Blink LED.  
    wave >> led;  
}
```


The go() Function

The `go()` function provides a simple way to immediately launch a ramp from the current value to another, or simply from the current value towards a new goal. It sets `to` and `duration` or `speed` in a single call and immediately starts the transition from the current value.

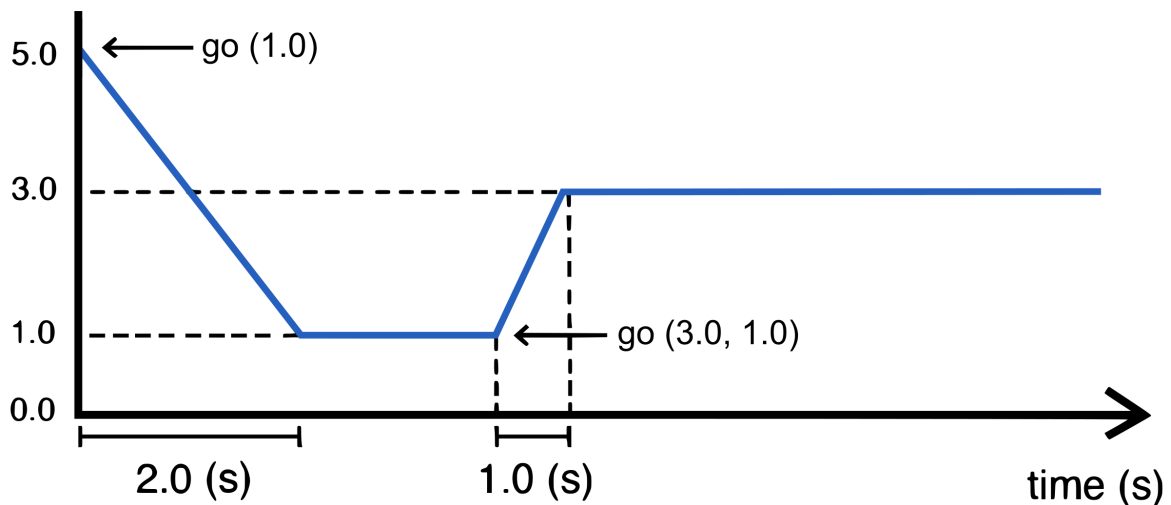
The two main forms are:

- `go(to)`: transition from the ramp's current value to `to` using the current duration or speed.
- `go(to, durationOrSpeed)`: transition from the ramp's current value to `to` in `duration` seconds OR `speed`, depending on the current mode

Tip: An optional easing function can be passed as the last argument of any `go()` call. For example, `go(2.0, easeInSine)` and `go(2.0, 3.0, easeInSine)` go to value 2.0 using easing function `easeInSine` (3.0 is the value of duration or speed in the second version).

The following diagram shows what happens to a ramp signal:

- Initially the ramp is at value 5.0 with duration of 2.0 seconds.
- Calling `go(1.0)` starts a ramp from 5.0 to 1.0, using current duration of 2.0 seconds.
- Later, calling `go(3.0, 1.0)` starts ramping from current value to value 3.0, overriding duration with a 1.0 second duration.



Example

The example below uses `go()` to create a continuous random walk, chaining from the current value to a new random target each time the previous transition completes. It uses speed instead of duration.

```
#include <Plaquette.h>

Ramp zigZagRamp;

Plotter plotter(115200);

void begin() {
  // Set speed to 1.0 per second.
  zigZagRamp.speed(1.0);
  // Apply an easing function (optional).
  zigZagRamp.easing(easeOutSine);
  // Go from zero (initial value) to a random value.
  zigZagRamp.go( randomFloat(-10.0, 10.0) );
}

void step() {
  if (zigZagRamp.finished()) // event: ramp finished
  {
    // Ramp from current value to new random value, increasing speed by 1 each time.
    zigZagRamp.go(zigZagRamp + randomFloat(-10.0, 10.0), zigZagRamp.speed() + 1);
  }

  // Send ramp value to plotter for visualization.
  zigZagRamp >> plotter;
}
```

Ramp as a Timer

A ramp can be used as a timer (like an analog version of *Alarm*). By default, ramp goes from 0 to 1 (0% to 100%) over a given duration. The ramp can thus be used as a measure of progress over time. The example below uses the ramp value to cycle through four phases, changing the frequency of a LED pulse at each transition:

```
#include <Plaquette.h>

// Ramp over 5 seconds. Default ramp always runs from 0 to 1.
Ramp timer(5.0);

// Square wave oscillator driving the LED.
Wave pulse(SQUARE);

// Built-in LED.
DigitalOut led(LED_BUILTIN);

void begin() {
  timer.start(); // start the timer
}
```

(continues on next page)

(continued from previous page)

```

void step() {
    // Change wave frequency based on progress thresholds.
    if (timer < 0.25) {
        pulse.frequency(0.5); // slow pulse: warming up
    }
    else if (timer < 0.5) {
        pulse.frequency(3.0); // fast pulse: running
    }
    else if (timer < 0.75) {
        pulse.frequency(1.0); // medium pulse: cooling down
    }
    else {
        pulse.frequency(0.25); // very slow pulse: standby
    }

    // Flow wave to LED.
    pulse >> led;

    // Restart the timer when it completes, increasing duration each time.
    if (timer.finished()) {
        timer.duration( timer.duration() + 5 );
        timer.start();
    }
}

```

Reference

class **Ramp** : public pq::Unit, public pq::AbstractTimer

Provides a ramping / tweening mechanism that allows smooth transitions between two values.

Public Functions

Ramp(*Engine* &engine = *Engine::primary()*)

Constructor.

Parameters

engine – the engine running this unit

Ramp(float duration, *Engine* &engine = *Engine::primary()*)

Constructor with duration.

Parameters

- **duration** – duration of the ramp
- **engine** – the engine running this unit

virtual float **get**()

Returns value of ramp.

virtual float **put**(float value)

Forces value in the ramp.

If this happens while the ramp is running, it will interrupt the ramp.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

void **easing**(easing_function easing)

Sets easing function to apply to ramp.

Parameters

easing – the easing function

inline void **noEasing**()

Remove easing function (linear/no easing).

inline virtual float **to**() const

Returns destination value of the ramp,.

virtual void **to**(float to)

Assigns destination value of the ramp, starting from current value.

Parameters

to – the final value

inline virtual float **from**() const

Returns initial value of the ramp.

virtual void **from**(float from)

Assign initial value of the ramp.

Parameters

from – the initial value

virtual void **fromTo**(float from, float to)

Assign initial and final values of the ramp.

Parameters

- **from** – the initial value
- **to** – the final value

inline virtual float **duration**() const override

Returns the duration of the ramp (in seconds).

virtual void **duration**(float duration) override

Sets the duration of the ramp (in seconds).

virtual void **speed**(float speed)

Sets the speed (rate of change) of the ramp in change-per-second.

virtual float **speed**() const

Returns speed (rate of change) of the ramp in change-per-second.

inline Parameter **Speed()**

Returns speed (rate of change) as a parameter.

virtual void **start()**

Starts/restarts the ramp. Will repeat the last ramp.

virtual void **go**(float from, float to, float durationOrSpeed, easing_function easing = 0)

Starts a new ramp.

Parameters

- **from** – the initial value
- **to** – the final value
- **durationOrSpeed** – the duration of the ramp (in seconds) or speed (in change-per-second) depending on mode
- **easing** – the easing function (optional).

virtual void **go**(float to, float durationOrSpeed, easing_function easing = 0)

Starts a new ramp, starting from current value.

Parameters

- **to** – the final value
- **durationOrSpeed** – the duration of the ramp (in seconds) or speed (in change-per-second) depending on mode
- **easing** – the easing function (optional)

virtual void **go**(float to, easing_function easing = 0)

Starts a new ramp, starting from current value (keeping the same duration/speed).

Parameters

- **to** – the final value
- **easing** – the easing function (optional)

virtual void **mode**(uint8_t mode)

Changes the mode of the component (RAMP_DURATION or RAMP_SPEED).

inline uint8_t **mode**() const

Returns the mode of the component (RAMP_DURATION or RAMP_SPEED).

inline virtual bool **finished**()

Returns true iff the ramp just finished its process this step.

inline virtual void **onFinish**(EventCallback callback)

Registers event callback on finish event.

virtual void **setTime**(float time)

Forces current time (in seconds).

float **durationToSpeed**(float duration) const

Returns speed based on duration.

float **speedToDuration**(float speed) const

Returns duration based on speed.

virtual void **start**(float to, float durationOrSpeed, easing_function easing = 0)

Deprecated:

virtual void **start**(float from, float to, float durationOrSpeed, easing_function easing = 0)

Deprecated:

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliSeconds**() const

Returns engine time in milliseconds.

inline uint64_t **microSeconds**() const

Returns engine time in microseconds.

inline unsigned long **nSteps**() const

Returns number of engine steps.

inline float **sampleRate**() const

Returns engine sample rate.

inline float **samplePeriod**() const

Returns enginesample period.

inline **operator float**()

Object can be used directly to access its value.

inline explicit **operator bool**()

Operator that allows usage in conditional expressions.

virtual void **start**(float duration)

Starts/restarts the chronometer with specific duration.

inline virtual Parameter **Duration**()

Returns duration as a parameter.

virtual float **progress**() const

The progress of the timer process (in %).

inline virtual bool **isFinished**() const

Returns true iff the chronometer has finished its process.

inline virtual bool **isComplete**() const

Deprecated:

virtual void **pause**()

Interrupts the chronometer.

virtual void **resume**()

Resumes process.

inline virtual float **elapsed**() const

The time currently elapsed by the chronometer (in seconds).

virtual bool **hasPassed**(float timeout) const

Returns true iff elapsed time has passed given timeout.

virtual bool **hasPassed**(float timeout, bool restartIfPassed)

Deprecated:

Returns true iff elapsed time has passed given timeout (optional argument to automatically restart if true).

virtual void **addTime**(float time)

Adds/subtracts time to the chronometer.

inline virtual bool **isRunning**() const

Returns true iff the chronometer is currently running.

inline bool **isStarted**() const

Deprecated:

virtual void **stop**()

Interrupts and resets to zero.

virtual void **togglePause**()

Toggles pause/unpause.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

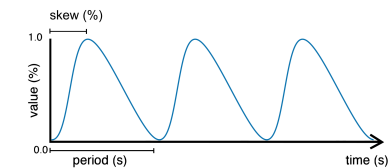
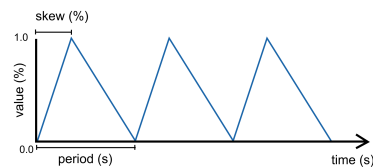
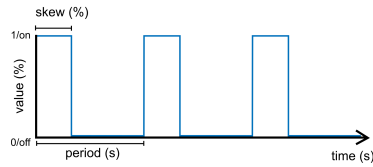
- *Alarm*
- *Chronometer*
- *Easings*
- *Metronome*
- *Wave*

3.2.2 Wave

An analog source unit that generates a *wave* signal in range [0, 1].

Shape

There are three potential wave types that can be set using the shape parameter: *SQUARE* (default), *TRIANGLE* or *SINE*.



Parameters

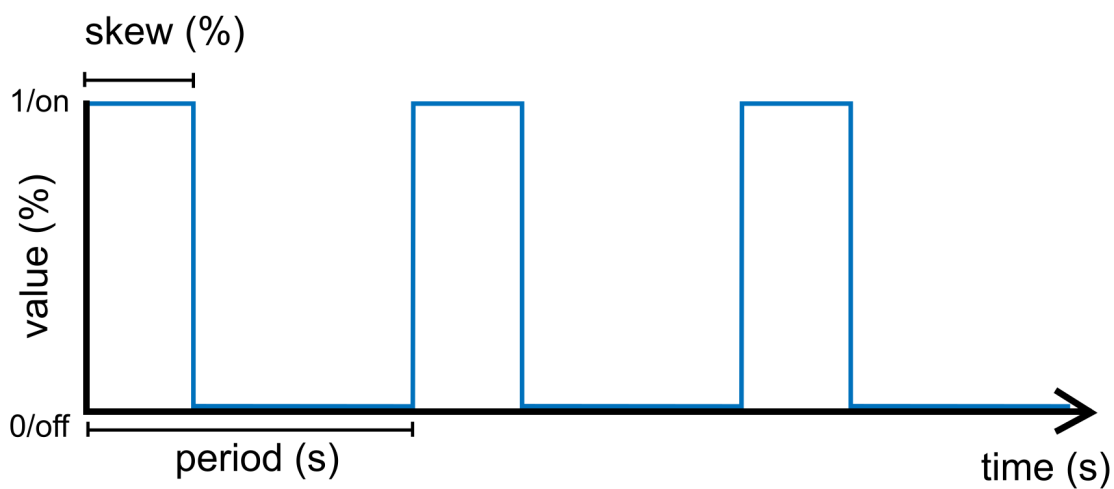
Name	Description	Range	Setter	Getter	Flow
period	Duration of one complete cycle (in seconds).	> 0	period(v)	period()	Period()
frequency	Cycles per second, inverse of period (in Hz).	> 0	frequency(v)	frequency()	Frequency()
bpm	Cycles per minute (alternative to frequency).	> 0	bpm(v)	bpm()	Bpm()
skew	Balance between rising/falling portions of cycle.	[0, 1]	skew(v)	skew()	Skew()
phase	Current point in the wave cycle (as % of period).	[0, 1]	phase(v)	phase()	Phase()
phaseShift	Phase offset (as % of period).	[0, 1]	phaseShift(v)	phaseShift()	PhaseShift()
jitter	<i>Randomness level</i> for timing variation.	[0, 1]	jitter(v)	jitter()	Jitter()

Events

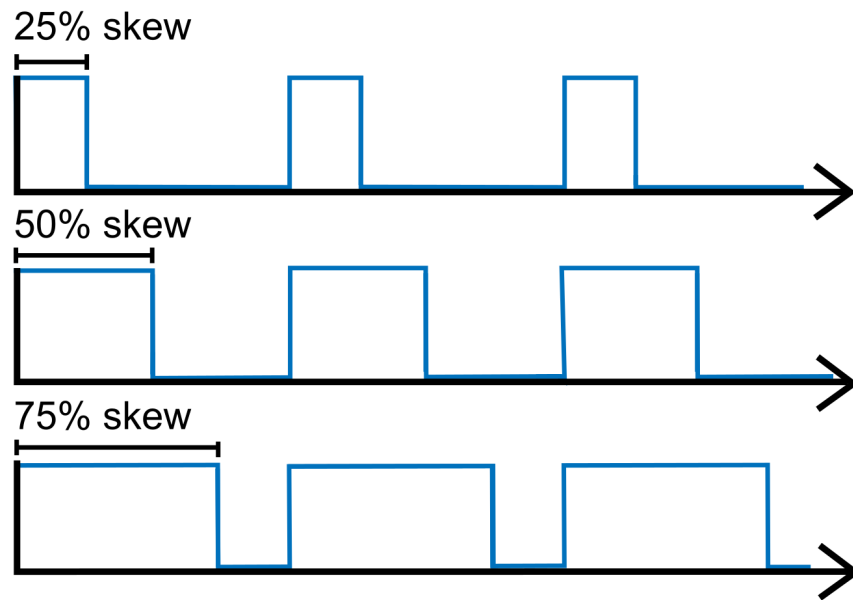
When	Trigger	Callback
Wave completes a full cycle	<code>passedPeriod()</code>	<code>onPassPeriod(c)</code>
Wave passes its skew point	<code>passedSkew()</code>	<code>onPassSkew(c)</code>

Square Wave

Generates a [square wave](#) signal when the `shape` parameter is set to `SQUARE`. The square wave is the default for this parameter.



For the `SQUARE` wave, the *skew* represents the proportion of time (expressed as a percentage) in each cycle (period) during which the wave is “on” – in other words, its *duty cycle*.



Example

Makes the built-in LED blink with a period of 4 seconds. Because the duty cycle is set to 25%, the LED will stay on for 1 second and then off for 3 seconds.

```
#include <Plaquette.h>

DigitalOut led(13);

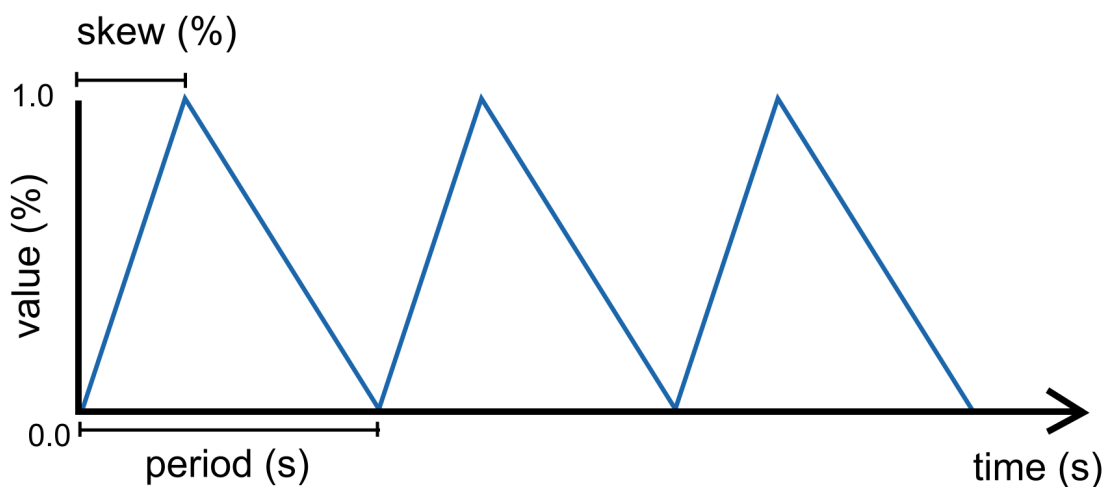
Wave blinkOsc(SQUARE, 4.0);

void begin() {
    blinkOsc.skew(0.25); // Sets the duty cycle to 25%
}

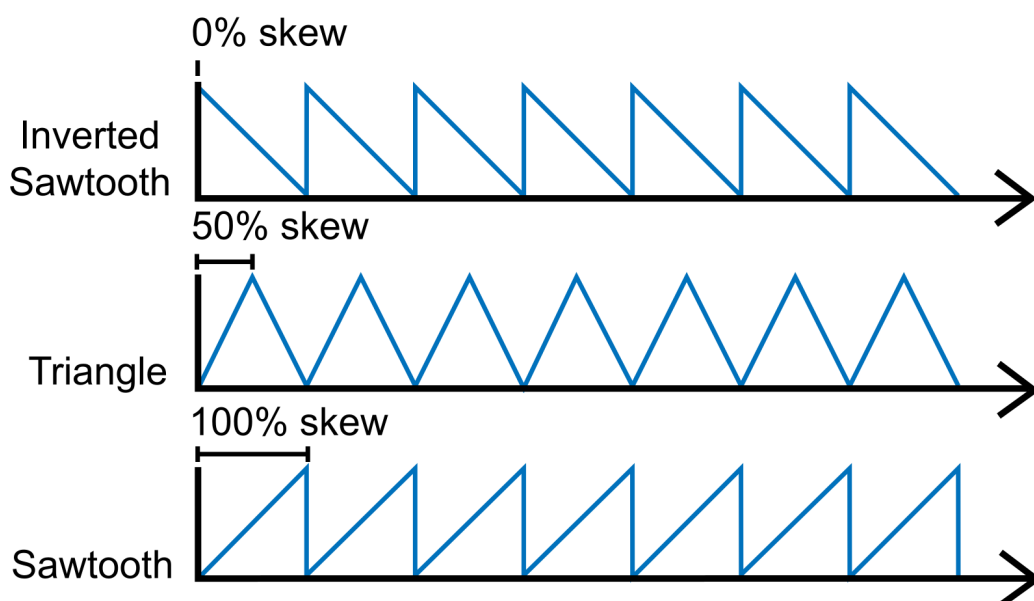
void step() {
    blinkOsc >> led;
}
```

Triangle Wave

Generates a wave such as the [triangle wave](#) and the [sawtooth wave](#) when the shape parameter is set to TRIANGLE.



In this case, the skew parameter represents the “turning point” during the period at which the signals reaches its maximum and starts going down again. Changing the skew allows to generate different kinds of triangular-shaped waves. For example, by setting skew to 1.0 (100%) one obtains a *sawtooth* wave; by setting it to 0.0 (0%) an *inverted sawtooth* is created; anything in between generates different flavors of *triangle* waves.



Example

Controls a set of traffic lights that go: red, yellow, green, red, yellow, green, and so on. It uses a sawtooth to iterate through these three states.

```
#include <Plaqueette.h>

DigitalOut green(10);
DigitalOut yellow(11);
DigitalOut red(12);

Wave osc(TRIANGLE, 10.0);

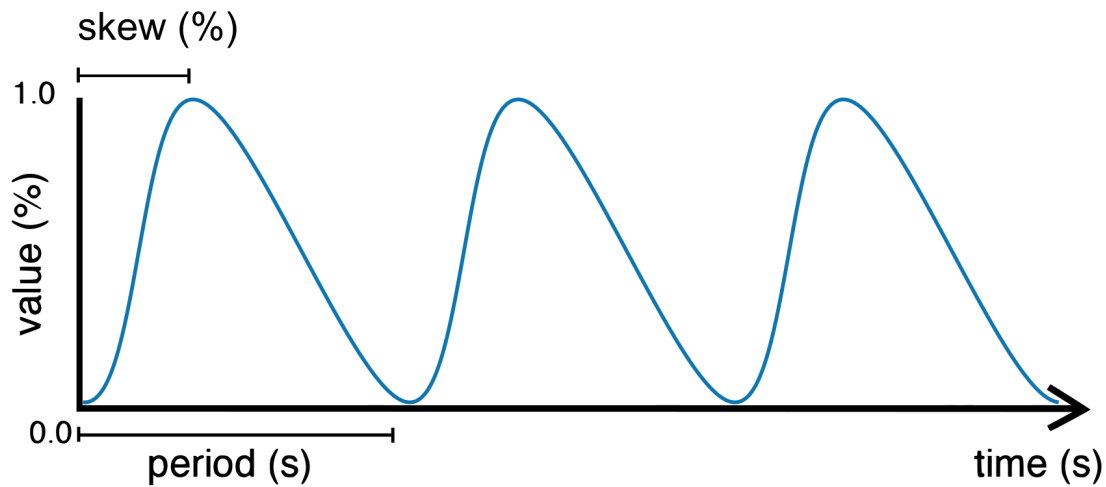
void begin() {
  // Setting skew to 1.0 converts triangle wave to a sawtooth wave.
  osc.skew(1.0);
}

void step() {
  // Shut down all lights.
  0 >> led >> yellow >> green;
  // Switch appropriate LED.
  if (osc < 0.4)
    green.on();
  else if (osc < 0.6)
    yellow.on();
  else
    red.on();
}
```

Sine Wave

Generates a sinusoid or *sine wave* when the *shape* parameter is set to SINE. The signal is remapped to oscillate between 0 and 1 (rather than -1 and 1 as the traditional sine wave function).

Here, the *skew* parameter controls when the sine wave reaches its peak within a cycle. A skew value of 0.5 (default) yields a standard symmetric sine wave. Lower values shift the peak earlier (left-skewed), while higher values shift it later (right-skewed), allowing for asymmetric sine shapes while preserving smoothness.



Example

Pulses an LED.

```
#include <Plaquette.h>

AnalogOut led(9);

Wave osc(SINE);

void begin() {
  osc.frequency(5.0); // frequency of 5 Hz
}

void step() {
  osc >> led;
}
```

Randomization

In addition to controlling shape, period, and skew, ``Wave`` units can also generate **randomized oscillations** in a similar manner as *Metronome* units using the `randomize` function. This allows the oscillation to feel less mechanical and more organic, closer to natural rhythms like breathing, heartbeat variations, or the flicker of firelight.

When randomness is active, the wave no longer produces perfectly periodic oscillations. Instead, each cycle's duration is perturbed according to the chosen randomness level. However, on the long run, the average period of oscillation will match the wave's period parameter.

Example

Pulse an LED. Uses a low-frequency oscillator (LFO) to slowly modify the wave's randomness.

```
#include <Plaquette.h>

AnalogOut led(9);

Wave osc(SINE); // Default value of wave period is 1 second.

Wave lfo(SINE, 20.0); // 20-seconds oscillator

void step() {
    osc.randomize(lfo);
    osc >> led;
}
```

class **Wave** : public pq::AbstractWave

Sine oscillator. Phase is expressed as % of period.

Public Functions

Wave(float period, *Engine* &engine = *Engine::primary()*)

Constructor (creates default square wave).

Parameters

- **period** – the period of oscillation (in seconds)
- **engine** – the engine running this unit

explicit **Wave**(WaveShape shape, *Engine* &engine = *Engine::primary()*)

Constructor.

Defaults to period of 1 second.

Parameters

- **shape** – the wave shape
- **engine** – the engine running this unit

Wave(float period, float skew, *Engine* &engine = *Engine::primary()*)

Constructor (creates default square wave).

Parameters

- **period** – the period of oscillation (in seconds)
- **skew** – the duty-cycle as a value in [0, 1]
- **engine** – the engine running this unit

Wave(WaveShape shape, float period, *Engine* &engine = *Engine::primary()*)

Constructor.

Parameters

- **shape** – the wave shape

- **period** – the period of oscillation (in seconds)
- **engine** – the engine running this unit

Wave(WaveShape shape, float period, float skew, *Engine* &engine = *Engine::primary*())

Constructor.

Parameters

- **shape** – the wave shape
- **period** – the period of oscillation (in seconds)
- **skew** – the duty-cycle as a value in [0, 1]
- **engine** – the engine running this unit

void **shape**(WaveShape shape)

Sets wave shape.

Parameters

shape – the wave shape (SQUARE, TRIANGLE, SINE)

inline WaveShape **shape**() const

Returns current wave shape.

virtual float **get**()

Returns value in [0, 1].

virtual float **shiftBy**(float phaseShift) const

Returns oscillator's value with given phase shift (in % of period).

Supports values outside [0,1], which will be wrapped accordingly. Eg. `shiftBy(0.2)` returns future value of oscillator after 20% of its period would have passed.

Parameters

phaseShift – the phase shift (in % of period)

Returns

the value of oscillator with given phase shift

virtual float **shiftByTime**(float timeShift) const

Returns oscillator's value with given phase shift expressed in time (in seconds).

Parameters

timeShift – the shift in time (seconds)

Returns

the value of oscillator with time shift

virtual float **atPhase**(float phase) const

Returns the oscillator's value at a given absolute phase (in % of period).

Supports values outside [0,1], which will be wrapped accordingly. Eg: `atPhase(0.25)` returns the oscillator value at 25% of its period.

Parameters

phase – the absolute phase at which to evaluate the oscillator (in % of period)

Returns

the value of the oscillator at the given phase

virtual void **amplitude**(float amplitude)

Sets the amplitude of the wave.

Parameters

amplitude – a value in [0, 1] that determines the amplitude of the wave (centered at 0.5).

inline virtual float **amplitude**() const

Returns the amplitude of the wave.

virtual void **skew**(float skew)

Sets the skew of the signal as a % of period.

Parameters

skew – the skew as a value in [0, 1]

inline virtual float **skew**() const

Returns the skew of the signal.

inline virtual Parameter **Skew**()

Returns the skew as a parameter.

inline virtual void **width**(float width)

Deprecated:

Sets the width of the signal as a % of period.

Parameters

width – the skew as a value in [0, 1]

inline virtual float **width**() const

Deprecated:

Returns the skew of the signal.

inline virtual bool **passedPeriod**() const

Returns true on the step where the wave passed the period (end cycle).

inline virtual bool **passedSkew**() const

Returns true on the step where the wave passed the skew point.

virtual void **onPassPeriod**(EventCallback callback)

Registers event callback on wave pass period event.

virtual void **onPassSkew**(EventCallback callback)

Registers event callback on wave pass skew point event.

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliSeconds**() const

Returns engine time in milliseconds.

inline uint64_t **microSeconds**() const

Returns engine time in microseconds.

inline unsigned long **nSteps()** const

Returns number of engine steps.

inline float **sampleRate()** const

Returns engine sample rate.

inline float **samplePeriod()** const

Returns enginesample period.

inline **operator float()**

Object can be used directly to access its value.

inline virtual float **put**(float value)

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline explicit **operator bool()**

Operator that allows usage in conditional expressions.

virtual void **start()**

Starts/restarts the oscillator.

virtual void **period**(float period)

Sets the period (in seconds).

Parameters

period – the period of oscillation (in seconds)

inline virtual float **period()** const

Returns the period (in seconds).

inline Parameter **Period()**

Returns the period as a parameter.

virtual void **frequency**(float frequency)

Sets the frequency (in Hz).

Parameters

frequency – the frequency of oscillation (in Hz)

inline virtual float **frequency()** const

Returns the frequency (in Hz).

inline Parameter **Frequency()**

Returns the frequency as a parameter.

virtual void **bpm**(float bpm)

Sets the frequency in beats-per-minute.

Parameters

bpm – the frequency of oscillation (in BPM)

inline virtual float **bpm()** const

Returns the frequency (in BPM).

inline Parameter **Bpm()**

Returns the BPM as a parameter.

virtual void **phase**(float phase)

Sets the phase at % of period.

Parameters

phase – the phase (in % of period)

inline virtual float **phase**() const

Returns the phase (in % of period).

inline Parameter **Phase**()

Returns the phase as a parameter.

virtual void **phaseShift**(float phaseShift)

Sets the phase shift (ie.

the offset, in % of period).

Warning: This function is disabled if randomness() > 0.

Parameters

phaseShift – the phase shift (in % of period)

virtual float **phaseShift**() const

Returns the phase shift (ie.

the offset, in % of period).

Warning: This function always returns 0 when randomness() > 0.

inline Parameter **PhaseShift**()

Returns the phase shift as a parameter.

virtual void **jitter**(float jitter)

Sets the jittering level in [0, 1] (0: no jitter, 1: max jitter).

virtual float **jitter**() const

Returns the randomness level in [0, 1].

inline virtual void **noJitter**()

Disables jittering.

inline Parameter **Jitter**()

Returns the jitter as a parameter.

virtual float **jitteredPeriod**() const

Returns the period actually used for the current cycle.

When jitter() == 0, this is identical to period(). When jitter() > 0, this returns the stochastic (jittered) period currently in effect.

virtual float **jitteredFrequency()** const

Returns the frequency actually used for the current cycle.

When jitter() == 0, this is identical to frequency(). When jitter() > 0, this returns the stochastic (jittered) frequency currently in effect.

virtual float **timeToPhase**(float time) const

Utility function to convert time to phase.

Parameters

time – relative time in seconds

Returns

the equivalent phase

virtual void **setTime**(float time)

Forces current time (in seconds).

Warning: This function is disabled if randomness() > 0.

virtual void **addTime**(float time)

Adds time to current time (in seconds).

Warning: This function is disabled if randomness() > 0.

inline virtual bool **isRunning**() const

Returns true iff the wave is currently running.

inline virtual bool **isForward**() const

Returns true iff the wave is moving forward in time.

inline virtual void **setForward**(bool isForward)

Sets the direction of oscillation.

Parameters

isForward – true iff the wave is moving forward in time

inline virtual void **forward**()

Sets the direction of oscillation to move forward in time.

inline virtual void **reverse**()

Sets the direction of oscillation to move backward in time.

inline virtual void **toggleReverse**()

Toggles the direction of oscillation.

virtual void **stop**()

Interrupts and resets to zero.

virtual void **pause**()

Interrupts process.

virtual void **resume**()

Resumes process.

virtual void **togglePause()**

Toggles pause/unpause.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- *Metronome*
- *Ramp*

3.3 Timing

Time-management source units.

3.3.1 Alarm

An alarm clock digital source unit. Counts time and becomes “on” when time is up. The alarm can be started, stopped, and resumed.

When started, the alarm stays “off” until it reaches its timeout duration, after which it becomes “on”.

Parameters

Name	Description	Range	Setter	Getter	Flow
duration	Time before the alarm triggers (in seconds).	> 0	duration(v)	duration()	Duration()

Events

When	Trigger	Callback
Digital value toggles (on/off)	changed()	onChange(c)
Digital value goes from on to off	fell()	onFall(c)
Timed process completes	finished()	onFinish(c)
Digital value goes from off to on	rose()	onRise(c)

Example

Uses an alarm to activate built-in LED. Button is used to reset the alarm at random periods of time.

```
#include <Plaquette.h>

Alarm myAlarm(2.0); // an alarm with 2 seconds duration

DigitalOut led(13);

DigitalIn button(2, INTERNAL_PULLUP);

void begin() {
    myAlarm.start(); // start alarm
}

void step() {
    // Activate LED when alarm rings.
    myAlarm >> led; // the alarm will stay "on" until it is stopped or restarted

    // Reset alarm when button is pushed.
    if (myAlarm && button.rose())
    {
        // Restarts the alarm with a random duration between 1 and 5 seconds.
        myAlarm.duration(randomFloat(1.0, 5.0));
        myAlarm.start();
    }
}
```

Reference

class **Alarm** : public pq::DigitalSource, public pq::AbstractTimer
Chronometer class which becomes “on” after a given duration.

Public Functions

Alarm(*Engine* &engine = *Engine::primary()*)

Constructor.

Parameters

engine – the engine running this unit

Alarm(float duration, *Engine* &engine = *Engine::primary()*)

Constructor with duration.

Parameters

- **duration** – duration of the alarm
- **engine** – the engine running this unit

inline virtual bool **finished**()

Returns true iff the alarm just finished its process this step.

inline virtual void **onFinish**(EventCallback callback)

Registers event callback on finish event.

virtual void **setTime**(float time)

Set alarm at specific time.

inline virtual bool **isOn**()

Returns true iff the input is “on”.

inline virtual bool **putOn**(bool value)

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline virtual bool **rose**()

Returns true if the value rose.

inline virtual bool **fell**()

Returns true if the value fell.

inline virtual bool **changed**()

Returns true if the value changed.

inline virtual bool **toggle**()

Switches between on and off.

inline virtual int8_t **changeState**()

Difference between current and previous value of the unit.

inline virtual void **onRise**(EventCallback callback)

Registers event callback on rise event.

inline virtual void **onFall**(EventCallback callback)

Registers event callback on fall event.

inline virtual void **onChange**(EventCallback callback)

Registers event callback on change event.

inline virtual bool **isOff**()

Returns true iff the input is “off”.

inline virtual int **getInt**()

Returns value as integer (0 or 1).

inline virtual float **get**()

Returns value as float (either 0.0 or 1.0).

inline virtual bool **on**()

Sets output to “on” (ie. true, 1).

inline virtual bool **off**()

Sets output to “off” (ie. false, 0).

inline virtual float **put**(float value)

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

inline **operator bool**()

Operator that allows usage in conditional expressions.

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliSeconds**() const

Returns engine time in milliseconds.

inline uint64_t **microSeconds**() const

Returns engine time in microseconds.

inline unsigned long **nSteps**() const

Returns number of engine steps.

inline float **sampleRate**() const

Returns engine sample rate.

inline float **samplePeriod**() const

Returns enginesample period.

virtual void **start**()

Starts/restarts the chronometer.

virtual void **start**(float duration)

Starts/restarts the chronometer with specific duration.

virtual void **duration**(float duration)

Sets the duration of the chronometer.

inline virtual float **duration**() const

Returns duration.

inline virtual Parameter **Duration**()

Returns duration as a parameter.

virtual float **progress**() const

The progress of the timer process (in %).

inline virtual bool **isFinished**() const

Returns true iff the chronometer has finished its process.

inline virtual bool **isComplete**() const

Deprecated:

virtual void **pause()**

Interrupts the chronometer.

virtual void **resume()**

Resumes process.

inline virtual float **elapsed()** const

The time currently elapsed by the chronometer (in seconds).

virtual bool **hasPassed**(float timeout) const

Returns true iff elapsed time has passed given timeout.

virtual bool **hasPassed**(float timeout, bool restartIfPassed)

Deprecated:

Returns true iff elapsed time has passed given timeout (optional argument to automatically restart if true).

virtual void **addTime**(float time)

Adds/subtracts time to the chronometer.

inline virtual bool **isRunning**() const

Returns true iff the chronometer is currently running.

inline bool **isStarted**() const

Deprecated:

virtual void **stop**()

Interrupts and resets to zero.

virtual void **togglePause**()

Toggles pause/unpause.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- *Chronometer*
- *Metronome*
- *Ramp*
- *Wave*

3.3.2 Chronometer

An analog unit that counts time in seconds. It can be started, stopped, paused, and resumed.

Example

Uses a chronometer to change the frequency a blinking LED. Restarts after 10 seconds.

```
#include <Plaquette.h>

Chronometer chrono;

DigitalOut led(13);

Wave osc(1.0); // a square oscillator

void begin() {
    chrono.start(); // start chrono
}

void step() {
    // Adjust oscillator's duty cycle according to current timer progress.
    osc.frequency(chrono);

    // Apply oscillator to LED state.
    osc >> led;

    // If the chronometer reaches 10 seconds: restart it.
    if (chrono >= 10.0)
    {
        // Restarts the chronometer.
        chrono.start();
    }
}
```

Reference

class **Chronometer** : public pq::Unit, public pq::AbstractChronometer

Public Functions

Chronometer(*Engine* &engine = *Engine::primary()*)

Constructor.

inline virtual float **get**()

Returns elapsed time since start (in seconds).

virtual float **put**(float value)

Sets current time in seconds and returns it.

inline float **seconds()** const

Returns engine time in seconds.

inline uint32_t **milliSeconds()** const

Returns engine time in milliseconds.

inline uint64_t **microSeconds()** const

Returns engine time in microseconds.

inline unsigned long **nSteps()** const

Returns number of engine steps.

inline float **sampleRate()** const

Returns engine sample rate.

inline float **samplePeriod()** const

Returns enginesample period.

inline **operator float()**

Object can be used directly to access its value.

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

This function guarantees that the value is within [toLow, toHigh]. If the unit's values are unbounded, returns `get()` constrained to [toLow, toHigh].

inline explicit **operator bool()**

Operator that allows usage in conditional expressions.

virtual void **pause()**

Interrupts the chronometer.

virtual void **resume()**

Resumes process.

inline virtual float **elapsed()** const

The time currently elapsed by the chronometer (in seconds).

virtual bool **hasPassed**(float timeout) const

Returns true iff elapsed time has passed given timeout.

virtual bool **hasPassed**(float timeout, bool restartIfPassed)

Deprecated:

Returns true iff elapsed time has passed given timeout (optional argument to automatically restart if true).

virtual void **setTime**(float time)

Forces current time (in seconds).

virtual void **addTime**(float time)

Adds/subtracts time to the chronometer.

inline virtual bool **isRunning()** const

Returns true iff the chronometer is currently running.

inline bool **isStarted()** const

Deprecated:

virtual void **start()**

Starts/restarts.

virtual void **stop()**

Interrupts and resets to zero.

virtual void **togglePause()**

Toggles pause/unpause.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- [Alarm](#)
- [Metronome](#)
- [Ramp](#)

3.3.3 Metronome

A digital source unit that emits a pulse at a regular pace. It is conceptually similar to an [Alarm](#) that automatically restarts after each trigger. Metronomes are ideal for **triggering periodic events** in interactive systems, such as blinking LEDs, synchronizing motor movements, or scheduling actions at regular intervals. Each time the metronome “ticks,” it evaluates to **true** for a single frame, and **false** otherwise.

Parameters

Name	Description	Range	Setter	Getter	Flow
period	Duration of one complete cycle (in seconds).	> 0	<code>period(v)</code>	<code>period()</code>	<code>Period()</code>
frequency	Cycles per second, inverse of period (in Hz).	> 0	<code>frequency(v)</code>	<code>frequency()</code>	<code>Frequency()</code>
bpm	Cycles per minute (alternative to frequency).	> 0	<code>bpm(v)</code>	<code>bpm()</code>	<code>Bpm()</code>
phase	Current point in the cycle (as % of period).	[0, 1]	<code>phase(v)</code>	<code>phase()</code>	<code>Phase()</code>
phaseShift	Phase offset (as % of period).	[0, 1]	<code>phaseShift(v)</code>	<code>phaseShift()</code>	<code>PhaseShift()</code>
jitter	<i>Randomness level</i> for timing variation.	[0, 1]	<code>jitter(v)</code>	<code>jitter()</code>	<code>Jitter()</code>

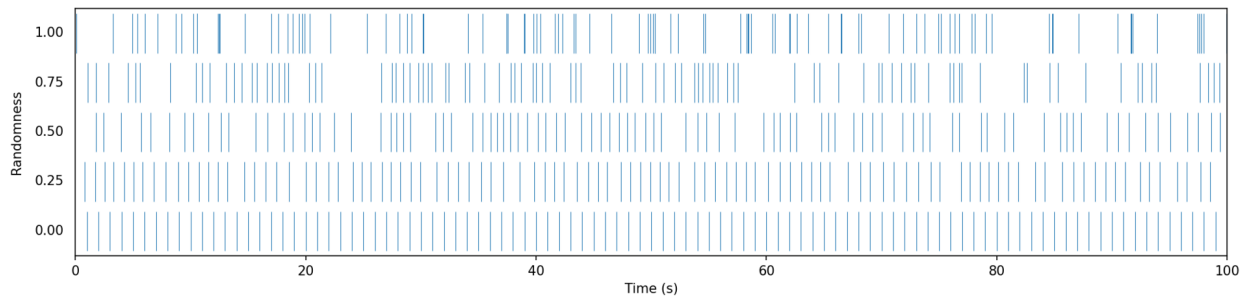
Events

When	Trigger	Callback
Unit emits a pulse	unit itself	onBang(c)

Randomization

Metronomes can be used to generate **randomized patterns** using the `jitter()` function. This makes their ticks feel less mechanical and closer to natural rhythms such as raindrops falling, typing on a keyboard, or the reactions of a living entity. When activating randomization, the metronome does not trigger at perfectly regular intervals anymore. Instead, the length of each cycle is perturbed according to the chosen randomness level. At low values (close to 0), the timing remains close to steady with gentle variation. At high values (close to 1), the ticks may cluster together or leave longer pauses, while still averaging the correct period over time. This means that even with randomness, the metronome respects its `period` setting in the long run, but the *spacing* between individual ticks is altered.

This graph shows the effect of different randomization levels on event triggers (all metronomes have a one-second period):



Basic Example

The simplest use of a metronome is to trigger an LED blink every 0.5 seconds:

```
#include <Plaqueette.h>

Metronome myMetro(0.5); // tick every 0.5 seconds
DigitalOut led(13);

void step() {
  if (myMetro) {
    led.toggle();
  }
}
```

Multiple Metronomes

You can run several metronomes in parallel to control independent events:

```
#include <Plaquette.h>

DigitalOut led(LED_BUILTIN);
Metronome metroFast(0.5);    // Blink at 2 Hz
Metronome metroSlow(2.0);    // Trigger every 2 seconds
Monitor monitor(115200);     // create a monitor object

void step() {
  if (metroFast) led.toggle();
  if (metroSlow) println("Tick!");
}
```

Randomized Metronome

The following sketch blinks the LED at an *average* rate of once per second, but with irregular spacing:

```
#include <Plaquette.h>

DigitalOut led(13);
Metronome metro(1.0); // base period of 1 second

void begin() {
  metro.jitter(1.0); // 100% jittering level
}

void step() {
  if (metro) {
    led.toggle();
  }
}
```

Reference

class **Metronome** : public pq::DigitalUnit, public pq::AbstractOscillator

Chronometer digital unit which emits 1/true/"on" for one frame, at a regular pace.

Public Functions

Metronome(*Engine* &engine = *Engine::primary()*)

Constructor.

Parameters

engine – the engine running this unit

Metronome(float period, *Engine* &engine = *Engine::primary()*)

Constructor.

Parameters

- **period** – the period of oscillation (in seconds)
- **engine** – the engine running this unit

inline virtual bool **isOn()**

Returns true iff the metronome fires.

virtual void **onBang**(EventCallback callback)

Registers event callback on metronome tick (“bang”) event.

inline virtual bool **isOff()**

Returns true iff the input is “off”.

inline virtual int **getInt()**

Returns value as integer (0 or 1).

inline virtual float **get()**

Returns value as float (either 0.0 or 1.0).

inline virtual bool **on()**

Sets output to “on” (ie. true, 1).

inline virtual bool **off()**

Sets output to “off” (ie. false, 0).

inline virtual float **put**(float value)

Pushes value into the unit.

Parameters

- value** – the value sent to the unit

Returns

the new value of the unit

inline virtual bool **putOn**(bool value)

Pushes value into the unit.

Parameters

- value** – the value sent to the unit

Returns

the new value of the unit

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

inline **operator bool**()

Operator that allows usage in conditional expressions.

inline float **seconds()** const

Returns engine time in seconds.

inline uint32_t **milliseconds()** const

Returns engine time in milliseconds.

inline uint64_t **microseconds()** const

Returns engine time in microseconds.

inline unsigned long **nSteps()** const

Returns number of engine steps.

inline float **sampleRate()** const

Returns engine sample rate.

inline float **samplePeriod()** const

Returns enginesample period.

virtual void **start()**

Starts/restarts the oscillator.

virtual void **period**(float period)

Sets the period (in seconds).

Parameters

period – the period of oscillation (in seconds)

inline virtual float **period()** const

Returns the period (in seconds).

inline Parameter **Period()**

Returns the period as a parameter.

virtual void **frequency**(float frequency)

Sets the frequency (in Hz).

Parameters

frequency – the frequency of oscillation (in Hz)

inline virtual float **frequency()** const

Returns the frequency (in Hz).

inline Parameter **Frequency()**

Returns the frequency as a parameter.

virtual void **bpm**(float bpm)

Sets the frequency in beats-per-minute.

Parameters

bpm – the frequency of oscillation (in BPM)

inline virtual float **bpm()** const

Returns the frequency (in BPM).

inline Parameter **Bpm()**

Returns the BPM as a parameter.

virtual void **phase**(float phase)

Sets the phase at % of period.

Parameters

phase – the phase (in % of period)

inline virtual float **phase()** const

Returns the phase (in % of period).

inline Parameter **Phase()**

Returns the phase as a parameter.

virtual void **phaseShift**(float phaseShift)

Sets the phase shift (ie.
the offset, in % of period).

Warning: This function is disabled if randomness() > 0.

Parameters

phaseShift – the phase shift (in % of period)

virtual float **phaseShift**() const

Returns the phase shift (ie.
the offset, in % of period).

Warning: This function always returns 0 when randomness() > 0.

inline Parameter **PhaseShift**()

Returns the phase shift as a parameter.

virtual void **jitter**(float jitter)

Sets the jittering level in [0, 1] (0: no jitter, 1: max jitter).

virtual float **jitter**() const

Returns the randomness level in [0, 1].

inline virtual void **noJitter**()

Disables jittering.

inline Parameter **Jitter**()

Returns the jitter as a parameter.

virtual float **jitteredPeriod**() const

Returns the period actually used for the current cycle.

When jitter() == 0, this is identical to period(). When jitter() > 0, this returns the stochastic (jittered) period currently in effect.

virtual float **jitteredFrequency**() const

Returns the frequency actually used for the current cycle.

When jitter() == 0, this is identical to frequency(). When jitter() > 0, this returns the stochastic (jittered) frequency currently in effect.

virtual float **timeToPhase**(float time) const

Utility function to convert time to phase.

Parameters

time – relative time in seconds

Returns

the equivalent phase

virtual void **setTime**(float time)

Forces current time (in seconds).

Warning: This function is disabled if randomness() > 0.

virtual void **addTime**(float time)

Adds time to current time (in seconds).

Warning: This function is disabled if randomness() > 0.

inline virtual bool **isRunning**() const

Returns true iff the wave is currently running.

inline virtual bool **isForward**() const

Returns true iff the wave is moving forward in time.

inline virtual void **setForward**(bool isForward)

Sets the direction of oscillation.

Parameters

isForward – true iff the wave is moving forward in time

inline virtual void **forward**()

Sets the direction of oscillation to move forward in time.

inline virtual void **reverse**()

Sets the direction of oscillation to move backward in time.

inline virtual void **toggleReverse**()

Toggles the direction of oscillation.

virtual void **stop**()

Interrupts and resets to zero.

virtual void **pause**()

Interrupts process.

virtual void **resume**()

Resumes process.

virtual void **togglePause**()

Toggles pause/unpause.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

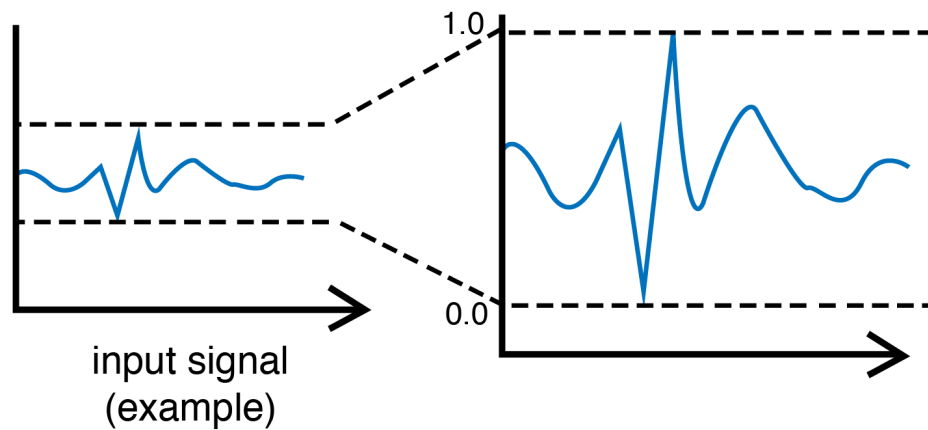
- *Alarm*
- *Chronometer*
- *Ramp*
- *Wave*

3.4 Filters

Filtering units for real-time signal processing.

3.4.1 MinMaxScaler

This filtering unit regularizes incoming signals by remapping them into a new interval of $[0, 1]$. It does so by keeping track of the minimum and the maximum values ever taken by the signal and rescales it such that the minimum value of the signal is mapped to 0 and the maximum value is mapped to 1.



In order to accommodate signals that might be changing through time, the user can specify a “decay time window” to control the rate of decay of the minimum and maximum boundaries. The principle is similar to the how the *Smoother* and the *Normalizer* make use of *exponential moving average*.

Caution: This filtering unit works well as long as there are no “outliers” in the signal (ie. extreme values) that appear in rare conditions. Such values will replace the minimum or maximum value and greatly restrict the spread of the filtered values.

There are three ways to prevent this:

1. Specifying a decay window using the `time(decayTime)` function.
2. Smoothing incoming values using the `smooth()` method or a *Smoother* unit before sending to the `MinMaxScaler`.
3. Using a regularization unit that is less prone to outliers such as the *Normalizer*.

Example

Reacts to high input values by activating an output LED. Scaler is used to automatically adapt to incoming sensor values.

```
#include <Plaquette.h>

// Analog sensor (eg. photocell or microphone).
AnalogIn sensor(A0);

// Min-max scaler.
MinMaxScaler scaler;

// Output indicator LED.
DigitalOut led(13);

void step() {
    // Rescale value.
    sensor >> scaler;

    // Light led on threshold of 80%.
    (scaler > 0.8) >> led;
}
```

Reference

class **MinMaxScaler** : public pq::MovingFilter

Regularizes signal into [0,1] by rescaling it using the min and max values.

Public Functions

MinMaxScaler(*Engine* &engine = *Engine::primary()*)

Default constructor.

Assigns infinite time window.

Parameters

engine – the engine running this unit

MinMaxScaler(float timeWindow, *Engine* &engine = *Engine::primary()*)

Constructor with time window.

Parameters

- **timeWindow** – the time window (in seconds)

- **engine** – the engine running this unit

inline float **minValue()** const

Returns the current min. value.

inline float **maxValue()** const

Returns the current max. value.

virtual void **reset()**

Resets the moving filter.

virtual void **reset**(float estimatedMeanValue)

Resets the filter with a prior estimate of the mean value.

virtual void **reset**(float estimatedMinValue, float estimatedMaxValue)

Resets the moving filter with a prior estimate of the min and max values.

virtual float **put**(float value)

Pushes value into the unit.

If **isRunning()** is false the filter will not be updated but will just return the filtered value.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

virtual float **filter**(float value)

Returns the filtered value (without calibrating).

virtual void **resumeCalibrating()**

Switches to calibration mode (default).

Calls to **put(value)** will return filtered value AND update the normalization statistics.

virtual void **pauseCalibrating()**

Switches to non-calibration mode: calls to **put(value)** will return filtered value without updating the normalization statistics.

virtual void **toggleCalibrating()**

Toggles calibration mode.

virtual bool **isCalibrating()** const

Returns true iff the moving filter is in calibration mode.

inline unsigned int **nSamples()** const

Returns the number of samples that have been processed thus far.

inline virtual bool **isPreInitialized()** const

Returns true if the moving filter has been initialized with a starting range at reset.

inline virtual float **get()**

Returns value in [0, 1].

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

inline float **seconds()** const

Returns engine time in seconds.

```

inline uint32_t milliseconds() const
    Returns engine time in milliseconds.

inline uint64_t microseconds() const
    Returns engine time in microseconds.

inline unsigned long nSteps() const
    Returns number of engine steps.

inline float sampleRate() const
    Returns engine sample rate.

inline float samplePeriod() const
    Returns enginesample period.

inline operator float()
    Object can be used directly to access its value.

inline explicit operator bool()
    Operator that allows usage in conditional expressions.

virtual void infiniteTimeWindow()
    Sets time window to infinite.

virtual void noTimeWindow()
    Sets time window to no time window.

virtual void timeWindow(float seconds)
    Changes the time window (expressed in seconds).

inline virtual float timeWindow() const
    Returns the time window (expressed in seconds).

inline virtual bool timeWindowIsInfinite() const
    Returns true if time window is infinite.

virtual void cutoff(float hz)
    Changes the time window cutoff frequency (expressed in Hz).

virtual float cutoff() const
    Returns the time window cutoff frequency (expressed in Hz).

```

Public Static Functions

```

static inline bool analogToDigital(float f)
    Converts analog (float) value to digital (bool) value.

static inline float digitalToAnalog(bool b)
    Converts digital (bool) value to analog (float) value.

```

See Also

- *Normalizer*
- *RobustScaler*
- *Smoother*

3.4.2 Normalizer

This filtering unit regularizes incoming signals by normalizing them around a target mean and standard deviation. It works by computing the normal distribution of the incoming data (mean and standard variation) and uses this information to re-normalize the data according to a different normal distribution (target mean and variance).

By default, the unit computes the mean and variance over all the data ever received. However, it can instead compute over a time window using an [exponential moving average](#).

Example

Uses a normalizer to analyze input sensor values and detect extreme values.

```
#include <Plaqueette.h>

// Analog sensor (eg. photocell or microphone).
AnalogIn sensor(A0);

// Normalizer with mean 0 and standard deviation 1.
Normalizer normalizer(0, 1);

// Output indicator LED.
DigitalOut led(13);

void step() {
    // Normalize value.
    sensor >> normalizer;

    // Light led if value differs from mean by more
    // than twice the standard deviation.
    (abs(normalizer) > 2.0) >> led;
}
```

Reference

class **Normalizer** : public pq::MovingFilter, public pq::MovingStats

Adaptive normalizer: normalizes values on-the-run using exponential moving averages over mean and standard deviation.

Public Functions

Normalizer(*Engine* &engine = *Engine::primary()*)

Default constructor.

Assigns infinite time window. Will renormalize data around a mean of 0.5 and a standard deviation of 0.15.

Parameters

engine – the engine running this unit

Normalizer(float timeWindow, *Engine* &engine = *Engine::primary()*)

Constructor with time window.

Will renormalize data around a mean of 0.5 and a standard deviation of 0.15.

Parameters

- **timeWindow** – the time window over which the normalization applies (in seconds)
- **engine** – the engine running this unit

Normalizer(float mean, float stdDev, *Engine* &engine = *Engine::primary()*)

Constructor with infinite time window.

Parameters

- **mean** – the target mean
- **stdDev** – the target standard deviation

Normalizer(float mean, float stdDev, float timeWindow, *Engine* &engine = *Engine::primary()*)

Constructor with time window.

Parameters

- **mean** – the target mean
- **stdDev** – the target standard deviation
- **timeWindow** – the time window over which the normalization applies (in seconds)

inline void **targetMean**(float mean)

Sets target mean of normalized values.

Parameters

mean – the target mean

inline float **targetMean**() const

Returns target mean.

inline void **targetStdDev**(float stdDev)

Sets target standard deviation of normalized values.

Parameters

stdDev – the target standard deviation

inline float **targetStdDev**() const

Returns target standard deviation.

virtual void **reset**()

Resets the statistics.

virtual void **reset**(float estimatedMeanValue)

Resets the filter with a prior estimate of the mean value.

virtual void **reset**(float estimatedMinValue, float estimatedMaxValue)

Resets the moving filter with a prior estimate of the min and max values.

virtual float **put**(float value)

Pushes value into the unit.

If `isRunning()` is false the filter will not be updated but will just return the filtered value.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

virtual float **filter**(float value)

Returns the filtered value (without calibrating).

virtual float **lowOutlierThreshold**(float nStdDev = 1.5f) const

Returns value above which value is considered to be a low outlier (below average).

Parameters

nStdDev – the number of standard deviations (typically between 1 and 3); low values = more sensitive

virtual float **highOutlierThreshold**(float nStdDev = 1.5f) const

Returns value above which value is considered to be a high outlier (above average).

Parameters

nStdDev – the number of standard deviations (typically between 1 and 3); low values = more sensitive

bool **isClamped**() const

Return true iff the normalized value is clamped within reasonable range.

void **clamp**(float nStdDev = NORMALIZER_DEFAULT_CLAMP_STDDEV)

Assign clamping value.

Values will then be clamped between reasonable range (`targetMean()` +/- `nStdDev * targetStdDev()`).

Parameters

nStdDev – the number of standard deviations (default: 3.333333333)

void **noClamp**()

Remove clamping.

virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

virtual void **resumeCalibrating**()

Switches to calibration mode (default).

Calls to `put(value)` will return filtered value AND update the normalization statistics.

virtual void **pauseCalibrating**()

Switches to non-calibration mode: calls to `put(value)` will return filtered value without updating the normalization statistics.

virtual void **toggleCalibrating()**
Toggles calibration mode.

virtual bool **isCalibrating()** const
Returns true iff the moving filter is in calibration mode.

inline unsigned int **nSamples()** const
Returns the number of samples that have been processed thus far.

inline virtual bool **isPreInitialized()** const
Returns true if the moving filter has been initialized with a starting range at reset.

inline virtual float **get()**
Returns value in [0, 1].

inline float **seconds()** const
Returns engine time in seconds.

inline uint32_t **milliSeconds()** const
Returns engine time in milliseconds.

inline uint64_t **microSeconds()** const
Returns engine time in microseconds.

inline unsigned long **nSteps()** const
Returns number of engine steps.

inline float **sampleRate()** const
Returns engine sample rate.

inline float **samplePeriod()** const
Returns enginesample period.

inline **operator float()**
Object can be used directly to access its value.

inline explicit **operator bool()**
Operator that allows usage in conditional expressions.

virtual void **infiniteTimeWindow()**
Sets time window to infinite.

virtual void **noTimeWindow()**
Sets time window to no time window.

virtual void **timeWindow(float seconds)**
Changes the time window (expressed in seconds).

inline virtual float **timeWindow()** const
Returns the time window (expressed in seconds).

inline virtual bool **timeWindowIsInfinite()** const
Returns true if time window is infinite.

virtual void **cutoff(float hz)**
Changes the time window cutoff frequency (expressed in Hz).

virtual float **cutoff**() const

Returns the time window cutoff frequency (expressed in Hz).

virtual float **update**(float value, float alpha)

Adds a value to the statistics (returns the mean).

inline virtual float **mean**() const

Returns an exponential moving average of the samples.

inline virtual float **meanSquared**() const

Return an exponential moving variance of the squared samples.

inline virtual float **var**() const

Returns an exponential moving variance of the samples.

virtual float **stdDev**() const

Returns the standard deviation of the samples.

virtual float **normalize**(float value) const

Returns the normalized value according $N(0, 1)$.

virtual float **normalize**(float value, float mean, float stdDev) const

Returns the normalized value according to the computed statistics (mean and variance).

virtual bool **isOutlier**(float value, float nStdDev = 1.5f) const

Returns true if the value is considered an outlier.

Parameters

- **value** – the raw value to be tested (non-normalized)
- **nStdDev** – the number of standard deviations (typically between 1 and 3); low values = more sensitive

Returns

true if value is nStdDev number of standard deviations above or below mean

virtual bool **isLowOutlier**(float value, float nStdDev = 1.5f) const

Returns true if the value is considered a low outlier (below average).

Parameters

- **value** – the raw value to be tested (non-normalized)
- **nStdDev** – the number of standard deviations (typically between 1 and 3); low values = more sensitive

Returns

true if value is nStdDev number of standard deviations below mean

virtual bool **isHighOutlier**(float value, float nStdDev = 1.5f) const

Returns true if the value is considered a high outlier (above average).

Parameters

- **value** – the raw value to be tested (non-normalized)
- **nStdDev** – the number of standard deviations (typically between 1 and 3); low values = more sensitive

Returns

true if value is nStdDev number of standard deviations above mean

inline virtual float **stddev**() const

Deprecated:

Returns the standard deviation of the samples.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- *MinMaxScaler*
- *RobustScaler*
- *Smoother*

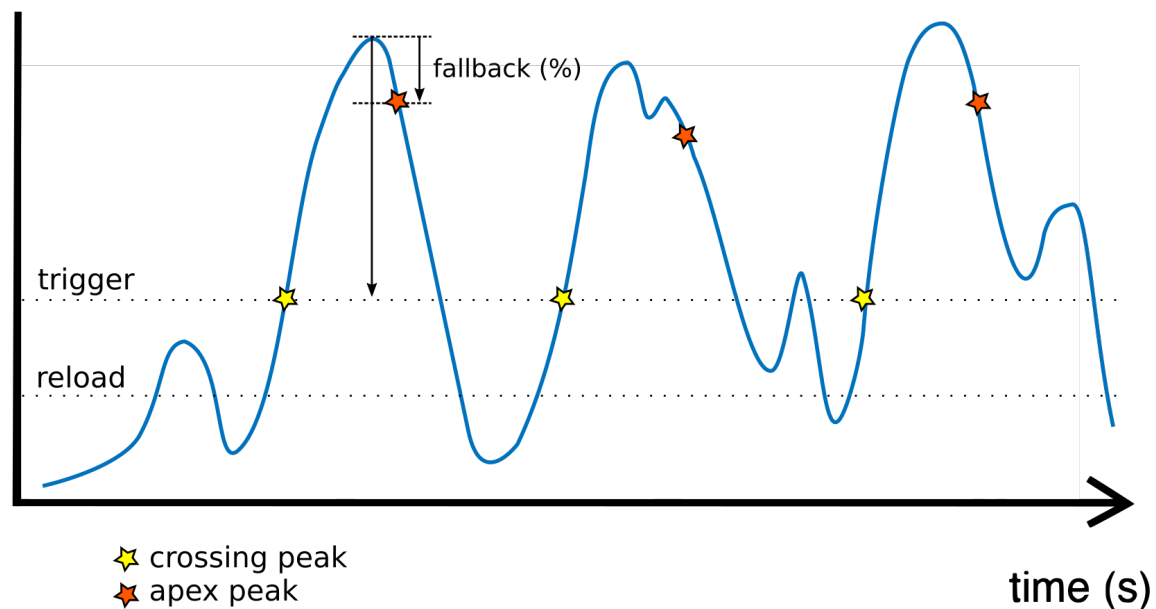
3.4.3 PeakDetector

This unit detects peaks (minima or maxima) in an incoming signal. Peaks are detected based on crossing a trigger threshold above (or below) which a peak is detected.

Two different ways are supported to do this:

- In **crossing** modes (PEAK_RISING and PEAK_FALLING) the peak is detected *as soon as the signal crosses* the `triggerThreshold`.
- In **apex** modes (PEAK_MAX and PEAK_MIN) the peak is detected after the signal crosses the `triggerThreshold`, reaches its apex, and then *falls back* by a certain proportion (%) between the threshold and the apex (controlled by the `fallbackTolerance` parameter).

In all cases, after a peak is detected, the detector will wait until the signal crosses back the `reloadThreshold` (which can be adjusted to control detection sensitivity) before it can be triggered again.



In summary, the four different modes available are:

- **PEAK_RISING** : peak detected as soon as `value >= triggerThreshold`, then wait until `value < reloadThreshold`
- **PEAK_FALLING** : peak detected as soon as `value <= triggerThreshold`, then wait until `value > reloadThreshold`
- **PEAK_MAX** : peak detected after `value >= triggerThreshold` and then *falls back* after peaking; then waits until `value < reloadThreshold`
- **PEAK_MIN** : peak detected after `value <= triggerThreshold` and then *falls back* after peaking; then waits until `value > reloadThreshold`

Important: Before sending a signal to a PeakDetector unit, it is recommended to normalize signals, preferably using the *Normalizer* unit. Furthermore, to avoid a noisy signal to generate false peaks, it is recommended to smooth the signal by calling the source unit's `smooth()` method or by using a *Smooother* unit.

Events

When	Trigger	Callback
Unit emits a pulse	unit itself	onBang(c)

Example

Uses a Normalizer and a PeakDetector to analyze input sensor values and detect peaks. Toggle and LED each time a peak is detected.

```
#include <Plaquette.h>

// Analog sensor (eg. photocell or microphone).
AnalogIn sensor(A0);

// Normalization unit to normalize values.
Normalizer normalizer;

// Peak detector. Threshold is set at 1.5 standard deviations above normal.
PeakDetector detector(normalizer.highOutlierThreshold(1.5)); // default mode = PEAK_MAX
// NOTE: You can change mode using optional 2nd parameter, example:
// PeakDetector detector(1.5, PEAK_FALLING));

// Digital LED output.
DigitalOut led;

void begin() {
    // Adjust reload threshold to smaller value than reloadThreshold.
    detector.reloadThreshold(normalizer.highOutlierThreshold(1.0));

    // Adjust fallback tolerance as % between apex and trigger threshold.
    detector.fallbackTolerance(0.2); // 0.2 = 20% (default: 10%)

    // Smooth signal to avoid false peaks due to noise.
    sensor.smooth();

    // Set a time window of 1 minute (60 seconds) on normalizer.
    // This will allow the normalier to slowly readjust itself
    // if the lighting conditions change.
    normalizer.timeWindow(60.0f);
};

void step() {
    // Signal is normalized and sent to peak detector.
    sensor >> normalizer >> detector;

    // Toggle LED when peak detector triggers.
    if (detector)
        led.toggle();
}
```

Reference

class **PeakDetector** : public pq::DigitalUnit

Emits a “bang” signal when another signal peaks.

Public Functions

PeakDetector(float triggerThreshold, *Engine* &engine = *Engine::primary()*)

Constructor with default mode (PEAK_MAX).

Parameters

- **triggerThreshold** – value that triggers peak detection
- **engine** – the engine running this unit

PeakDetector(float triggerThreshold, uint8_t mode, *Engine* &engine = *Engine::primary()*)

Constructor.

Possible modes are:

- **PEAK_RISING** : peak detected when value becomes $\geq$ triggerThreshold, then wait until it becomes $<$ reloadThreshold (*)
- **PEAK_FALLING** : peak detected when value becomes $\leq$ triggerThreshold, then wait until it becomes $>$ reloadThreshold (*)
- **PEAK_MAX** : peak detected after value becomes $\geq$ triggerThreshold and then falls back after peaking; then waits until it becomes $<$ reloadThreshold (*)
- **PEAK_MIN** : peak detected after value becomes $\leq$ triggerThreshold and then rises back after peaking; then waits until it becomes $>$ reloadThreshold (*)

Parameters

- **triggerThreshold** – value that triggers peak detection
- **mode** – peak detection mode
- **engine** – the engine running this unit

void **triggerThreshold**(float triggerThreshold)

Sets triggerThreshold.

inline float **triggerThreshold**() const

Returns triggerThreshold.

void **reloadThreshold**(float reloadThreshold)

Sets minimal threshold that “resets” peak detection in crossing (rising/falling) and peak (min/max) modes.

inline float **reloadThreshold**() const

Returns minimal value “drop” for reset.

void **fallbackTolerance**(float fallbackTolerance)

Sets minimal relative “drop” after peak to trigger detection in peak (min/max) modes, expressed as proportion (%) of peak minus triggerThreshold.

inline float **fallbackTolerance**() const

Returns minimal relative “drop” after peak to trigger detection in peak modes.

bool **modeInverted**() const

Returns true if mode is PEAK_FALLING or PEAK_MIN.

bool **modeCrossing**() const

Returns true if mode is PEAK_RISING or PEAK_FALLING.

bool **modeApex**() const

Returns true if mode is PEAK_MAX or PEAK_MIN.

void **mode**(uint8_t mode)

Sets mode.

inline uint8_t **mode**() const

Returns mode.

virtual float **put**(float value)

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline virtual bool **isOn**()

Returns true iff the triggerThreshold is crossed.

virtual void **onBang**(EventCallback callback)

Registers event callback on peak detection.

inline virtual bool **isOff**()

Returns true iff the input is “off”.

inline virtual int **getInt**()

Returns value as integer (0 or 1).

inline virtual float **get**()

Returns value as float (either 0.0 or 1.0).

inline virtual bool **on**()

Sets output to “on” (ie. true, 1).

inline virtual bool **off**()

Sets output to “off” (ie. false, 0).

inline virtual bool **putOn**(bool value)

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

inline **operator bool**()

Operator that allows usage in conditional expressions.

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliSeconds**() const

Returns engine time in milliseconds.

inline uint64_t **microSeconds**() const

Returns engine time in microseconds.

inline unsigned long **nSteps**() const

Returns number of engine steps.

inline float **sampleRate**() const

Returns engine sample rate.

inline float **samplePeriod**() const

Returns enginesample period.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- *Normalizer*
- *MinMaxScaler*
- *Smoother*

3.4.4 RobustScaler

This filtering unit regularizes incoming signals by remapping them into a new interval of [0, 1]. It focuses on what the signal does *most of the time* instead of reacting to rare extreme values. It does this by keeping track of two boundaries:

- a **lower boundary** (low quantile), below which the signal almost never goes, and
- an **upper boundary** (high quantile), above which the signal almost never goes.

You can think of these as “typical low” and “typical high” values. Once these boundaries are known, the signal is rescaled so that the lower boundary becomes 0 and the upper boundary becomes 1.

Because it pays attention to the typical behaviour of the signal, while ignoring rare spikes or sudden jumps, the **RobustScaler** is a good choice for noisy sensors, biological signals, or any data that behaves unpredictably.

Adjusting Robustness

The **span** controls how much of your data is considered “typical” and placed between the two boundaries.

- A span of 0.8 (80%) means that the scaler tries to place about 80% of the most common values between its lower and upper boundaries.
- A span of 0.95 (95%) means it tries to place about 95% of values there, and so on.

You can set it in code like this:

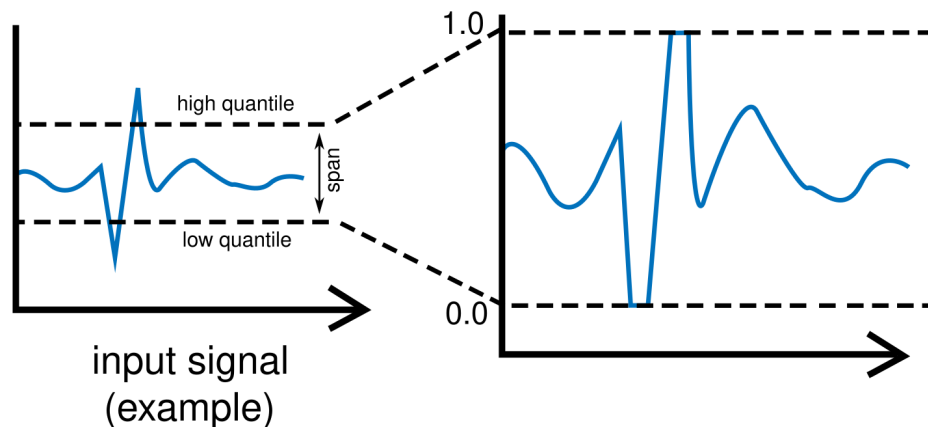
```
RobustScaler scaler;
scaler.span(0.8); // Keep about 80% of values between 0 and 1.
```

Intuition

A good way to understand the span is to imagine a **corridor** in which your signal is allowed to move. The span controls how wide that corridor is.

- **Higher span (e.g. 0.9 or 0.95)** This corresponds to a **wide corridor**: most of the signal’s behaviour fits comfortably inside it. The scaler becomes more resistant to spikes or sudden jumps, but extreme values will simply hit the corridor walls and stay clamped near 0 or 1.
- **Lower span (e.g. 0.6 or 0.7)** This corresponds to a **narrow corridor**: the scaler pays attention to a smaller core of typical values. It reacts more strongly to changes, but even moderate deviations may be treated as if they were already “far out.”

Choosing a span is therefore a balance between *robustness* (wide corridor, large span) and *sensitivity* (narrow corridor, small span).



Example

Filters input values from a noisy sensor. The scaler adapts to the usual behaviour of the signal but ignores occasional spikes. An LED turns on when the signal goes beyond the upper “typical” range.

```
#include <Plaqueette.h>

// Example: noisy analog sensor.
AnalogIn sensor(A0);

// Creates a robust scaler.
RobustScaler scaler;

// Simple square wave.
Wave wave;

// LED output.
DigitalOut led(13);

void begin() {
    // Sets span to 90%.
    scaler.span(0.9);
}

void step() {
    // Rescale value based on typical low/high ranges.
    sensor >> scaler;

    // Adjust period based on rescaled value.
    wave.period(scaler.mapTo(0.5, 2));

    // Blink LED.
    wave >> led;
}
```

Note: For readers familiar with statistics: the `RobustScaler` functions by estimating two quantiles of the incoming signal. If the span is s , the scaler computes:

- a lower quantile at $(1 - s) / 2$, and
- an upper quantile at $(1 + s) / 2$.

For example:

- `span(0.8)` → lower = 10th percentile, upper = 90th percentile
- `span(0.9)` → lower = 5th percentile, upper = 95th percentile

The incoming value is then linearly mapped between these two quantiles.

Reference

class **RobustScaler** : public pq::MovingFilter

Regularizes signal into [0,1] using adaptive quantile tracking (robust to outliers).

Public Functions

RobustScaler(*Engine* &engine = *Engine::primary*())

Default constructor.

RobustScaler(float timeWindow, *Engine* &engine = *Engine::primary*())

Constructor with custom quantile levels and time window.

Parameters

timeWindow – The adaptation window in seconds.

RobustScaler(float timeWindow, float span, *Engine* &engine = *Engine::primary*())

Constructor with custom quantile levels and time window.

Parameters

- **timeWindow** – The adaptation window in seconds.
- **span** – Corresponds to percentage coverage of value in [0, 1].

virtual void **span**(float span)

Sets the span (in [0, 1]) of the quantile to track.

virtual float **span**() const

Returns the current span.

virtual void **lowQuantileLevel**(float level)

Sets the low quantile level (in [0, 0.5]). Low quantile will automatically be set to 1 - low.

inline virtual float **lowQuantileLevel**() const

Returns the current low quantile level.

virtual void **highQuantileLevel**(float level)

Sets the high quantile level (in [0.5, 1]). Low quantile will automatically be set to 1 - high.

inline virtual float **highQuantileLevel**() const

Returns the current high quantile level.

inline virtual float **lowQuantile**() const

Returns the current low quantile.

inline virtual float **highQuantile**() const

Returns the current high quantile.

inline virtual float **stdDev**() const

Returns the current standard deviation.

virtual void **reset**()

Resets the filter.

virtual void **reset**(float estimatedMeanValue)

Resets the filter with a prior estimate of the mean value.

virtual void **reset**(float estimatedMinValue, float estimatedMaxValue)

Resets the moving filter with a prior estimate of the min and max values.

inline virtual float **get**()

Returns value of scaler.

virtual float **put**(float value)

Pushes a new value and returns the scaled output.

virtual float **filter**(float value)

Returns the filtered value (without calibrating).

virtual void **resumeCalibrating**()

Switches to calibration mode (default).

Calls to put(value) will return filtered value AND update the normalization statistics.

virtual void **pauseCalibrating**()

Switches to non-calibration mode: calls to put(value) will return filtered value without updating the normalization statistics.

virtual void **toggleCalibrating**()

Toggles calibration mode.

virtual bool **isCalibrating**() const

Returns true iff the moving filter is in calibration mode.

inline unsigned int **nSamples**() const

Returns the number of samples that have been processed thus far.

inline virtual bool **isPreInitialized**() const

Returns true if the moving filter has been initialized with a starting range at reset.

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliSeconds**() const

Returns engine time in milliseconds.

inline uint64_t **microSeconds**() const

Returns engine time in microseconds.

inline unsigned long **nSteps**() const

Returns number of engine steps.

inline float **sampleRate**() const

Returns engine sample rate.

inline float **samplePeriod**() const

Returns enginesample period.

inline **operator float**()

Object can be used directly to access its value.

inline explicit **operator bool**()

Operator that allows usage in conditional expressions.

virtual void **infiniteTimeWindow**()

Sets time window to infinite.

virtual void **noTimeWindow**()

Sets time window to no time window.

virtual void **timeWindow**(float seconds)

Changes the time window (expressed in seconds).

inline virtual float **timeWindow**() const

Returns the time window (expressed in seconds).

inline virtual bool **timeWindowIsInfinite**() const

Returns true if time window is infinite.

virtual void **cutoff**(float hz)

Changes the time window cutoff frequency (expressed in Hz).

virtual float **cutoff**() const

Returns the time window cutoff frequency (expressed in Hz).

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

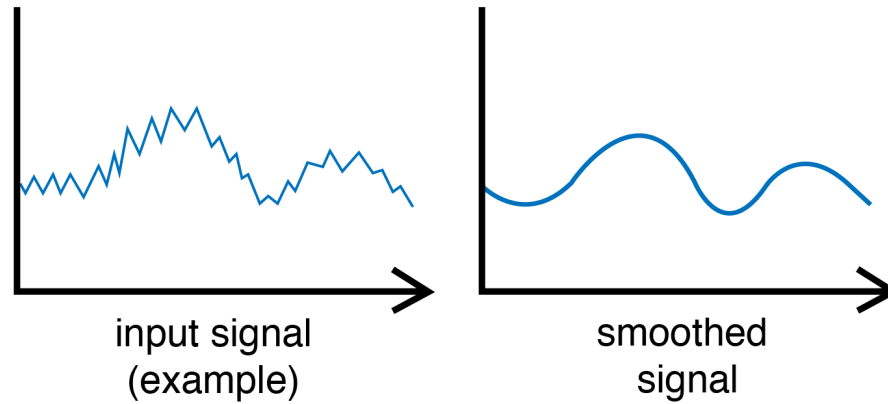
Converts digital (bool) value to analog (float) value.

See Also

- *MinMaxScaler*
- *Normalizer*
- *Smoother*

3.4.5 Smoother

Smooths the incoming signal by removing fast variations and noise (high frequencies).



Example

Smooth a sensor over time.

```
#include <Plaqueette.h>

AnalogIn sensor(A0);

// Smooths over time window of 10 seconds.
Smoother smoother(10.0);

Plotter plotter(115200); // Initialize serial plotter at 115200 bps

void step() {
    // Smooth value and send it to serial output.
    sensor >> smoother >> plotter;
}
```

Note: The filter uses an [exponential moving average](#) which corresponds to a form of [low-pass filter](#).

Reference

class **Smoother** : public pq::MovingFilter
 Simple moving average transform filter.

Public Functions

Smoother(*Engine* &engine = *Engine::primary()*)
 Constructor with default smoothing.

Parameters

engine – the engine running this unit

Smoother(float timeWindow, *Engine* &engine = *Engine::primary()*)
 Constructor with smoothing window.

Parameters

- **timeWindow** – the time window over which the smoothing applies (in seconds)
- **engine** – the engine running this unit

virtual void **reset**()
 Resets the filter.

virtual void **reset**(float estimatedMeanValue)
 Resets the filter with a prior estimate of the mean value.

virtual void **reset**(float estimatedMinValue, float estimatedMaxValue)
 Resets the moving filter with a prior estimate of the min and max values.

virtual float **put**(float value)
 Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

virtual float **filter**(float value)
 Returns the filtered value (without calibrating).

virtual void **resumeCalibrating**()
 Switches to calibration mode (default).
 Calls to put(value) will return filtered value AND update the normalization statistics.

virtual void **pauseCalibrating**()
 Switches to non-calibration mode: calls to put(value) will return filtered value without updating the normalization statistics.

virtual void **toggleCalibrating**()
 Toggles calibration mode.

virtual bool **isCalibrating**() const
 Returns true iff the moving filter is in calibration mode.

`inline unsigned int nSamples() const`
Returns the number of samples that have been processed thus far.

`inline virtual bool isPreInitialized() const`
Returns true if the moving filter has been initialized with a starting range at reset.

`inline virtual float get()`
Returns value in [0, 1].

`inline virtual float mapTo(float toLow, float toHigh)`
Maps value to new range.

`inline float seconds() const`
Returns engine time in seconds.

`inline uint32_t milliSeconds() const`
Returns engine time in milliseconds.

`inline uint64_t microSeconds() const`
Returns engine time in microseconds.

`inline unsigned long nSteps() const`
Returns number of engine steps.

`inline float sampleRate() const`
Returns engine sample rate.

`inline float samplePeriod() const`
Returns enginesample period.

`inline operator float()`
Object can be used directly to access its value.

`inline explicit operator bool()`
Operator that allows usage in conditional expressions.

`virtual void infiniteTimeWindow()`
Sets time window to infinite.

`virtual void noTimeWindow()`
Sets time window to no time window.

`virtual void timeWindow(float seconds)`
Changes the time window (expressed in seconds).

`inline virtual float timeWindow() const`
Returns the time window (expressed in seconds).

`inline virtual bool timeWindowIsInfinite() const`
Returns true if time window is infinite.

`virtual void cutoff(float hz)`
Changes the time window cutoff frequency (expressed in Hz).

`virtual float cutoff() const`
Returns the time window cutoff frequency (expressed in Hz).

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- *AnalogIn*
- *DigitalIn*

3.5 Communication

Units for communication between embedded device and computer (debugging, logging, visualization).

3.5.1 Monitor

An output unit that writes textual information to a print device, typically the serial line. The Monitor is intended for **logging, debugging, and human-readable output**, rather than structured data plotting like the *Plotter*.

To create a monitor on the default serial port with a specific baudrate:

```
Monitor monitor(baudrate);
```

Plaquette automatically uses the first Monitor you create as the default device where information is printed. This means that instead of writing:

```
monitor.println("Hello world!");
```

You can just write:

```
println("Hello world!");
```

Tip: On Arduino, you can read the text printed by the Monitor unit using the *Serial Monitor* by selecting **Tools > Serial Monitor**. Make sure the baudrate matches the value used to create the Monitor.

Example

Printing messages and values to the serial monitor.

```
#include <Plaquette.h>

Monitor monitor(115200);

void step() {
  println("Stepping...");
```

(continues on next page)

(continued from previous page)

```
print("Seconds: ");
seconds >> monitor;
println(); // new line
}
```

Notes and Warnings

- The Monitor is **not** intended for structured data export or plotting; use *Plotter* for that purpose.
- Blocking read operations (such as `readStringUntil()`) are intentionally not exposed to avoid unintended blocking behavior.
- Precision for numeric output can be configured on the Monitor by calling its function `precision()` and applies to floating-point values printed through it.

Reference

class **Monitor** : public pq::Unit, public Print

Write-only monitor unit for textual output.

Monitor is a Plaquette Unit and a Print proxy. It is intended for human-readable output (debugging, status messages) and deliberately does not expose Stream read functionality to avoid blocking behavior.

The backend can be any Print-compatible object (Serial, file, etc.). When bound to a serial device, Monitor can auto-start it at a given baud rate.

Public Functions

explicit **Monitor**(unsigned long baudRate, *Engine* &engine = *Engine::primary()*)

Construct a Monitor using the default Serial device.

Monitor(SerialType &device, unsigned long baudRate, *Engine* &engine = *Engine::primary()*)

Construct a Monitor using a specific serial device.

explicit **Monitor**(Print &device, *Engine* &engine = *Engine::primary()*)

Construct a Monitor using a generic Print backend.

No auto-start is performed.

inline Print &**device**() const

Get the current Print backend.

void **precision**(uint8_t digits)

Set the number of digits to print after the decimal point.

inline uint8_t **precision**() const

Get the number of digits to print after the decimal point by default.

virtual float **put**(float value) override

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline virtual float **get()** override

Returns value (last value that was put()).

virtual size_t **write**(uint8_t b) override

Core Print override.

All print()/println() calls funnel through this method.

inline float **seconds()** const

Returns engine time in seconds.

inline uint32_t **milliSeconds()** const

Returns engine time in milliseconds.

inline uint64_t **microSeconds()** const

Returns engine time in microseconds.

inline unsigned long **nSteps()** const

Returns number of engine steps.

inline float **sampleRate()** const

Returns engine sample rate.

inline float **samplePeriod()** const

Returns enginesample period.

inline **operator float()**

Object can be used directly to access its value.

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

This function guarantees that the value is within [toLow, toHigh]. If the unit's values are unbounded, returns get() constrained to [toLow, toHigh].

inline explicit **operator bool()**

Operator that allows usage in conditional expressions.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- [Plotter](#)
- [Arduino Serial Monitor](#)
- [Arduino serial](#)

3.5.2 Plotter

An output unit that streams values to a text-based output using a configurable format. The Plotter is designed for quick signal inspection in tools such as the Arduino Serial Plotter, while also supporting structured formats (e.g., CSV, JSON) for logging and external processing.

Values are sent in clear text and arranged in **rows**. Each call to `put()` (usually via the `>>` operator) appends one value to the current row; the row is then finalized automatically at the end of the Plaquette engine step (or when the Plotter decides to close the row).

The Plotter can optionally take a comma-separated list of **labels** (`const char*`) such as `"wave,signal"`. When labels are provided, some formats will include a header (e.g., CSV) or will use key/value rendering (e.g., JSON objects, `"label":value`).

Example

Streaming multiple values per row, with labels.

Tip: On Arduino, you can visualize the data using the [Serial Plotter](#) by selecting **Tools > Serial Plotter**.

```
#include <Plaquette.h>
//declare the baud rate to suit your application.
//OPTIONAL: After declaring the baud rate,
//you can create labels for incoming signals with a single comma-separated string.
Plotter plotter(115200, "wave,signal");

Wave wave(SINE);
Signal signal;

void step() {
    wave >> plotter;
    signal >> plotter;
}
```

You can add labels to the output by specifying them at the creation of the `Plotter` unit:

```
Plotter plotter(115200, "wave,signal");
```

You can also output in Comma Separated Values format by using presets (`PLOTTER_CSV`, `PLOTTER_JSON`):

```
void begin() {
    plotter.format(PLOTTER_CSV);
}
```

Depending on the chosen format and whether labels are provided, labels may be rendered as a header (CSV), as keys (JSON objects), or as `label:value` pairs (default space-delimited mode with labels).

Reference

class **Plotter** : public pq::Unit

Public Functions

inline const char ***labels**() const

Returns labels.

void **format**(PlotterFormatPreset preset)

Sets format based on preset.

void **format**(PlotterFormat format)

Sets format based on custom format.

inline PlotterFormat **format**()

Returns current format.

inline void **precision**(uint8_t digits)

Sets decimal precision of values.

Parameters

digits – the number of digits after the point

inline uint8_t **precision**() const

Returns the decimal precision of values.

virtual float **put**(float value) override

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline virtual float **get**() override

Returns last value.

void **beginPlot**()

Begins new plot.

void **endPlot**()

Ends current plot.

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliSeconds**() const

Returns engine time in milliseconds.

inline uint64_t **microSeconds**() const

Returns engine time in microseconds.

inline unsigned long **nSteps**() const

Returns number of engine steps.

inline float **sampleRate()** const

Returns engine sample rate.

inline float **samplePeriod()** const

Returns enginesample period.

inline **operator float()**

Object can be used directly to access its value.

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

This function guarantees that the value is within [toLow, toHigh]. If the unit's values are unbounded, returns `get()` constrained to [toLow, toHigh].

inline explicit **operator bool()**

Operator that allows usage in conditional expressions.

Public Static Functions

static PlotterFormat **formatFromPreset**(PlotterFormatPreset preset, const char *labelsSchema = nullptr)

Returns a PlotterFormat based on presets and labels.

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- [*Monitor*](#)
- [Arduino Serial Plotter](#)
- [Arduino serial](#)

3.6 Fields

Field units represent a spatial function over the normalized range [0, 1]. Unlike typical arrays indexed by integers, a field is sampled using fractional positions and returns a value based on internal logic or data. Fields are useful for plotting, shaping, transforming, or visualizing signals across space.

3.6.1 PivotField

This field unit generates a spatial response curve based on a **pivot point** around which the field transitions happens. It can output sharp steps, smooth ramps, bumps or notches, across the normalized range [0, 1], making it ideal for creating animations on arrays of actuators such as LEDs or motors (eg. VU-meters, fades, or envelope-like visualizations).

Modes

The behavior of the field is defined by its mode:

- PIVOT_FALLING (default): Values are 1.0 from the left up to the pivot, then fall to 0.0.
- PIVOT_RISING : Values are 0.0 on the left and rise to 1.0 after the pivot.
- PIVOT_BUMP: Peak at the pivot: 1.0 around the pivot, tapering down on both sides.
- PIVOT_NOTCH: Inverse of PIVOT_BUMP: 0.0 around the pivot, 1.0 elsewhere.

Adjustments

The curve can be finely configured using many different parameters:

- **rampWidth**: Ramp width as a fraction of total range (e.g., 0.2 for 20% width). Default: 0 (no ramp).
- **center**: Point from which the curve starts (in [0, 1]). Examples:
 - 0.0 (default) Start from left side.
 - 0.5 Start from middle going in both directions.
 - 0.75 Start from $\frac{3}{4}$ (75%) going in both directions.
 - 1.0 Start from right side.
- **easing**: Optional *easing function* applied to the ramp. Default: easeNone.
- **rampShift**: Determines the horizontal shift of the ramp. Examples:
 - 0.0 Ramp ends at the pivot point.
 - 0.5 (default) Ramp is right in the middle of the pivot point.
 - 1.0 Ramp starts at the pivot point.
- **bumpWidth**: When choosing PIVOT_BUMP or PIVOT_NOTCH, determines the width of the peak. Default: 0.25 (25%).

Example

This example uses a single PWM analog LED and two potentiometers to show the behavior of a PivotField. The first potentiometer is used to control the pivot point, then the second potentiometer is used to reveal the values of the *PivotField* from 0 to 1.

```
#include <Plaquette.h>

// Potentiometer #1 to control the pivot point.
AnalogIn pot(A0);

// Potentiometer #2 for exploring the field.
AnalogIn slider(A1);

// The PWM LED.
AnalogOut led(9);

// The pivot field.
PivotField field;
```

(continues on next page)

(continued from previous page)

```

void begin() {
    // Parameters. Change at will to explore different configurations.
    field.rampWidth(0.2);
    field.easing(easeInSine);
    field.mode(PIVOT_FALLING); // default
    field.center(0); // default
    field.bumpWidth(0.25); // default
}

void step() {
    // Send potentiometer value to the pivot field.
    pot >> field;

    // Send field value at position of slider to LED.
    field.at(slider) >> led;
}

```

Reference

class **PivotField** : public pq::AbstractField

Public Functions

PivotField(*Engine* &engine = *Engine::primary*())

Constructor.

inline virtual **~PivotField**()

virtual float **at**(float proportion) override

Returns value at given proportion in [0, 1].

Parameters

proportion – the proportion of the field to read

Returns

the value

inline virtual float **get**() override

Returns value.

inline virtual float **put**(float value) override

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline void **mode**(PivotFieldMode mode)

Sets mode to use.

Parameters**mode** – the mode to setinline PivotFieldMode **mode**() const

Returns mode.

inline void **easing**(easing_function easing)

Sets easing function to apply to ramp.

Parameters**easing** – the easing functioninline void **noEasing**()

Remove easing function (linear/no easing).

void **rampWidth**(float rampWidth)

Sets ramp width as % of field range.

Parameters**rampWidth** – the ramp width in [0, 1]inline void **noRampWidth**()

Removes ramp width.

inline float **rampWidth**() const

Returns ramp width.

void **rampShift**(float rampShift)

Sets ramp shift in [0, 1] (default: 0.5 = center).

Parameters**rampShift** – the ramp shift in [0, 1]inline float **rampShift**() const

Returns ramp shift.

void **bumpWidth**(float bumpWidth)

Sets bump width as % of field range.

Only applies to PIVOT_BUMP and PIVOT_NOTCH modes.

Parameters**bumpWidth** – the bump width in [0, 1]inline float **bumpWidth**() const

Returns bump width.

inline void **center**(float center)

Sets center of the ramp in [0, 1].

Parameters**center** – the center in [0, 1]inline float **center**() const

Returns center of the ramp.

template<typename T>

inline void **populate**(T *array, size_t size, bool wrap = false)

Fills an array with values from this field.

Parameters

- **array** – the array to read into
- **size** – the size of the array
- **wrap** – if true, the array is considered to be a circular buffer that wraps around

virtual void **clearEvents()**

inline float **seconds()** const

Returns engine time in seconds.

inline uint32_t **milliSeconds()** const

Returns engine time in milliseconds.

inline uint64_t **microSeconds()** const

Returns engine time in microseconds.

inline unsigned long **nSteps()** const

Returns number of engine steps.

inline float **sampleRate()** const

Returns engine sample rate.

inline float **samplePeriod()** const

Returns enginesample period.

inline **operator float()**

Object can be used directly to access its value.

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

This function guarantees that the value is within [toLow, toHigh]. If the unit's values are unbounded, returns get() constrained to [toLow, toHigh].

inline explicit **operator bool()**

Operator that allows usage in conditional expressions.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- *[] (arrays)*
- *TimeSliceField*
- *Easings*
- *Ramp*

3.6.2 TimeSliceField

This field unit stores a **series of values collected over time**, which can then be sampled spatially like an array across the normalized range [0, 1]. It is useful for plotting time-varying signals, such as mapping audio or sensor input onto an LED strip or a motor array.

The unit gathers samples over a defined time period, storing them in its internal memory. When it is ready to be read, it fires an `updated` event which can be used to update outputs only when necessary.

The size `SIZE` of the internal memory (ie., the number of values that are stored internally) and the overall time period (in seconds) over which the field applies is specified at creation time:

```
TimeSliceField<SIZE> field(period);
```

The unit supports two modes:

1. In the **block mode** (default) the buffer fills up and fires the `updated` event when it is full; then it is emptied and starts filling up again.
2. In the **rolling mode** the buffer fills up, then...
 - ... when it is full it fires `updated`;
 - ... when enough time has passed that a new value needs to be replaced, the older value is removed and the buffer is shifted by one value and it fires `updated` to signal a change.

Events

When	Trigger	Callback
New data is available	<code>updated()</code>	<code>onUpdate(c)</code>

Example

Here is a simple example that uses a *TimeSliceField* to collect data from a sine wave and render it to an LED strip.

```
#include <Plaquette.h>

// The number of LEDs.
const int N_LEDS = 8;

// An array of LEDs.
DigitalOut leds[] = { 2, 3, 4, 5, 6, 7, 8, 9 }; // shorthand for DigitalOut leds[] = {
↳ DigitalOut(2), DigitalOut(3), DigitalOut(4), ... };

// The sine wave with a period of one second.
Wave wave(SINE, 1.0f);

// The time slice field (over 2 seconds period).
TimeSliceField<N_LEDS> timeSlice(2.0f);

void begin() {
  // Set field to rolling mode.
  timeSlice.rolling();
}
```

(continues on next page)

(continued from previous page)

```
void step() {  
    // Update the field.  
    wave >> timeSlice;  
  
    // Update the LEDs.  
    if (timeSlice.updated()) {  
        for (int i=0; i<N_LEDS; i++) {  
            float proportion = mapTo01(i, 0, N_LEDS-1); // maps i to [0, 1]  
            timeSlice.at(proportion) >> leds[i]; // send to LED  
        }  
    }  
}
```

Reference

template<size_t COUNT>

class **TimeSliceField** : public pq::AbstractField

TimeSliceField generic class.

Template Parameters

COUNT – the size of the buffer

Public Functions

inline **TimeSliceField**(float period, *Engine* &engine = *Engine::primary*())

Constructor.

Parameters

period – the period in seconds

inline virtual ~**TimeSliceField**()

inline virtual float **at**(float proportion) override

Returns value at given proportion in [0, 1].

Parameters

proportion – the proportion of the field to read

Returns

the value

inline virtual float **get**() override

Returns value.

inline virtual float **put**(float value) override

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline void **period**(float period)

Sets period over which the time slice occurs.

Parameters

period – the new period (in seconds)

inline float **period**() const

Returns period.

inline float **atIndex**(size_t index)

Returns value at given index.

inline size_t **count**() const

Returns count.

inline bool **updated**()

Returns true if the field has been updated and is ready to be used.

inline bool **isFull**()

Returns true if the field is full.

inline void **reset**()

Resets the field.

inline void **setRolling**(bool rolling)

Sets rolling mode.

Parameters

rolling – the rolling mode

inline void **rolling**()

Activates rolling mode.

inline void **noRolling**()

Deactivates rolling mode.

inline bool **isRolling**() const

Returns true if rolling mode is active.

inline virtual void **onUpdate**(EventCallback callback)

Registers event callback on update event.

template<typename T>

inline void **populate**(T *array, size_t size, bool wrap = false)

Fills an array with values from this field.

Parameters

- **array** – the array to read into
- **size** – the size of the array
- **wrap** – if true, the array is considered to be a circular buffer that wraps around

virtual void **clearEvents**()

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliseconds()** const

Returns engine time in milliseconds.

inline uint64_t **microseconds()** const

Returns engine time in microseconds.

inline unsigned long **nSteps()** const

Returns number of engine steps.

inline float **sampleRate()** const

Returns engine sample rate.

inline float **samplePeriod()** const

Returns enginesample period.

inline **operator float()**

Object can be used directly to access its value.

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

This function guarantees that the value is within [toLow, toHigh]. If the unit's values are unbounded, returns get() constrained to [toLow, toHigh].

inline explicit **operator bool()**

Operator that allows usage in conditional expressions.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- [\[\] \(arrays\)](#)
- [PivotField](#)

3.7 Functions

Standalone utility functions.

3.7.1 mapFloat()

Re-maps a number from one range to another. That is, a value of `fromLow` would get mapped to `toLow`, a value of `fromHigh` to `toHigh`, and values in-between to values in-between, proportionally.

```
float y = mapFloat(x, 10.0, 50.0, 100.0, 0.0);
```

The function also handles negative numbers well, so that this example

```
float y = mapFloat(x, 10.0, 50.0, 100.0, -100.0);
```

is also valid and works well.

By default, does *not* constrain output to stay within the `[fromHigh, toHigh]` range, because out-of-range values are sometimes intended and useful. In order to constrain the return value within range, you can use one of the alternative modes: * the `CONSTRAIN` mode to simply keep the value within range by restricting extreme values as in `constrain()` <<https://www.arduino.cc/reference/en/language/functions/math/constrain/>> * the `WRAP` mode to wrap the values around as in `wrap()`

```
mapFloat(x, 10.0, 50.0, 100.0, -100.0, CONSTRAIN);
mapFloat(x, 10.0, 50.0, 100.0, -100.0, WRAP);
```

Note: The “lower bounds” (`fromLow` and `toLow`) of either range may be larger or smaller than the “upper bounds” (`fromHigh` and `toHigh`) so the `mapFloat()` function may be used to reverse a range of numbers.

Important: Unlike the Arduino `map()` function, `mapFloat()` uses floating-point math and *will* generate fractions.

Example

```
#include <Plaquette.h>

Wave oscillator(1.0);

DigitalOut led(13);

void step() {
    // Change frequency between 2Hz and 15Hz over a 30 seconds period, then the frequency
    // will stay at 15Hz.
    float freq = mapFloat(seconds(), 0.0, 30.0, 2.0, 15.0, CONSTRAIN); // try removing
    // CONSTRAIN and see what happens
    oscillator.frequency(freq);

    // Send to LED.
    oscillator >> led;
}
```

Reference

float pq: **mapFloat**(double value, double fromLow, double fromHigh, double toLow, double toHigh, MapMode mode = UNCONSTRAIN)

Re-maps a number from one range to another.

Parameters

- **value** – the number to map
- **fromLow** – the lower bound of the value's current range
- **fromHigh** – the upper bound of the value's current range
- **toLow** – the lower bound of the value's target range
- **toHigh** – the upper bound of the value's target range
- **mode** – set to **CONSTRAIN** to constrain the return value between toLow and toHigh or **WRAP** for the value to wrap around

Returns

the mapped value

See Also

- [*mapFrom01\(\)*](#)
- [*mapTo01\(\)*](#)

3.7.2 mapFrom01()

Re-maps a number in the range [0, 1] to another range. That is, a value of 0 would get mapped to toLow, a value of 1 to toHigh, values in-between to values in-between, etc.

```
mapFrom01(x, toLow, toHigh)
```

is equivalent to:

```
mapFloat(x, 0, 1, toLow, toHigh)
```

By default, does *not* constrain output to stay within the [fromHigh, toHigh] range, because out-of-range values are sometimes intended and useful. In order to constrain the return value within range, use the **CONSTRAIN** argument as the last parameter:

```
mapFrom01(x, toLow, toHigh, CONSTRAIN)
```

See [*mapFloat\(\)*](#) for more details.

Example

```
#include <Plaquette.h>

Wave modulator(SINE, 10.0);

Wave oscillator(SQUARE, 1.0);

DigitalOut led(13);

void step() {
    // Change duty-cycle of oscillator in range [0.2, 0.8].
    float skew = mapFrom01(modulator, 0.2, 0.8); // alternative: modulator.mapTo(0.2, 0.8)
    oscillator.skew(skew);

    // Send to LED.
    oscillator >> led;
}
```

Reference

float pq: **mapFrom01**(double value, double toLow, double toHigh, MapMode mode = UNCONSTRAIN)

Re-maps a number in range [0, 1] to a new range.

Parameters

- **value** – the number to map (in [0,1])
- **toLow** – the lower bound of the value’s target range
- **toHigh** – the upper bound of the value’s target range
- **mode** – set to **CONSTRAIN** to constrain the return value between toLow and toHigh or **WRAP** for the value to wrap around

Returns

the mapped value in [toLow, toHigh]

See Also

- *mapFloat()*
- *mapTo01()*

3.7.3 mapTo01()

Re-maps a number between 0.0 and 1.0. That is, a value of **fromLow** would get mapped to 0.0, a value of **fromHigh** to 1.0, values in-between to values in-between, etc.

```
mapTo01(x, fromLow, fromHigh)
```

is equivalent to:

```
mapFloat(x, fromLow, fromHigh, 0, 1)
```

By default, does *not* constrain output to stay within the [fromHigh, toHigh] range, because out-of-range values are sometimes intended and useful. In order to constrain the return value within range, use the **CONSTRAIN** argument as the last parameter:

```
mapTo01(x, fromLow, fromHigh, CONSTRAIN)
```

See *mapFloat()* for more details.

Example

```
#include <Plaquette.h>

AnalogOut led(9);

void step() {
    // Generate a sinusoidal values between -1 and 1.
    float x = sin(seconds());

    // Remap to the range [0, 1] and send to LED.
    mapTo01(x, -1, 1) >> led;
}
```

Reference

float pq: **mapTo01**(double value, double fromLow, double fromHigh, MapMode mode = UNCONSTRAIN)

Re-maps a number to the [0, 1] range.

Parameters

- **value** – the number to map
- **fromLow** – the lower bound of the value's current range
- **fromHigh** – the upper bound of the value's current range
- **mode** – set to **CONSTRAIN** to constrain the return value between toLow and toHigh or **WRAP** for the value to wrap around

Returns

the mapped value in [0, 1]

See Also

- *mapFloat()*
- *mapFrom01()*

3.7.4 randomFloat()

This function returns a random real-valued number.

Example

```
#include <Plaquette.h>

DigitalOut led(13);

void step() {
  // 2% probability to toggle the LED
  if (randomFloat() < 0.02)
    led.toggle();
}
```

Reference

float pq::randomFloat()

Generates a uniform random number in the interval [0,1).

float pq::randomFloat(float max)

Generates a uniform random number in the interval [0,max).

float pq::randomFloat(float min, float max)

Generates a uniform random number in the interval [min,max) (b>a).

See Also

- random()
- randomTrigger()

3.7.5 randomTrigger()

Interactive designers commonly want some events to happen **occasionally**, not on every step, in a random fashion. This function makes that easy: it gives you a simple yes/no (true/false) answer that, when checked in each step(), will trigger roughly once per second, minute, or any other time window that is specified.

You can think of it as a “random but controlled” switch: you decide how sparse or frequent the events should be (for example, about once every 5 seconds), and the function takes care of the rest, even if your loop runs very fast.

Example

```
#include <Plaqueette.h>

DigitalOut led(13);

void step() {
    // Returns true on average once every 5 seconds.
    if (randomTrigger(5.0))
        led.toggle(); // toggle LED
}
```

Reference

`bool pq::randomTrigger(float timeWindow)`

Randomly triggers an event about once per time window, on average based on sampling rate of primary engine.

Call this function once in each `step()`. It will occasionally return true, with the frequency adjusted so that you get roughly one event for each `timeWindow` period, no matter how fast your loop is running.

Parameters

timeWindow – duration of the window (in seconds)

Returns

true when an event occurs during this sample

`bool pq::randomTrigger(float timeWindow, float samplePeriod)`

Randomly triggers an event about once per time window, on average.

Call this function once in each loop or sample. It will occasionally return true, with the frequency adjusted so that you get roughly one event for each `timeWindow` period, no matter how fast your loop is running.

Parameters

- **timeWindow** – Duration of the window
- **samplePeriod** – Duration of a sample (in same unit as `timeWindow` eg. seconds)

Returns

true when an event occurs during this sample

See Also

- `random()`
- `randomFloat()`

3.7.6 seconds()

This function returns the number of seconds since the program started.

Example

```
#include <Plaquette.h>

DigitalOut led(13, DIRECT);

void begin() {
  // Initialize LED as "off".
  led.off();
}

void step() {
  // Switch the LED on after 10 seconds.
  if (seconds() > 10)
    led.on();
}
```

Reference

float pq::seconds(bool referenceTime = true)

Returns time in seconds.

Optional parameter allows to ask for reference time (default) which will yield the same value through one iteration of step(), or “real” time which will return the current total running time.

Parameters

referenceTime – determines whether the function returns the reference time or the real time

Returns

the time in seconds

See Also

- `micros()`
- `millis()`

3.7.7 wrap()

Restricts a value to an interval [low, high) by wrapping it around.

Code	Result
<code>wrap(1.0, 1.0, 5.0)</code>	1.0
<code>wrap(3.0, 1.0, 5.0)</code>	3.0
<code>wrap(4.9999, 1.0, 5.0)</code>	4.9999
<code>wrap(5.0, 1.0, 5.0)</code>	1.0
<code>wrap(6.0, 1.0, 5.0)</code>	2.0
<code>wrap(-1.0, 1.0, 5.0)</code>	3.0

Two alternative versions are provided:

Version	Equivalent to
<code>wrap(x, high)</code>	<code>wrap(x, 0.0, high)</code>
<code>wrap01(x)</code>	<code>wrap(x, 0.0, 1.0)</code>

Example

Ramp LED up and then back to zero once every 10 second:

```
#include <Plaquette.h>

AnalogOut led(9);

void begin() {
}

void step() {
    wrap(seconds(), 10.0) >> led;
}
```

Reference

float pq: **wrap**(double x, double low, double high)

Restricts value to the interval [low, high) by wrapping it around.

Parameters

- **x** – the value to wrap
- **low** – the lower boundary
- **high** – the higher boundary

Returns

the value wrapped around [low, high) or [high, low) if high < low

float pq: **wrap**(double x, double high)

Restricts value to the interval [0, high) by wrapping it around.

Parameters

- **x** – the value to wrap
- **high** – the higher bound

Returns

the value wrapped around [0, high) or [high, 0) if high is negative

float pq: **wrap01**(double x)

Restricts value to the interval [0, 1) by wrapping it around.

Parameters

x – the value to wrap

Returns

the value wrapped around [0, 1).

See Also

- `mapFloat()`
- `mapTo01()`

3.8 Structure

Core structural functions and operators.

3.8.1 Engine

A control structure that acts like the **conductor of an orchestra**, managing an ensemble of **units**. It handles their initialization, updates, and timing, ensuring that all components remain synchronized.

By default, all units are automatically added to the **primary engine** which can be accessed using the global object `Plaquette`. By using **secondary engines** you can organize and optimize your code, allowing for multi-tasking, grouping units, switching between ensembles of units, and save power by running engines at lower frequency.

Usage

To create and use an engine, simply declare it:

```
Engine myEngine;
```

To assign units to a specific engine, add it as the last parameter at construction:

```
Wave wave(1.0, 0.2, myEngine); // Use myEngine instead of the default
```

Each engine provides its own time measurements:

```
float sec = myEngine.seconds(); // Time since engine started, in seconds
uint32_t ms = myEngine.milliseconds(); // Time in milliseconds
uint64_t us = myEngine.microseconds(); // Time in microseconds
```

You will need to call the engine's `begin()` function at initialization, and then its `step()` function at a regular pace.

Sample Rate

The Engine `step()` function computes timings and performs background update operations on units. It is possible to set a specific (target) sample rate using `sampleRate(rate)` to reduce computation load for operations that do not need to be performed as fast as possible.

The engine's `step()` function returns:

- `true` if the step has been fully performed
- `false` if it is still waiting for the next “tick”

As an example, if one sets a 100 Hz sample rate on engine `myEngine` by calling `myEngine.sampleRate(100)`, the `myEngine.step()` function should return `true` about 100 times per second and `false` otherwise.

To fully support custom sample rates and avoid performing unnecessary operations on units, the `step()` function should be used as a **guard condition** in the main stepping loop.

```
void begin() {
    myEngine.sampleRate(100); // Target sample rate: 100 Hz.
}

void loop() {
    if (!myEngine.step()) // Guard condition.
        return; // Exit the loop.

    // The operations below will be performed at ~100 Hz.
    if (button)
        wave >> led;
    ...
}
```

For more in-depth explanations and examples please read *Synchronizing Groups of Units with Secondary Engines*.

Example

This example demonstrates the use of a secondary engine on an Arduino Uno or Nano using timer2 interrupt.

```
#include <Plaquette.h>

Engine timerEngine; // The secondary timer engine.

Metronome serialMetro(1.0); // Metronome (primary engine).

Metronome toggleMetro(0.25, timerEngine); // Metronome (timer engine).
DigitalOut led(LED_BUILTIN, timerEngine); // Built-in LED (timer engine).

Monitor monitor(115200); // Serial monitor.

// Primary engine begin().
void begin() {
    timerEngine.begin(); // Begin timer engine.
    timerSetup();        // Initialize timer interrupt timer2.
}

// Primary engine step().
void step() {
    if (serialMetro)
        println("step");
}

// Timer2 interrupt: will be called at 1kHz frequency.
ISR(TIMER2_COMPA_vect) {
    timerEngine.step(); // Step engine.

    if (toggleMetro) // Toggle LED on metro bang.
        led.toggle();
}

// Timer2 setup for 1kHz on AVR.
```

(continues on next page)

(continued from previous page)

```

void timerSetup() {
    // Stop Timer2
    TCCR2A = 0;
    TCCR2B = 0;
    TCNT2  = 0;

    // Set compare match register for 1 kHz increments.
    // 16 MHz / (prescaler * 1000) - 1 = OCR2A
    // Try prescaler = 128 => OCR2A = (16e6 / (128 * 1000)) - 1 = ~124
    OCR2A = 124;

    TCCR2A |= (1 << WGM21); // CTC mode (Clear Timer on Compare Match).
    TCCR2B |= (1 << CS22) | (1 << CS20); // Set prescaler to 128.
    TIMSK2 |= (1 << OCIE2A); // Enable Timer2 compare interrupt.

    sei(); // Enable global interrupts.
}
    
```

Reference

class **Engine**

The main Plaquette static class containing all the units.

Public Functions

void preBegin()

Initializes all components (calls begin() on all of them).

void postBegin()

Performs additional tasks after the class to begin().

inline void preStep()

Updates all components (calls step() on all of them).

inline bool timeStep()

Performs additional tasks after the class to step().

Returns

true if the program should

inline void begin()

Function to be used within the PlaquetteLib context (needs to be called at top of setup() method).

inline bool step()

Function to be used within the PlaquetteLib context (needs to be called at top of loop() method).

inline void end()

Optional function to be used within the PlaquetteLib context.

No need to call it if the program is looping indefinitely. Call if the program stops at some point.

inline size_t **nUnits()**

Returns the current number of units.

float **seconds**(bool referenceTime = true) const

Returns time in seconds.

Optional parameter allows to ask for reference time (default) which will yield the same value through one iteration of step(), or “real” time which will return the current total running time.

Parameters

referenceTime – determines whether the function returns the reference time or the real time

Returns

the time in seconds

uint32_t **milliSeconds**(bool referenceTime = true) const

Returns time in milliseconds.

Optional parameter allows to ask for reference time (default) which will yield the same value through one iteration of step(), or “real” time which will return the current total running time.

Parameters

referenceTime – determines whether the function returns the reference time or the real time

Returns

the time in milliseconds

uint64_t **microSeconds**(bool referenceTime = true) const

Returns time in microseconds.

Optional parameter allows to ask for reference time (default) which will yield the same value through one iteration of step(), or “real” time which will return the current total running time.

Parameters

referenceTime – determines whether the function returns the reference time or the real time

Returns

the time in microseconds

inline unsigned long **nSteps()** const

Returns number of steps.

inline bool **hasAutoSampleRate()** const

Returns true iff the auto sample rate mode is enabled (default).

void **autoSampleRate()**

Enables auto sample rate mode (default).

void **sampleRate**(float sampleRate)

Sets sample rate to a fixed value, thus disabling auto sampling rate.

void **samplePeriod**(float samplePeriod)

Sets sample period to a fixed value, thus disabling auto sampling rate.

inline float **sampleRate()** const

Returns sample rate.

inline float **samplePeriod()** const

Returns sample period.

inline uint32_t **deltaTimeMicroSeconds()** const

Returns time between steps (in microseconds).

inline float **deltaTimeSecondsTimesFixed32Max()** const

Returns time between steps, expressed in fixed point propotion.

inline bool **isPrimary()** const

Returns true if this Engine is the main.

bool **randomTrigger**(float timeWindow)

Randomly triggers an event about once per time window, on average.

Call this function once in each step(). It will occasionally return true, with the frequency adjusted so that you get roughly one event for each `timeWindow` period, no matter how fast your loop is running.

Parameters

timeWindow – duration of the window (in seconds)

Returns

true when an event occurs during this sample

void **referenceClock**(unsigned long (*clockFunction)())

Sets base function returning microseconds.

Default: `micros()`.

Parameters

clockFunction – pointer to a function returning microseconds

Public Static Functions

static *Engine* &**primary()**

Returns the main instance of Plaquette.

See Also

- *begin()*
- *step()*
- *seconds()*
- *Synchronizing Groups of Units with Secondary Engines*

3.8.2 Value Units

Plaquette's **value-units** Float, Integer, and Boolean wrap basic types (float, int or bool), allowing to use them transparently as **units** in a Plaquette flow. These units can be used like a normal variables but also send to and receive from other units using the flow operator (>>). You can think of value-units as *variables that can participate in data flows*.

The *flow operator* >> connects is **not** compatible with primitive types such as float, int, and bool.

Value units address this problem by turning primitive variables into Plaquette units.

This code shows valid and invalid uses of the flow operator:

```
#include <Plaquette.h>

Wave wave(1.0f); // wave oscillator
DigitalOut led(LED_BUILTIN); // output LED
float x = 0.0f; // primitive type variable (float)

void step() {
    wave >> led; // ✓ valid : unit >> unit
    x >> led; // ✓ valid : float >> unit
    wave >> x; // ✗ error : unit >> float
}
```

Example

This example combines two oscillators, stores the result in a Float unit, and reuses it to control two LEDs.

```
#include <Plaquette.h>

// Create two wave oscillators.
Wave waveA(SINE, 1.0f);
Wave waveB(SINE, 1.2f);

// Create two LED outputs.
AnalogOut led1(9);
AnalogOut led2(11);

// Create a Float to store the combined signal.
Float mix;

void step() {
    // Sum the two waves and store the result.
    (waveA + waveB) >> mix;
    // Compute average.
    mix /= 2;
    // Reuse the same value in multiple places.
    mix >> led1;
    // Use math to invert the signal.
    (1-mix) >> led2;
}
```

Example

This example uses the Boolean unit type to combine two button states into a single logical condition. It activates an LED only when both buttons are pressed and plots the states.

```
#include <Plaquette.h>

// Create two button inputs.
DigitalIn buttonA(BUTTON_A_PIN);
DigitalIn buttonB(BUTTON_B_PIN);
```

(continues on next page)

(continued from previous page)

```
// Create LED output.
DigitalOut led(LED_BUILTIN);

// Create monitor.
Plotter plotter(115200, "pressA,pressB,pressBoth"); // plotter with baud rate and labels

// Create a Boolean to store the combined condition.
Boolean bothPressed = false;

void step() {
    // Combine the two button states.
    (buttonA && buttonB) >> bothPressed;
    // Send to LED.
    bothPressed >> led;
    // Plot states.
    buttonA >> plotter;
    buttonB >> plotter;
    bothPressed >> plotter;
}
```

See Also

- `>>` (*flow*)

3.8.3 prepare()

The `prepare()` function is an optional function that is called at the very beginning of a sketch, before any units are initialized. Use it when you need to perform operations that must happen before Plaquette units call their `begin()` methods, typically when using non-Plaquette objects or functions that need to be built or configured before units are initialized.

Tip: In Plaquette, function `prepare()` is rarely used and is **optional**: only declare it if you need to perform operations before units are initialized.

Example

```
#include <Plaquette.h>

AnalogIn sensor(A0);
DigitalOut led(13);

void prepare() {
    // Early setup before units initialize.
    Serial.begin(115200);
}

void begin() {
```

(continues on next page)

(continued from previous page)

```
// Runs after units are initialized.
sensor.smooth();
}

void step() {
  sensor >> led;
}
```

See Also

- *begin()*
- *step()*

3.8.4 begin()

The `begin()` function is optionally called at the start of a sketch to initialize units, start using libraries, etc. The `begin()` function will only run once, after each powerup or reset of the board.

Hint: Function `begin()` is the Plaquette equivalent of Arduino's `setup()`. However, Plaquette takes care of many of the initialization calls that need to be done in Arduino such as `pinMode()`. Therefore in many cases it will contain only a few calls, or can be omitted completely.

Example

```
#include <Plaquette.h>

Wave oscillator(1.0);
AnalogIn input(A0);

void begin() {
  oscillator.period(1.0);
  oscillator.skew(0.75);
  input.smooth();
}

void step() {
  // ...
}
```

Tip: In Plaquette, function `begin()` is **optional**: only declare it if you need to perform a specific operation at startup.

Example

```
#include <Plaquette.h>

Wave oscillator(1.0);
Plotter plotter(115200);

// begin() function is not declared

void step() {
    oscillator >> plotter; // Send the wave to the plotter for visualization
}
```

See Also

- *prepare()*
- *step()*

3.8.5 step()

The `step()` function does precisely what its name suggests, and performs one processing step that loops indefinitely as fast as possible, allowing your program to change and respond. Use it to actively control the board.

Hint: Function `step()` is the Plaquette equivalent of Arduino's `loop()`.

Important: It is highly recommended that this function executes as fast as possible. Hence, one should performing computationally-intensive processing or calling blocking functions such as `delay()`

Tip: In Plaquette, function `step()` is **optional**: only declare it if you need it... (you will, most of the time!)

Example

```
#include <Plaquette.h>

DigitalIn button(2);

DigitalOut led(13);

void step() {
    button >> led;
}
```

See Also

- [prepare\(\)](#)
- [begin\(\)](#)

3.8.6 [] (arrays)

An array is a collection of variables or objects that are accessed with an index number. Arrays can be complicated, but using simple arrays is relatively straightforward.

For a general description of arrays, please refer to [this page](#).

Arrays of Plaquette units such as *DigitalIn*, *Wave*, and *MinMaxScaler* can be easily created using the following syntax:

```
UnitType array[] = { UnitType(...), UnitType(...), ... };
```

For example, the following code will create an array of three (3) digital outputs on pins 10, 11, and 12:

```
DigitalOut leds[] = { DigitalOut(10), DigitalOut(11), DigitalOut(12) };
```

When initializing a unit with a single parameter, one can simply use the value of the parameter at creation time. Hence the previous code could be rewritten as:

```
DigitalOut leds[] = { 10, 11, 12 };
```

When more than a single parameter is used, however, it needs to be called explicitly with the unit name:

```
Wave oscillators[] = { 1.0, 2.0, Wave(3.0, 0.8) };
```

Warning: Units in array need to be all of the same type. In other words, it is not currently possible to mix different types of objects such as *DigitalIn* and *Wave* in the same array.

Example

```
#include <Plaquette.h>

AnalogOut leds[] = { 9, 10, 11 };

// Creates three different kinds of oscillators with a 2 seconds period.
Wave oscillators[] = { Wave(SQUARE, 2.0), Wave(TRIANGLE, 2.0), Wave(SINE, 2.0) };

void step() {
    // Send each oscillator to its corresponding LED.
    for (int i=0; i<3; i++) {
        oscillators[i] >> leds[i];
    }
}
```


3.8.7 . (dot)

Provides access to an object's methods and data. An object is one instance of a class and may contain both methods (object functions) and data (object variables and constants), as specified in the class definition. The dot operator directs the program to the information encapsulated within an object.

Example

Switches LED on every 4 seconds.

```
#include <Plaquette.h>

DigitalOut led(13);

void begin() {
    led.off();
}

void step() {
    if (round(seconds()) % 4 == 0)
        led.on();
    else
        led.off();
}
```

Syntax

```
object.method()
object.variable
```

3.8.8 >> (flow)

Sends data across units from left to right. This operator is specific to Plaquette and can be used in a chained manner.

The operation uses the `get()` and `put()` methods of units in such a way that:

```
input >> output;
```

is equivalent to:

```
output.put(input.get());
```

Numerical and boolean values can also be used:

```
12 >> output;
0.8 >> output;
true >> output;
```

Example

```
#include <Plaquette.h>

AnalogIn sensor(A0);

MinMaxScaler scaler;

Plotter plotter(115200);

AnalogOut led(9);

void step() {
  // Rescale value and send the result to LED.
  sensor >> scaler >> led;

  // You can also use flow operators to stream data to a Plotter.
  sensor >> plotter; // prints sensor value
  scaler >> plotter; // prints rescaled value
}
```

Syntax

```
input >> output
input >> filter >> output
```

3.9 Extra

Extra units and functions.

3.9.1 Easings

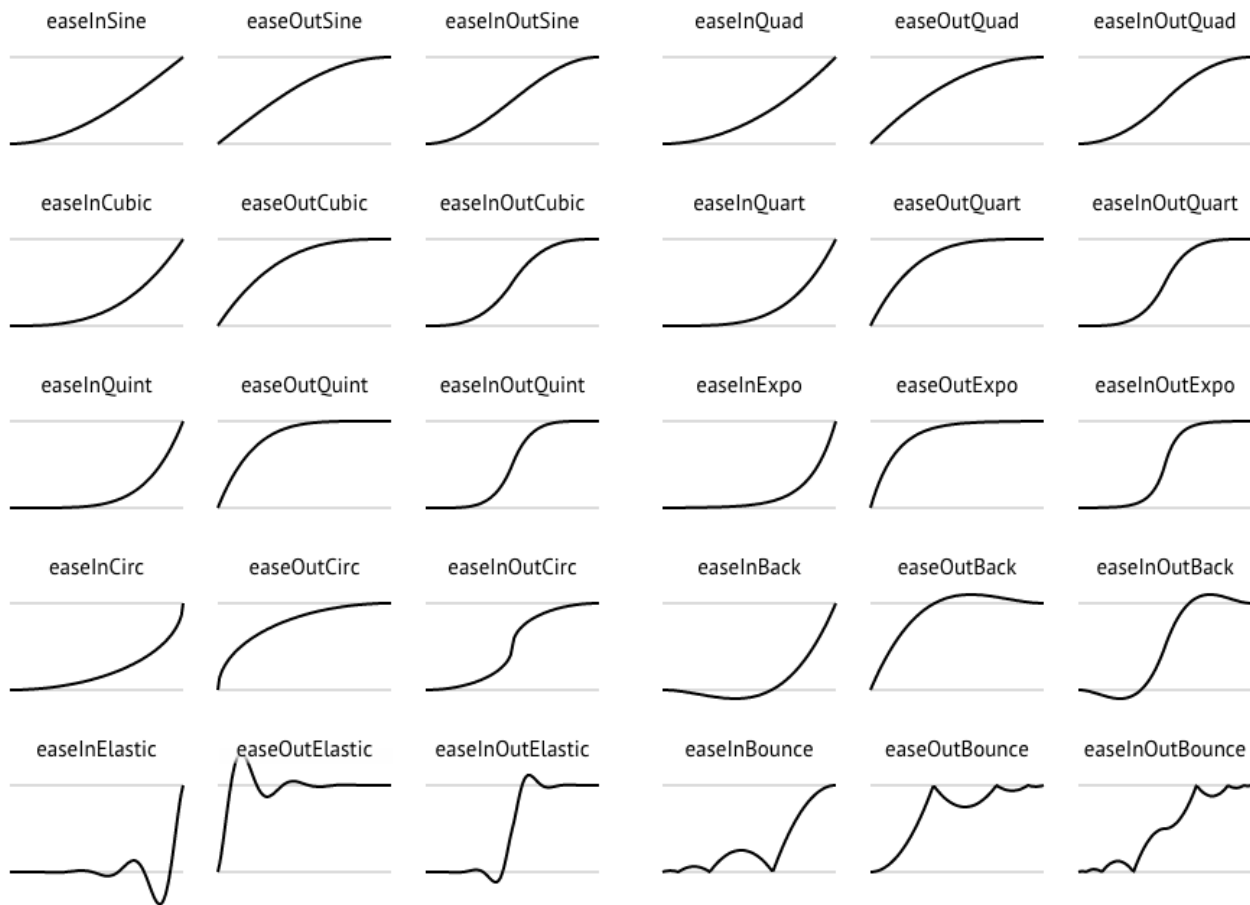
Easing functions apply non-linear effects to changing values, in order to create expressive real-time outputs. Plaquette provides users with a wide range of such functions, typically used with a Ramp unit.

All easing functions have the same signature:

```
float easeFunction(float t)
```

Value x should be between 0.0 and 1.0, the returned value is also between 0.0 and 1.0.

This is the list of all easing functions (source: <http://easings.net>):



See Also

- [Ramp](#)

3.9.2 ContinuousServoOut

A source unit that controls a continuous rotation servo-motor. A continuous servo-motor can move indefinitely forward or backwards.

Servo motors have three wires: power, ground, and signal. The power wire is typically red, and should be connected to the 5V pin on the Arduino board. The ground wire is typically black or brown and should be connected to a ground pin on the Arduino board. The signal pin is typically yellow, orange or white and should be connected to a digital pin on the Arduino board. Note that servos draw considerable power, so if you need to drive more than one or two, you'll probably need to power them from a separate supply (i.e. not the +5V pin on your Arduino). Be sure to connect the grounds of the Arduino and external power supply together.

Example

Every time a button is pushed, the motor is stopped. Then upon button release it starts moving in the opposite direction.

```
#include <Plaquette.h>

// The servo-motor output on pin 9.
ContinuousServoOut servo(9);

// The push-button.
DigitalIn button(2);

// Preserves the servo last speed value.
float lastValue = 0;

void begin() {
    // Debounce button.
    button.debounce();
    // Initialize servo position
    servo.put(1.0);
}

void step() {
    if (button) {
        // Save speed.
        lastValue = servo.get();
        // Stop servo.
        servo.stop();
    }
    else if (button.fell()) {
        // Reset speed.
        servo.put(lastValue);
        // ... then invert it.
        servo.reverse();
    }
}
```

class **ContinuousServoOut** : public pq::AbstractServoOut
 Continuous servo-motor.

Public Functions

ContinuousServoOut(uint8_t pin, *Engine* &engine = *Engine::primary*())
 Constructor for a continuous rotation servo-motor.

Parameters

pin – the pin number

virtual void **stop**()
 Stops the servo-motor.

virtual void **invert**()
 Sends servomotor in opposite direction.

virtual float **put**(float value)

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline uint8_t **pin**() const

Returns the pin this servomotor is attached to.

inline virtual bool **isActive**()

Returns true if the servomotor is active.

virtual void **setIsActive**(bool active)

Activates or deactivates the servomotor.

inline virtual void **activate**()

Activates the servomotor (default).

inline virtual void **deactivate**()

Deactivates the servomotor.

inline virtual float **get**()

Returns value in [0, 1].

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliSeconds**() const

Returns engine time in milliseconds.

inline uint64_t **microSeconds**() const

Returns engine time in microseconds.

inline unsigned long **nSteps**() const

Returns number of engine steps.

inline float **sampleRate**() const

Returns engine sample rate.

inline float **samplePeriod**() const

Returns enginesample period.

inline **operator float**()

Object can be used directly to access its value.

inline explicit **operator bool**()

Operator that allows usage in conditional expressions.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- *AnalogOut*
- *ServoOut*

3.9.3 ServoOut

A source unit that controls a standard servo-motor.

Servo motors have three wires: power, ground, and signal. The power wire is typically red, and should be connected to the 5V pin on the Arduino board. The ground wire is typically black or brown and should be connected to a ground pin on the Arduino board. The signal pin is typically yellow, orange or white and should be connected to a digital pin on the Arduino board. Note that servos draw considerable power, so if you need to drive more than one or two, you'll probably need to power them from a separate supply (i.e. not the +5V pin on your Arduino). Be sure to connect the grounds of the Arduino and external power supply together.

Example

Sweeps the shaft of a servo motor back and forth across 180 degrees.

```
#include <Plaqueette.h>

// The servo-motor output on pin 9.
ServoOut servo(9);

// Oscillator to make the servo sweep.
Wave oscillator(SINE, 2.0);

void begin() {
    // Position the servo in center.
    servo.center();
}

void step() {
    // Updates the value and send it back as output.
    oscillator >> servo;
}
```

class **ServoOut** : public pq::AbstractServoOut

Standard servo-motor (angular).

Public Functions

ServoOut(uint8_t pin, *Engine* &engine = *Engine::primary*())

Constructor for a standard servo-motor.

Parameters

pin – the pin number

virtual float **putAngle**(float angle)

Sets the servomotor position to a specific angle between 0 and 180 degrees.

Parameters

angle – the angle in degrees

Returns

the current angle

virtual float **getAngle**()

Return the current angular angle in [0, 180].

virtual float **put**(float value)

Pushes value into the unit.

Parameters

value – the value sent to the unit

Returns

the new value of the unit

inline uint8_t **pin**() const

Returns the pin this servomotor is attached to.

inline virtual bool **isActive**()

Returns true if the servomotor is active.

virtual void **setIsActive**(bool active)

Activates or deactivates the servomotor.

inline virtual void **activate**()

Activates the servomotor (default).

inline virtual void **deactivate**()

Deactivates the servomotor.

inline virtual float **get**()

Returns value in [0, 1].

inline virtual float **mapTo**(float toLow, float toHigh)

Maps value to new range.

inline float **seconds**() const

Returns engine time in seconds.

inline uint32_t **milliseconds**() const

Returns engine time in milliseconds.

inline uint64_t **microseconds**() const

Returns engine time in microseconds.

inline unsigned long **nSteps()** const

Returns number of engine steps.

inline float **sampleRate()** const

Returns engine sample rate.

inline float **samplePeriod()** const

Returns enginesample period.

inline **operator float()**

Object can be used directly to access its value.

inline explicit **operator bool()**

Operator that allows usage in conditional expressions.

Public Static Functions

static inline bool **analogToDigital**(float f)

Converts analog (float) value to digital (bool) value.

static inline float **digitalToAnalog**(bool b)

Converts digital (bool) value to analog (float) value.

See Also

- *AnalogOut*
- *ContinuousServoOut*

LIBRARIES

RELATED INFO

5.1 Community Guidelines

Plaquette is an open-source software used by artists, makers, researchers, and developers interested in creative tangible computing. The goal of these guidelines is to help everyone collaborate and communicate in a friendly and inclusive way, whether you are reporting a bug, suggesting a feature, or sharing your ideas.

If you participate in any Plaquette space (issues, pull requests, or discussions on GitHub), please take a moment to read and follow these principles.

5.1.1 Be Respectful and Inclusive

Everyone is welcome, regardless of experience level, background, or identity. We ask that you be kind, patient, and supportive of others' learning processes.

All interactions on GitHub are covered by the [Code of Conduct](#).

If you ever experience or witness behavior that makes you uncomfortable, please reach out to the maintainers directly.

5.1.2 Contributing

Plaquette grows through its community of users and contributors. You can help by:

- Reporting bugs or unexpected behavior.
- Suggesting new features or improvements.
- Fixing typos, improving examples, or clarifying documentation.
- Sharing creative projects that use Plaquette.

All contribution instructions and coding conventions are described in our [Contribution Guidelines](#).

5.1.3 Reporting Bugs and Requesting Features

To make it easier for everyone:

- First, check the existing issues on GitHub to see if your topic already exists.
- If not, open a new issue and choose one of these forms:
 - **Bug report** if something isn't working.
 - **Feature idea** if you'd like to suggest an improvement.
 - **Help needed** if you're stuck and need a hand.
 - **Quick note** for a short comment, idea, or thought.

All discussions happen transparently on the [issue tracker](#).

5.1.4 Getting Support

If you need help using Plaquette please start by browsing the [documentation](#).

You can also look at examples on [GitHub](#) or directly from the Arduino application in menu **File > Examples > Plaquette**.

If you still have trouble, open a **Help needed** issue on the [issue tracker](#) and describe what you're trying to do. There's no wrong question!

5.1.5 License and Attribution

Plaquette is free software under the **GNU GPL v3 or later** license. See the [LICENSE](#) file for details.

5.2 Credits

5.2.1 Core Developers

- Sofian Audry (main code, API design, documentation) • [Website](#) • [GitHub](#)
- Thomas Ouellet Fredericks (original concept, API design) • [Website](#) • [GitHub](#)

5.2.2 Contributors

- Erin Gee (API design, documentation, testing) • [Website](#) • [GitHub](#)
- Luana Belinsky (testing) • [Website](#) • [GitHub](#)
- Marianne Fournier (documentation) • [GitHub](#)
- Ian Donnelly (logo) • [Website](#)
- Matthew Loewen (code) • [Website](#) • [GitHub](#)
- Samuel Favreau (code) • [Website](#)

5.2.3 Partners

- [mXlab](#)
- [LFO](#)
- [EnsadLab](#)
- [SAT](#)

5.2.4 Funding

- [Canada Council for the Arts](#)
- [NSERC](#)
- [FRQSC](#)

Plaquette's base source code was produced as part of a research project at labXmodal. A special thanks to [Chris Salter](#) for his support.

5.2.5 Inspiration

- [Arduino](#)
- [ChuckK](#)
- [mbed](#)
- [Processing](#)
- [Pure Data](#)
- [TouchDesigner](#)

5.3 License

Plaquette is distributed under the [Gnu General Public License v 3.0](#).

The text of the Plaquette documentation is licensed under a [Creative Commons Attribution-ShareAlike 3.0 License](#). Parts of the text was copied and/or adapted from the [Arduino documentation](#). Code samples in the guide are released into the public domain.

The Plaquette documentation is licensed under a [Creative Commons Attribution-Share Alike 3.0 License](#). Parts of the documentation has been borrowed and/or adapted from the [Arduino Reference](#) and from the [Processing Reference](#) texts.

A

- Alarm (C++ class), 135
- Alarm::addTime (C++ function), 138
- Alarm::Alarm (C++ function), 135
- Alarm::analogToDigital (C++ function), 138
- Alarm::changed (C++ function), 136
- Alarm::changeState (C++ function), 136
- Alarm::digitalToAnalog (C++ function), 138
- Alarm::Duration (C++ function), 137
- Alarm::duration (C++ function), 137
- Alarm::elapsed (C++ function), 138
- Alarm::fell (C++ function), 136
- Alarm::finished (C++ function), 135
- Alarm::get (C++ function), 136
- Alarm::getInt (C++ function), 136
- Alarm::hasPassed (C++ function), 138
- Alarm::isComplete (C++ function), 137
- Alarm::isFinished (C++ function), 137
- Alarm::isOff (C++ function), 136
- Alarm::isOn (C++ function), 136
- Alarm::isRunning (C++ function), 138
- Alarm::isStarted (C++ function), 138
- Alarm::mapTo (C++ function), 137
- Alarm::microSeconds (C++ function), 137
- Alarm::milliSeconds (C++ function), 137
- Alarm::nSteps (C++ function), 137
- Alarm::off (C++ function), 136
- Alarm::on (C++ function), 136
- Alarm::onChange (C++ function), 136
- Alarm::onFall (C++ function), 136
- Alarm::onFinish (C++ function), 135
- Alarm::onRise (C++ function), 136
- Alarm::operator bool (C++ function), 137
- Alarm::pause (C++ function), 137
- Alarm::progress (C++ function), 137
- Alarm::put (C++ function), 136
- Alarm::putOn (C++ function), 136
- Alarm::resume (C++ function), 138
- Alarm::rose (C++ function), 136
- Alarm::samplePeriod (C++ function), 137
- Alarm::sampleRate (C++ function), 137
- Alarm::seconds (C++ function), 137
- Alarm::setTime (C++ function), 136
- Alarm::start (C++ function), 137
- Alarm::stop (C++ function), 138
- Alarm::toggle (C++ function), 136
- Alarm::togglePause (C++ function), 138
- AnalogIn (C++ class), 100
- AnalogIn::AnalogIn (C++ function), 100
- AnalogIn::analogToDigital (C++ function), 101
- AnalogIn::cutoff (C++ function), 101
- AnalogIn::digitalToAnalog (C++ function), 101
- AnalogIn::get (C++ function), 100
- AnalogIn::infiniteTimeWindow (C++ function), 101
- AnalogIn::mapTo (C++ function), 100
- AnalogIn::microSeconds (C++ function), 100
- AnalogIn::milliSeconds (C++ function), 100
- AnalogIn::mode (C++ function), 101
- AnalogIn::noSmooth (C++ function), 101
- AnalogIn::noTimeWindow (C++ function), 101
- AnalogIn::nSteps (C++ function), 100
- AnalogIn::operator bool (C++ function), 100
- AnalogIn::operator float (C++ function), 100
- AnalogIn::pin (C++ function), 101
- AnalogIn::put (C++ function), 100
- AnalogIn::rawRead (C++ function), 100
- AnalogIn::read (C++ function), 100
- AnalogIn::samplePeriod (C++ function), 100
- AnalogIn::sampleRate (C++ function), 100
- AnalogIn::seconds (C++ function), 100
- AnalogIn::smooth (C++ function), 101
- AnalogIn::timeWindow (C++ function), 101
- AnalogIn::timeWindowIsInfinite (C++ function), 101
- AnalogOut (C++ class), 103
- AnalogOut::AnalogOut (C++ function), 103
- AnalogOut::analogToDigital (C++ function), 104
- AnalogOut::digitalToAnalog (C++ function), 104
- AnalogOut::get (C++ function), 103
- AnalogOut::invert (C++ function), 103
- AnalogOut::mapTo (C++ function), 103
- AnalogOut::microSeconds (C++ function), 103
- AnalogOut::milliSeconds (C++ function), 103
- AnalogOut::mode (C++ function), 104

AnalogOut::nSteps (C++ function), 103
 AnalogOut::operator bool (C++ function), 104
 AnalogOut::operator float (C++ function), 103
 AnalogOut::pin (C++ function), 104
 AnalogOut::put (C++ function), 103
 AnalogOut::rawWrite (C++ function), 103
 AnalogOut::samplePeriod (C++ function), 103
 AnalogOut::sampleRate (C++ function), 103
 AnalogOut::seconds (C++ function), 103
 AnalogOut::write (C++ function), 103

C

Chronometer (C++ class), 139
 Chronometer::addTime (C++ function), 140
 Chronometer::analogToDigital (C++ function), 141
 Chronometer::Chronometer (C++ function), 139
 Chronometer::digitalToAnalog (C++ function), 141
 Chronometer::elapsed (C++ function), 140
 Chronometer::get (C++ function), 139
 Chronometer::hasPassed (C++ function), 140
 Chronometer::isRunning (C++ function), 140
 Chronometer::isStarted (C++ function), 140
 Chronometer::mapTo (C++ function), 140
 Chronometer::microSeconds (C++ function), 140
 Chronometer::milliSeconds (C++ function), 140
 Chronometer::nSteps (C++ function), 140
 Chronometer::operator bool (C++ function), 140
 Chronometer::operator float (C++ function), 140
 Chronometer::pause (C++ function), 140
 Chronometer::put (C++ function), 139
 Chronometer::resume (C++ function), 140
 Chronometer::samplePeriod (C++ function), 140
 Chronometer::sampleRate (C++ function), 140
 Chronometer::seconds (C++ function), 139
 Chronometer::setTime (C++ function), 140
 Chronometer::start (C++ function), 141
 Chronometer::stop (C++ function), 141
 Chronometer::togglePause (C++ function), 141
 ContinuousServoOut (C++ class), 206
 ContinuousServoOut::activate (C++ function), 207
 ContinuousServoOut::analogToDigital (C++ function), 208
 ContinuousServoOut::ContinuousServoOut (C++ function), 206
 ContinuousServoOut::deactivate (C++ function), 207
 ContinuousServoOut::digitalToAnalog (C++ function), 208
 ContinuousServoOut::get (C++ function), 207
 ContinuousServoOut::invert (C++ function), 206
 ContinuousServoOut::isActive (C++ function), 207
 ContinuousServoOut::mapTo (C++ function), 207
 ContinuousServoOut::microSeconds (C++ function), 207

ContinuousServoOut::milliSeconds (C++ function), 207
 ContinuousServoOut::nSteps (C++ function), 207
 ContinuousServoOut::operator bool (C++ function), 207
 ContinuousServoOut::operator float (C++ function), 207
 ContinuousServoOut::pin (C++ function), 207
 ContinuousServoOut::put (C++ function), 206
 ContinuousServoOut::samplePeriod (C++ function), 207
 ContinuousServoOut::sampleRate (C++ function), 207
 ContinuousServoOut::seconds (C++ function), 207
 ContinuousServoOut::setIsActive (C++ function), 207
 ContinuousServoOut::stop (C++ function), 206

D

DigitalIn (C++ class), 105
 DigitalIn::analogToDigital (C++ function), 108
 DigitalIn::changed (C++ function), 106
 DigitalIn::changeState (C++ function), 106
 DigitalIn::cutoff (C++ function), 108
 DigitalIn::debounce (C++ function), 107
 DigitalIn::debounceMode (C++ function), 107
 DigitalIn::DigitalIn (C++ function), 105
 DigitalIn::digitalToAnalog (C++ function), 108
 DigitalIn::fell (C++ function), 106
 DigitalIn::get (C++ function), 106
 DigitalIn::getInt (C++ function), 106
 DigitalIn::infiniteTimeWindow (C++ function), 108
 DigitalIn::isOff (C++ function), 106
 DigitalIn::isOn (C++ function), 106
 DigitalIn::mapTo (C++ function), 107
 DigitalIn::microSeconds (C++ function), 107
 DigitalIn::milliSeconds (C++ function), 107
 DigitalIn::mode (C++ function), 105, 107
 DigitalIn::noDebounce (C++ function), 107
 DigitalIn::noSmooth (C++ function), 107
 DigitalIn::noTimeWindow (C++ function), 108
 DigitalIn::nSteps (C++ function), 107
 DigitalIn::off (C++ function), 106
 DigitalIn::on (C++ function), 106
 DigitalIn::onChange (C++ function), 106
 DigitalIn::onFall (C++ function), 106
 DigitalIn::onRise (C++ function), 106
 DigitalIn::operator bool (C++ function), 107
 DigitalIn::pin (C++ function), 107
 DigitalIn::put (C++ function), 106
 DigitalIn::putOn (C++ function), 106
 DigitalIn::rawRead (C++ function), 105
 DigitalIn::read (C++ function), 105

DigitalIn::rose (C++ function), 106
 DigitalIn::samplePeriod (C++ function), 107
 DigitalIn::sampleRate (C++ function), 107
 DigitalIn::seconds (C++ function), 107
 DigitalIn::smooth (C++ function), 107
 DigitalIn::timeWindow (C++ function), 108
 DigitalIn::timeWindowIsInfinite (C++ function), 108
 DigitalIn::toggle (C++ function), 106
 DigitalOut (C++ class), 109
 DigitalOut::analogToDigital (C++ function), 111
 DigitalOut::changed (C++ function), 110
 DigitalOut::changeState (C++ function), 110
 DigitalOut::DigitalOut (C++ function), 109
 DigitalOut::digitalToAnalog (C++ function), 111
 DigitalOut::fell (C++ function), 110
 DigitalOut::get (C++ function), 110
 DigitalOut::getInt (C++ function), 110
 DigitalOut::isOff (C++ function), 110
 DigitalOut::isOn (C++ function), 109
 DigitalOut::mapTo (C++ function), 110
 DigitalOut::microSeconds (C++ function), 111
 DigitalOut::milliSeconds (C++ function), 111
 DigitalOut::mode (C++ function), 109, 111
 DigitalOut::nSteps (C++ function), 111
 DigitalOut::off (C++ function), 110
 DigitalOut::on (C++ function), 110
 DigitalOut::onChange (C++ function), 110
 DigitalOut::onFall (C++ function), 110
 DigitalOut::onRise (C++ function), 110
 DigitalOut::operator bool (C++ function), 110
 DigitalOut::pin (C++ function), 111
 DigitalOut::put (C++ function), 110
 DigitalOut::putOn (C++ function), 109
 DigitalOut::rawWrite (C++ function), 109
 DigitalOut::rose (C++ function), 110
 DigitalOut::samplePeriod (C++ function), 111
 DigitalOut::sampleRate (C++ function), 111
 DigitalOut::seconds (C++ function), 111
 DigitalOut::toggle (C++ function), 110
 DigitalOut::write (C++ function), 109

E

Engine (C++ class), 195
 Engine::autoSampleRate (C++ function), 196
 Engine::begin (C++ function), 195
 Engine::deltaTimeMicroSeconds (C++ function), 196
 Engine::deltaTimeSecondsTimesFixed32Max (C++ function), 197
 Engine::end (C++ function), 195
 Engine::hasAutoSampleRate (C++ function), 196
 Engine::isPrimary (C++ function), 197
 Engine::microSeconds (C++ function), 196

Engine::milliSeconds (C++ function), 196
 Engine::nSteps (C++ function), 196
 Engine::nUnits (C++ function), 195
 Engine::postBegin (C++ function), 195
 Engine::preBegin (C++ function), 195
 Engine::preStep (C++ function), 195
 Engine::primary (C++ function), 197
 Engine::randomTrigger (C++ function), 197
 Engine::referenceClock (C++ function), 197
 Engine::samplePeriod (C++ function), 196
 Engine::sampleRate (C++ function), 196
 Engine::seconds (C++ function), 196
 Engine::step (C++ function), 195
 Engine::timeStep (C++ function), 195

M

Metronome (C++ class), 143
 Metronome::addTime (C++ function), 147
 Metronome::analogToDigital (C++ function), 147
 Metronome::Bpm (C++ function), 145
 Metronome::bpm (C++ function), 145
 Metronome::digitalToAnalog (C++ function), 147
 Metronome::forward (C++ function), 147
 Metronome::Frequency (C++ function), 145
 Metronome::frequency (C++ function), 145
 Metronome::get (C++ function), 144
 Metronome::getInt (C++ function), 144
 Metronome::isForward (C++ function), 147
 Metronome::isOff (C++ function), 144
 Metronome::isOn (C++ function), 144
 Metronome::isRunning (C++ function), 147
 Metronome::Jitter (C++ function), 146
 Metronome::jitter (C++ function), 146
 Metronome::jitteredFrequency (C++ function), 146
 Metronome::jitteredPeriod (C++ function), 146
 Metronome::mapTo (C++ function), 144
 Metronome::Metronome (C++ function), 143
 Metronome::microSeconds (C++ function), 144
 Metronome::milliSeconds (C++ function), 144
 Metronome::noJitter (C++ function), 146
 Metronome::nSteps (C++ function), 144
 Metronome::off (C++ function), 144
 Metronome::on (C++ function), 144
 Metronome::onBang (C++ function), 144
 Metronome::operator bool (C++ function), 144
 Metronome::pause (C++ function), 147
 Metronome::Period (C++ function), 145
 Metronome::period (C++ function), 145
 Metronome::Phase (C++ function), 145
 Metronome::phase (C++ function), 145
 Metronome::PhaseShift (C++ function), 146
 Metronome::phaseShift (C++ function), 145, 146
 Metronome::put (C++ function), 144
 Metronome::putOn (C++ function), 144

Metronome::resume (C++ function), 147
 Metronome::reverse (C++ function), 147
 Metronome::samplePeriod (C++ function), 145
 Metronome::sampleRate (C++ function), 145
 Metronome::seconds (C++ function), 144
 Metronome::setForward (C++ function), 147
 Metronome::setTime (C++ function), 146
 Metronome::start (C++ function), 145
 Metronome::stop (C++ function), 147
 Metronome::timeToPhase (C++ function), 146
 Metronome::togglePause (C++ function), 147
 Metronome::toggleReverse (C++ function), 147
 MinMaxScaler (C++ class), 149
 MinMaxScaler::analogToDigital (C++ function), 151
 MinMaxScaler::cutoff (C++ function), 151
 MinMaxScaler::digitalToAnalog (C++ function), 151
 MinMaxScaler::filter (C++ function), 150
 MinMaxScaler::get (C++ function), 150
 MinMaxScaler::infiniteTimeWindow (C++ function), 151
 MinMaxScaler::isCalibrating (C++ function), 150
 MinMaxScaler::isPreInitialized (C++ function), 150
 MinMaxScaler::mapTo (C++ function), 150
 MinMaxScaler::maxValue (C++ function), 150
 MinMaxScaler::microSeconds (C++ function), 151
 MinMaxScaler::milliSeconds (C++ function), 150
 MinMaxScaler::MinMaxScaler (C++ function), 149
 MinMaxScaler::minValue (C++ function), 150
 MinMaxScaler::noTimeWindow (C++ function), 151
 MinMaxScaler::nSamples (C++ function), 150
 MinMaxScaler::nSteps (C++ function), 151
 MinMaxScaler::operator bool (C++ function), 151
 MinMaxScaler::operator float (C++ function), 151
 MinMaxScaler::pauseCalibrating (C++ function), 150
 MinMaxScaler::put (C++ function), 150
 MinMaxScaler::reset (C++ function), 150
 MinMaxScaler::resumeCalibrating (C++ function), 150
 MinMaxScaler::samplePeriod (C++ function), 151
 MinMaxScaler::sampleRate (C++ function), 151
 MinMaxScaler::seconds (C++ function), 150
 MinMaxScaler::timeWindow (C++ function), 151
 MinMaxScaler::timeWindowIsInfinite (C++ function), 151
 MinMaxScaler::toggleCalibrating (C++ function), 150
 Monitor (C++ class), 172
 Monitor::analogToDigital (C++ function), 173
 Monitor::device (C++ function), 172
 Monitor::digitalToAnalog (C++ function), 173

Monitor::get (C++ function), 173
 Monitor::mapTo (C++ function), 173
 Monitor::microSeconds (C++ function), 173
 Monitor::milliSeconds (C++ function), 173
 Monitor::Monitor (C++ function), 172
 Monitor::nSteps (C++ function), 173
 Monitor::operator bool (C++ function), 173
 Monitor::operator float (C++ function), 173
 Monitor::precision (C++ function), 172
 Monitor::put (C++ function), 172
 Monitor::samplePeriod (C++ function), 173
 Monitor::sampleRate (C++ function), 173
 Monitor::seconds (C++ function), 173
 Monitor::write (C++ function), 173

N

Normalizer (C++ class), 152
 Normalizer::analogToDigital (C++ function), 157
 Normalizer::clamp (C++ function), 154
 Normalizer::cutoff (C++ function), 155
 Normalizer::digitalToAnalog (C++ function), 157
 Normalizer::filter (C++ function), 154
 Normalizer::get (C++ function), 155
 Normalizer::highOutlierThreshold (C++ function), 154
 Normalizer::infiniteTimeWindow (C++ function), 155
 Normalizer::isCalibrating (C++ function), 155
 Normalizer::isClamped (C++ function), 154
 Normalizer::isHighOutlier (C++ function), 156
 Normalizer::isLowOutlier (C++ function), 156
 Normalizer::isOutlier (C++ function), 156
 Normalizer::isPreInitialized (C++ function), 155
 Normalizer::lowOutlierThreshold (C++ function), 154
 Normalizer::mapTo (C++ function), 154
 Normalizer::mean (C++ function), 156
 Normalizer::meanSquared (C++ function), 156
 Normalizer::microSeconds (C++ function), 155
 Normalizer::milliSeconds (C++ function), 155
 Normalizer::noClamp (C++ function), 154
 Normalizer::normalize (C++ function), 156
 Normalizer::Normalizer (C++ function), 153
 Normalizer::noTimeWindow (C++ function), 155
 Normalizer::nSamples (C++ function), 155
 Normalizer::nSteps (C++ function), 155
 Normalizer::operator bool (C++ function), 155
 Normalizer::operator float (C++ function), 155
 Normalizer::pauseCalibrating (C++ function), 154
 Normalizer::put (C++ function), 154
 Normalizer::reset (C++ function), 153, 154
 Normalizer::resumeCalibrating (C++ function), 154
 Normalizer::samplePeriod (C++ function), 155

Normalizer::sampleRate (C++ function), 155
 Normalizer::seconds (C++ function), 155
 Normalizer::stdDev (C++ function), 156
 Normalizer::stddev (C++ function), 156
 Normalizer::targetMean (C++ function), 153
 Normalizer::targetStdDev (C++ function), 153
 Normalizer::timeWindow (C++ function), 155
 Normalizer::timeWindowIsInfinite (C++ function), 155
 Normalizer::toggleCalibrating (C++ function), 154
 Normalizer::update (C++ function), 156
 Normalizer::var (C++ function), 156

P

PeakDetector (C++ class), 160
 PeakDetector::analogToDigital (C++ function), 162
 PeakDetector::digitalToAnalog (C++ function), 162
 PeakDetector::fallbackTolerance (C++ function), 160
 PeakDetector::get (C++ function), 161
 PeakDetector::getInt (C++ function), 161
 PeakDetector::isOff (C++ function), 161
 PeakDetector::isOn (C++ function), 161
 PeakDetector::mapTo (C++ function), 161
 PeakDetector::microSeconds (C++ function), 162
 PeakDetector::milliSeconds (C++ function), 162
 PeakDetector::mode (C++ function), 161
 PeakDetector::modeApex (C++ function), 161
 PeakDetector::modeCrossing (C++ function), 161
 PeakDetector::modeInverted (C++ function), 161
 PeakDetector::nSteps (C++ function), 162
 PeakDetector::off (C++ function), 161
 PeakDetector::on (C++ function), 161
 PeakDetector::onBang (C++ function), 161
 PeakDetector::operator bool (C++ function), 161
 PeakDetector::PeakDetector (C++ function), 160
 PeakDetector::put (C++ function), 161
 PeakDetector::putOn (C++ function), 161
 PeakDetector::reloadThreshold (C++ function), 160
 PeakDetector::samplePeriod (C++ function), 162
 PeakDetector::sampleRate (C++ function), 162
 PeakDetector::seconds (C++ function), 162
 PeakDetector::triggerThreshold (C++ function), 160
 PivotField (C++ class), 178
 PivotField::~~PivotField (C++ function), 178
 PivotField::analogToDigital (C++ function), 180
 PivotField::at (C++ function), 178
 PivotField::bumpWidth (C++ function), 179
 PivotField::center (C++ function), 179
 PivotField::clearEvents (C++ function), 180
 PivotField::digitalToAnalog (C++ function), 180
 PivotField::easing (C++ function), 179
 PivotField::get (C++ function), 178
 PivotField::mapTo (C++ function), 180
 PivotField::microSeconds (C++ function), 180
 PivotField::milliSeconds (C++ function), 180
 PivotField::mode (C++ function), 178, 179
 PivotField::noEasing (C++ function), 179
 PivotField::noRampWidth (C++ function), 179
 PivotField::nSteps (C++ function), 180
 PivotField::operator bool (C++ function), 180
 PivotField::operator float (C++ function), 180
 PivotField::PivotField (C++ function), 178
 PivotField::populate (C++ function), 179
 PivotField::put (C++ function), 178
 PivotField::rampShift (C++ function), 179
 PivotField::rampWidth (C++ function), 179
 PivotField::samplePeriod (C++ function), 180
 PivotField::sampleRate (C++ function), 180
 PivotField::seconds (C++ function), 180
 Plotter (C++ class), 175
 Plotter::analogToDigital (C++ function), 176
 Plotter::beginPlot (C++ function), 175
 Plotter::digitalToAnalog (C++ function), 176
 Plotter::endPlot (C++ function), 175
 Plotter::format (C++ function), 175
 Plotter::formatFromPreset (C++ function), 176
 Plotter::get (C++ function), 175
 Plotter::labels (C++ function), 175
 Plotter::mapTo (C++ function), 176
 Plotter::microSeconds (C++ function), 175
 Plotter::milliSeconds (C++ function), 175
 Plotter::nSteps (C++ function), 175
 Plotter::operator bool (C++ function), 176
 Plotter::operator float (C++ function), 176
 Plotter::precision (C++ function), 175
 Plotter::put (C++ function), 175
 Plotter::samplePeriod (C++ function), 176
 Plotter::sampleRate (C++ function), 175
 Plotter::seconds (C++ function), 175
 pq::mapFloat (C++ function), 186
 pq::mapFrom01 (C++ function), 187
 pq::mapTo01 (C++ function), 188
 pq::randomFloat (C++ function), 189
 pq::randomTrigger (C++ function), 190
 pq::seconds (C++ function), 191
 pq::wrap (C++ function), 192
 pq::wrap01 (C++ function), 192

R

Ramp (C++ class), 117
 Ramp::addTime (C++ function), 121
 Ramp::analogToDigital (C++ function), 121

Ramp::digitalToAnalog (C++ function), 121
 Ramp::Duration (C++ function), 120
 Ramp::duration (C++ function), 118
 Ramp::durationToSpeed (C++ function), 119
 Ramp::easing (C++ function), 118
 Ramp::elapsed (C++ function), 120
 Ramp::finished (C++ function), 119
 Ramp::from (C++ function), 118
 Ramp::fromTo (C++ function), 118
 Ramp::get (C++ function), 117
 Ramp::go (C++ function), 119
 Ramp::hasPassed (C++ function), 120
 Ramp::isComplete (C++ function), 120
 Ramp::isFinished (C++ function), 120
 Ramp::isRunning (C++ function), 121
 Ramp::isStarted (C++ function), 121
 Ramp::mapTo (C++ function), 118
 Ramp::microSeconds (C++ function), 120
 Ramp::milliSeconds (C++ function), 120
 Ramp::mode (C++ function), 119
 Ramp::noEasing (C++ function), 118
 Ramp::nSteps (C++ function), 120
 Ramp::onFinish (C++ function), 119
 Ramp::operator bool (C++ function), 120
 Ramp::operator float (C++ function), 120
 Ramp::pause (C++ function), 120
 Ramp::progress (C++ function), 120
 Ramp::put (C++ function), 117
 Ramp::Ramp (C++ function), 117
 Ramp::resume (C++ function), 120
 Ramp::samplePeriod (C++ function), 120
 Ramp::sampleRate (C++ function), 120
 Ramp::seconds (C++ function), 120
 Ramp::setTime (C++ function), 119
 Ramp::Speed (C++ function), 118
 Ramp::speed (C++ function), 118
 Ramp::speedToDuration (C++ function), 119
 Ramp::start (C++ function), 119, 120
 Ramp::stop (C++ function), 121
 Ramp::to (C++ function), 118
 Ramp::togglePause (C++ function), 121
 RobustScaler (C++ class), 165
 RobustScaler::analogToDigital (C++ function), 167
 RobustScaler::cutoff (C++ function), 167
 RobustScaler::digitalToAnalog (C++ function), 167
 RobustScaler::filter (C++ function), 166
 RobustScaler::get (C++ function), 166
 RobustScaler::highQuantile (C++ function), 165
 RobustScaler::highQuantileLevel (C++ function), 165
 RobustScaler::infiniteTimeWindow (C++ function), 166

RobustScaler::isCalibrating (C++ function), 166
 RobustScaler::isPreInitialized (C++ function), 166
 RobustScaler::lowQuantile (C++ function), 165
 RobustScaler::lowQuantileLevel (C++ function), 165
 RobustScaler::mapTo (C++ function), 166
 RobustScaler::microSeconds (C++ function), 166
 RobustScaler::milliSeconds (C++ function), 166
 RobustScaler::noTimeWindow (C++ function), 167
 RobustScaler::nSamples (C++ function), 166
 RobustScaler::nSteps (C++ function), 166
 RobustScaler::operator bool (C++ function), 166
 RobustScaler::operator float (C++ function), 166
 RobustScaler::pauseCalibrating (C++ function), 166
 RobustScaler::put (C++ function), 166
 RobustScaler::reset (C++ function), 165
 RobustScaler::resumeCalibrating (C++ function), 166
 RobustScaler::RobustScaler (C++ function), 165
 RobustScaler::samplePeriod (C++ function), 166
 RobustScaler::sampleRate (C++ function), 166
 RobustScaler::seconds (C++ function), 166
 RobustScaler::span (C++ function), 165
 RobustScaler::stdDev (C++ function), 165
 RobustScaler::timeWindow (C++ function), 167
 RobustScaler::timeWindowIsInfinite (C++ function), 167
 RobustScaler::toggleCalibrating (C++ function), 166

S

ServoOut (C++ class), 208
 ServoOut::activate (C++ function), 209
 ServoOut::analogToDigital (C++ function), 210
 ServoOut::deactivate (C++ function), 209
 ServoOut::digitalToAnalog (C++ function), 210
 ServoOut::get (C++ function), 209
 ServoOut::getAngle (C++ function), 209
 ServoOut::isActive (C++ function), 209
 ServoOut::mapTo (C++ function), 209
 ServoOut::microSeconds (C++ function), 209
 ServoOut::milliSeconds (C++ function), 209
 ServoOut::nSteps (C++ function), 209
 ServoOut::operator bool (C++ function), 210
 ServoOut::operator float (C++ function), 210
 ServoOut::pin (C++ function), 209
 ServoOut::put (C++ function), 209
 ServoOut::putAngle (C++ function), 209
 ServoOut::samplePeriod (C++ function), 210
 ServoOut::sampleRate (C++ function), 210
 ServoOut::seconds (C++ function), 209
 ServoOut::ServoOut (C++ function), 209

ServoOut::setIsActive (C++ function), 209
 Smoother (C++ class), 169
 Smoother::analogToDigital (C++ function), 171
 Smoother::cutoff (C++ function), 170
 Smoother::digitalToAnalog (C++ function), 171
 Smoother::filter (C++ function), 169
 Smoother::get (C++ function), 170
 Smoother::infiniteTimeWindow (C++ function), 170
 Smoother::isCalibrating (C++ function), 169
 Smoother::isPreInitialized (C++ function), 170
 Smoother::mapTo (C++ function), 170
 Smoother::microSeconds (C++ function), 170
 Smoother::milliSeconds (C++ function), 170
 Smoother::noTimeWindow (C++ function), 170
 Smoother::nSamples (C++ function), 169
 Smoother::nSteps (C++ function), 170
 Smoother::operator bool (C++ function), 170
 Smoother::operator float (C++ function), 170
 Smoother::pauseCalibrating (C++ function), 169
 Smoother::put (C++ function), 169
 Smoother::reset (C++ function), 169
 Smoother::resumeCalibrating (C++ function), 169
 Smoother::samplePeriod (C++ function), 170
 Smoother::sampleRate (C++ function), 170
 Smoother::seconds (C++ function), 170
 Smoother::Smoother (C++ function), 169
 Smoother::timeWindow (C++ function), 170
 Smoother::timeWindowIsInfinite (C++ function), 170
 Smoother::toggleCalibrating (C++ function), 169

T

TimeSliceField (C++ class), 182
 TimeSliceField::~~TimeSliceField (C++ function), 182
 TimeSliceField::analogToDigital (C++ function), 184
 TimeSliceField::at (C++ function), 182
 TimeSliceField::atIndex (C++ function), 183
 TimeSliceField::clearEvents (C++ function), 183
 TimeSliceField::count (C++ function), 183
 TimeSliceField::digitalToAnalog (C++ function), 184
 TimeSliceField::get (C++ function), 182
 TimeSliceField::isFull (C++ function), 183
 TimeSliceField::isRolling (C++ function), 183
 TimeSliceField::mapTo (C++ function), 184
 TimeSliceField::microSeconds (C++ function), 184
 TimeSliceField::milliSeconds (C++ function), 183
 TimeSliceField::noRolling (C++ function), 183
 TimeSliceField::nSteps (C++ function), 184
 TimeSliceField::onUpdate (C++ function), 183
 TimeSliceField::operator bool (C++ function), 184

TimeSliceField::operator float (C++ function), 184
 TimeSliceField::period (C++ function), 182, 183
 TimeSliceField::populate (C++ function), 183
 TimeSliceField::put (C++ function), 182
 TimeSliceField::reset (C++ function), 183
 TimeSliceField::rolling (C++ function), 183
 TimeSliceField::samplePeriod (C++ function), 184
 TimeSliceField::sampleRate (C++ function), 184
 TimeSliceField::seconds (C++ function), 183
 TimeSliceField::setRolling (C++ function), 183
 TimeSliceField::TimeSliceField (C++ function), 182
 TimeSliceField::updated (C++ function), 183

W

Wave (C++ class), 128
 Wave::addTime (C++ function), 133
 Wave::amplitude (C++ function), 129, 130
 Wave::analogToDigital (C++ function), 134
 Wave::atPhase (C++ function), 129
 Wave::Bpm (C++ function), 131
 Wave::bpm (C++ function), 131
 Wave::digitalToAnalog (C++ function), 134
 Wave::forward (C++ function), 133
 Wave::Frequency (C++ function), 131
 Wave::frequency (C++ function), 131
 Wave::get (C++ function), 129
 Wave::isForward (C++ function), 133
 Wave::isRunning (C++ function), 133
 Wave::Jitter (C++ function), 132
 Wave::jitter (C++ function), 132
 Wave::jitteredFrequency (C++ function), 132
 Wave::jitteredPeriod (C++ function), 132
 Wave::mapTo (C++ function), 130
 Wave::microSeconds (C++ function), 130
 Wave::milliSeconds (C++ function), 130
 Wave::noJitter (C++ function), 132
 Wave::nSteps (C++ function), 130
 Wave::onPassPeriod (C++ function), 130
 Wave::onPassSkew (C++ function), 130
 Wave::operator bool (C++ function), 131
 Wave::operator float (C++ function), 131
 Wave::passedPeriod (C++ function), 130
 Wave::passedSkew (C++ function), 130
 Wave::pause (C++ function), 133
 Wave::Period (C++ function), 131
 Wave::period (C++ function), 131
 Wave::Phase (C++ function), 132
 Wave::phase (C++ function), 132
 Wave::PhaseShift (C++ function), 132
 Wave::phaseShift (C++ function), 132
 Wave::put (C++ function), 131
 Wave::resume (C++ function), 133

Wave::reverse (*C++ function*), 133
Wave::samplePeriod (*C++ function*), 131
Wave::sampleRate (*C++ function*), 131
Wave::seconds (*C++ function*), 130
Wave::setForward (*C++ function*), 133
Wave::setTime (*C++ function*), 133
Wave::shape (*C++ function*), 129
Wave::shiftBy (*C++ function*), 129
Wave::shiftByTime (*C++ function*), 129
Wave::Skew (*C++ function*), 130
Wave::skew (*C++ function*), 130
Wave::start (*C++ function*), 131
Wave::stop (*C++ function*), 133
Wave::timeToPhase (*C++ function*), 133
Wave::togglePause (*C++ function*), 133
Wave::toggleReverse (*C++ function*), 133
Wave::Wave (*C++ function*), 128, 129
Wave::width (*C++ function*), 130